SISIG EH!

Sisig Developer Handbook

Every developer guide in the repository, in one file.

Generated 2026-11-07 with `pnpm docs:pdf`.
The Markdown files in the repository are the source of truth.

Contents

1. Sisig Platform README.md
2. Sisig Documentation docs/README.md
3. Local Development docs/LOCAL-DEVELOPMENT.md
4. Sisig Platform Architecture Guide docs/ARCHITECTURE.md
5. Supabase in Sisig docs/SUPABASE.md
6. Runbook: Remote Config, Kill Switches and Forced Update docs/runbooks/remote-config.md
7. Supabase database (DATA-01) supabase/README.md
8. Database conventions supabase/CONVENTIONS.md
9. DEVOPS-02: GitHub Actions CI Pipeline docs/DEVOPS-02-CI.md
10. Contributing to Sisig CONTRIBUTING.md
11. Sisig Mobile App apps/mobile/README.md
12. @sisig/claude packages/claude/README.md
13. Claude AI Skills Guide CLAUDE_SKILLS.md

Sisig Platform

Source: README.md

> Monorepo for the Sisig Eh! rewards app: a React Native (Expo SDK 51) mobile app, shared TypeScript packages, and a Supabase backend. pnpm workspaces, Turborepo, TypeScript strict mode.

Quick Start

You need a Mac with Node.js 18 or later, pnpm 9, JDK 17, the Android SDK and an Android emulator, plus an `apps/mobile/.env.local` file with the hosted Supabase URL and publishable key. docs/LOCAL-DEVELOPMENT.md walks through all of it, step by step.

With those in place, the first run is:

```
git clone https://github.com/Alexd2004/Sisig.git
cd Sisig
pnpm install
```

```
pnpm turbo run build --filter='@sisig/mobile^...'
```

```
pnpm -F @sisig/mobile exec expo run:android
```

The first block clones the repo and installs dependencies. The second builds the workspace packages, which the app imports from their `dist/` folders. The third builds the development client, installs it on the running emulator, and starts Metro. After that, the daily command is:

```
pnpm -F @sisig/mobile exec expo start --dev-client --android
```

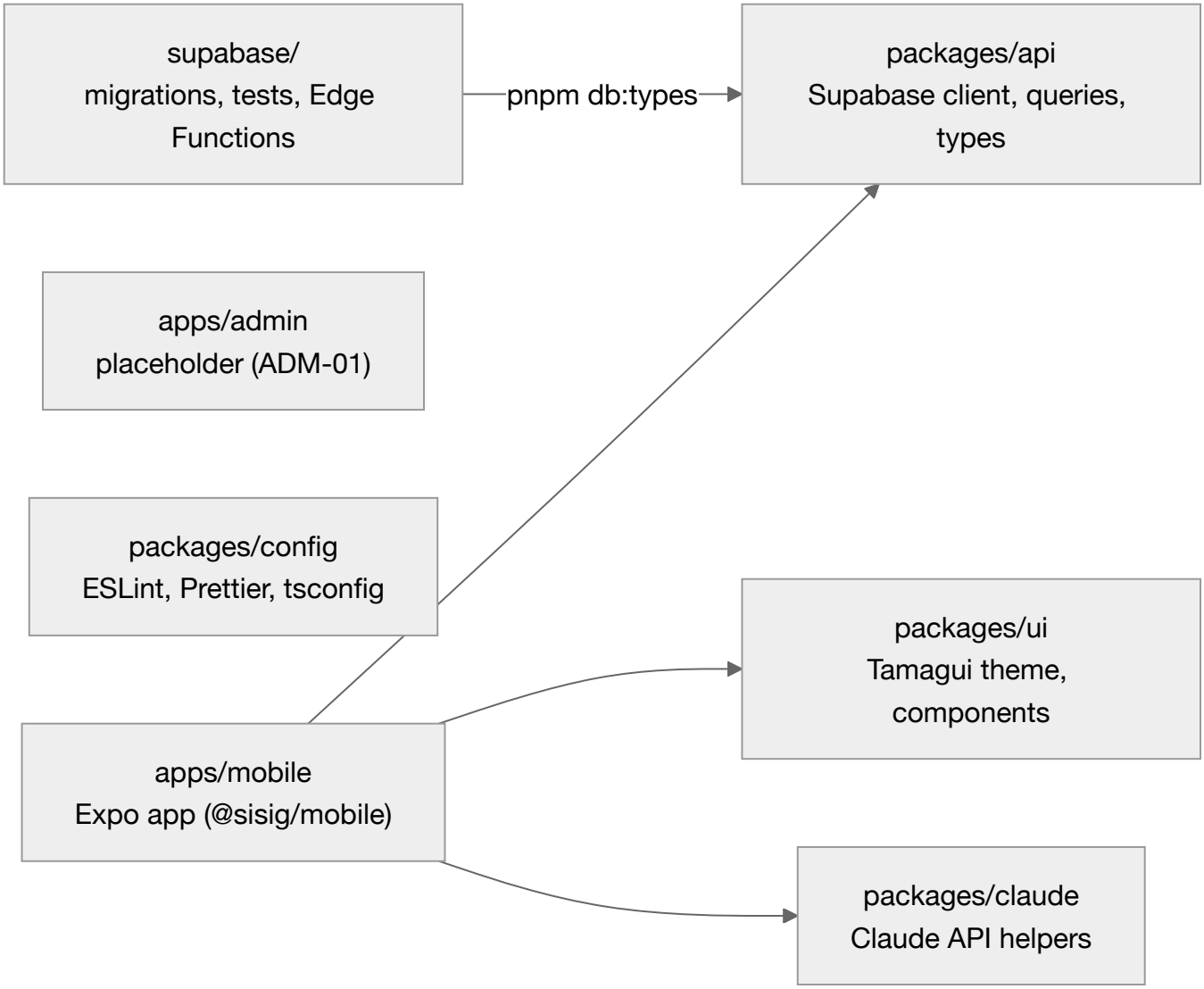
Expo Go cannot run this app. iOS is not set up yet (it needs the full Xcode app).

Documentation

docs/README.md lists every doc, what it covers and who it is for, and explains the PDF versions in `docs/pdf/`. The ones most people need:

- docs/LOCAL-DEVELOPMENT.md: set up a Mac and run the app.
- CONTRIBUTING.md: branches, commits, pull requests, database changes.
- docs/ARCHITECTURE.md: how the system fits together.
- docs/SUPABASE.md: the hosted Supabase project and applying migrations.
- docs/DEVOPS-02-CI.md: the CI checks.

Repo Map



Arrows from the app are runtime imports; the app loads each package from its built `dist/` folder. `pnpm db:types` generates `packages/api/src/database.types.ts` from a local database built from the migrations. `packages/config` holds the lint, format and TypeScript config every workspace extends.

```
Sisig/
├── apps/
│   ├── mobile/          # Expo app: app/ (Expo Router routes), src/ (components, lib, session,
navigation)
│   └── admin/           # Placeholder scripts only; framework chosen in ADM-01
├── packages/
│   ├── api/             # @sisig/api: Supabase client, query keys and hooks, generated database
types
│   ├── ui/              # @sisig/ui: Tamagui theme, brand tokens, Button, Text, TextInput
```

```
│   ├── claude/          # @sisig/claude: Claude API client and helpers
│   └── config/          # @sisig/config: shared ESLint, Prettier and TypeScript config
├── supabase/            # config.toml, migrations/, tests/database/ (pgTAP), functions/ (Edge
Functions), seed.sql
├── docs/                # Developer guides (index: docs/README.md)
├── .github/             # CI workflows, CODEOWNERS, PR template
├── package.json         # Root scripts and pnpm workspaces
├── turbo.json           # Turborepo task pipeline and cache outputs
└── CONTRIBUTING.md      # Branching, commits, pull requests, releases
```

Key Commands

Run these from the repo root.

Command	What it does
`pnpm install`	Install dependencies for every workspace
`pnpm turbo run build --filter='@sisig/mobile^...'`	Build the packages the mobile app depends on (into `dist/`)
`pnpm -F @sisig/mobile exec expo run:android`	Build and install the Android dev client, then start Metro
`pnpm -F @sisig/mobile exec expo start --dev-client --android`	Start Metro and open the installed dev client
`pnpm lint`	ESLint in every workspace that has the script (`apps/admin` is a placeholder)
`pnpm typecheck`	`tsc --noEmit` in every workspace that has the script (`apps/admin` is a placeholder); runs after the package builds it depends on
`pnpm test`	Jest in every workspace that has the script (`apps/admin` is a placeholder)
`pnpm format`	Prettier, writing changes
`pnpm format:check`	Prettier check without changes (fails today; see Known Issues)
`pnpm -F @sisig/mobile run doctor`	Run `expo-doctor` on the mobile app
`pnpm db:start`, `pnpm db:reset`, `pnpm db:test`, `pnpm db:lint`, `pnpm db:types`, `pnpm db:new <name>`	Local Supabase commands (need Docker); see supabase/README.md
`pnpm docs:pdf`	Regenerate the PDFs in `docs/pdf/`; see docs/README.md

Notes:

- `pnpm build` is not usable yet. It also runs the mobile app's `build` script (`expo build`), which the local Expo CLI rejects. Release builds need EAS, which is not set up (DEVOPS-04).
- Plain `pnpm doctor` runs pnpm's own checker, not `expo-doctor`, because `doctor` is a built-in pnpm command. Use `run doctor` as shown.
- `pnpm test --watch` does not work at the root: Turborepo rejects it (`ERROR unexpected argument '--watch' found`). Pass it to Jest in one package: `pnpm -F @sisig/mobile test -- --watch`.

Development Client

The app runs in its own development build, made with `expo-dev-client` (listed under `plugins` in `apps/mobile/app.json`). This is the team's chosen workflow; Expo Go is not supported. Some native packages are declared in `apps/mobile/package.json` (`@sentry/react-native`, `lottie-react-native`, `react-native-gesture-handler`, `expo-device`) but no code imports them yet.

- Build and install it with `expo run:android`. Rebuild it only when native code changes: a dependency with native code, or `apps/mobile/app.json`.
- `expo run:android` generates `apps/mobile/android/` and adds `android` and `ios` scripts to `apps/mobile/package.json`. These are not git-ignored yet. Do not commit them.
- `pnpm -F @sisig/mobile exec expo run:ios` needs the full Xcode app and is not set up on the team's Mac.

Details and troubleshooting: docs/LOCAL-DEVELOPMENT.md.

TypeScript Strict Mode

Every TypeScript workspace extends `packages/config/tsconfig.json`, which sets `strict`, `noImplicitAny`, `strictNullChecks`, `noUnusedLocals`, `noUnusedParameters`, `noImplicitReturns` and related flags. `pnpm typecheck` must pass with zero errors. `apps/admin` has placeholder scripts only, so it has no TypeScript yet.

Testing

Jest runs in each workspace, with React Native Testing Library for components. Examples:

- `packages/ui/src/Button.test.tsx`: component test
- `packages/ui/src/theme/contrast.test.ts`: WCAG AA contrast for every semantic colour pair
- `packages/api/src/hooks/useMenu.test.ts`: query options test
- `apps/mobile/src/navigation/index-redirect.test.tsx`: screen test for `app/index.tsx`

Keep test files out of `apps/mobile/app/`. Expo Router turns every file in that folder into a route, so a test there is bundled as a screen and crashes the app at launch (fixed in PR #18).

Database tests (pgTAP) live in `supabase/tests/database/` and run with `pnpm db:test`, which needs Docker. CI runs them on every PR.

Theme and Brand Tokens (THEME-01)

- **Brand values** live in one file: `packages/ui/src/theme/brand.ts`. Components never use raw hex colours. They use semantic names through `@sisig/ui` (`useAppColors`, `useThemedStyles`, `Text`, `TextInput`). `apps/mobile/src/no-raw-colors.test.ts` and `packages/ui/src/no-raw-colors.test.ts` fail if a component contains a hex literal.
- **Light and dark** follow the OS setting (`userInterfaceStyle: "automatic"` in `apps/mobile/app.json`). There is no in-app override in v1.
- **Font scaling** is capped at `FONT_SCALE_CAP` (1.5x) in `Text` and `TextInput`.
- **Contrast:** `packages/ui/src/theme/contrast.test.ts` checks WCAG AA (4.5:1) for every foreground/background pair in both themes. It runs under `pnpm test` in CI.
- **Rebrand:** edit `brand.ts`, then run `pnpm test`. The contrast test catches any pair that falls below AA. See docs/ARCHITECTURE.md ("Rebrand in 10 Minutes").

Pending client assets

The values below are placeholders until the client supplies final assets. Do not treat them as brand decisions.

Asset	Current placeholder	Where to change
Brand colour palette (light and dark)	Placeholder palette tuned to pass WCAG AA. It is not the brand palette.	`packages/ui/src/theme/brand.ts` (`palette`, `lightTheme`, `darkTheme`)
Brand fonts (body and heading)	Inter from Google Fonts via `@expo-google-fonts/inter` (Regular 400 and SemiBold 600). Package metadata lists `MIT AND OFL-1.1`. Confirm the licence before any release.	`brandFonts` in `brand.ts`; the `expo-font` plugin entry in `apps/mobile/app.json`; the imports in `apps/mobile/app/_layout.tsx`
Licensed font files and final brand fonts	Not yet received from the client	Client input. Replace the placeholder fonts when they arrive.
App icon, adaptive icon, splash and favicon	Solid placeholder images (PR #17). Final artwork comes with DSGN-01.	`apps/mobile/assets/`

Continuous Integration (DEVOPS-02)

Every pull request into `main`, and every push to `main`, runs two workflows in `.github/workflows/`:

Check (job)	What it runs
`lint-and-test`	ESLint, `tsc --noEmit`, Jest, `expo install --check`, package build, Android `expo export` bundle
`database`	Local Supabase start (migrations and seed), pgTAP tests, `supabase db lint`

Check (job)	What it runs
`types-drift`	`pnpm db:types` ; fails if `packages/api/src/database.types.ts` changes
`secrets`	gitleaks, plus greps for JWTs, service-role key references and Anthropic keys in `apps/` and `packages/`
`deno-tests`	Edge Function tests, `deno fmt --check` , `deno lint`

Branch protection on `main` is not configured yet, so GitHub does not block a merge when a check fails. The team rule is to merge only when all five pass. Job details, local commands and fixes for common failures are in docs/DEVOPS-02-CI.md.

Branching and Code Review

See CONTRIBUTING.md for trunk-based development, branch names, Conventional Commits with the Jira key as scope, the PR process, database changes and releases.

`.github/CODEOWNERS` routes review requests by area. Every area currently points at @Alexd2004, because the DEV-A, DEV-B and DEV-C team handles do not exist yet. Any developer may approve a PR.

Known Issues

Checked on 2026-11-06:

- `pnpm format:check` **fails** in `packages/api` , `packages/claude` , `packages/ui` and `apps/mobile` . It is not run in CI. See docs/DEVOPS-02-CI.md.
- `expo-doctor` **fails three checks** on a clean clone (`pnpm -F @sisig/mobile run doctor`):
  - `app.json` : `ios.supportsTabletMode` is not an allowed property.
  - `@types/react-native` is installed directly, but `react-native` ships its own types.
  - Missing peer dependency `expo-application` , required by `sentry-expo` .

  After `expo run:android` , it also warns about the generated native folders. That warning is local only.
- **Prebuild output is not git-ignored.** `apps/mobile/android/` , `apps/mobile/ios/` and the scripts prebuild adds to `apps/mobile/package.json` show up in `git status` .
- **iOS is not verified.** The Android dev client was built and run on an emulator on 2026-11-06. `expo run:ios` has not been run, because the team's Mac has no full Xcode.
- **Root scripts use the deprecated** `--parallel` **flag.** `dev` , `lint` , `typecheck` , `test` and the format scripts in the root `package.json` pass it to Turborepo. Turborepo 2.11.10 still ran the `^build` dependencies first in a no-cache timing check, but its docs do not guarantee that, so build the packages before starting Metro.
- **Branch protection is not configured** on `main` (see CONTRIBUTING.md).

Troubleshooting

Setup and runtime problems (Metro cannot resolve a package, old bundle, emulator offline, Java, disk space) are covered in docs/LOCAL-DEVELOPMENT.md. CI failures are covered in docs/DEVOPS-02-CI.md.

Further Reading

- Expo Router documentation
- Expo development builds
- Turborepo task caching
- pnpm workspaces
- React Native Testing Library
- Supabase CLI

Sisig Documentation

Source: docs/README.md

This is the index of the developer docs for the Sisig Eh! rewards app monorepo. Start with LOCAL-DEVELOPMENT.md if you are setting up a Mac for the first time, then read CONTRIBUTING.md before your first pull request.

All Docs

Doc	What it covers	Who it's for
README.md	Repo front page: what is in the repo, quick start, repo map, key commands, known issues	Everyone
CONTRIBUTING.md	Branch names, commit messages, the PR process, required checks, database changes, data layer rules, releases	Everyone who opens a pull request
docs/LOCAL-DEVELOPMENT.md	Mac setup from a clean clone: JDK 17, Android SDK, emulator, dev client, hosted Supabase; the daily routine; troubleshooting	New developers; anyone running the app
docs/ARCHITECTURE.md	System design: database, navigation, theme and data layer, and the decisions behind them	Developers changing how parts fit together; reviewers
docs/SUPABASE.md	The hosted Supabase dev project: CLI login and link, applying migrations with `supabase db push`, destructive migrations	Anyone who changes the database or applies migrations
docs/runbooks/remote-config.md	Runbook: switch off ordering or receipt scanning, force or recommend an app update, show an announcement, with the SQL for each	Whoever handles a POS outage, a critical bug or a release
docs/DEVOPS-02-CI.md	The GitHub Actions workflows: triggers, jobs, steps, caches, branch protection, CI failures	Anyone whose PR fails CI; repo admins
supabase/README.md	The local Supabase stack: `db:*` commands, folder layout, schema, migrations, generated types, tests, Edge Functions	Developers working on the database
supabase/CONVENTIONS.md	Rules for migrations, naming, schemas, policies, `SECURITY DEFINER` functions, Edge Functions and database tests	Database authors and reviewers
apps/mobile/README.md	The Expo mobile app: folder structure, routing, scripts, tests	Mobile developers
packages/claude/README.md	The `@sisig/claude` package: setup, API reference, usage	Developers using the Claude integration
CLAUDE_SKILLS.md	The Claude-powered features in the app and packages	Developers working on AI features

PDF Versions

PDF copies of the docs live in docs/pdf/:

File	Contents
`Sisig-Developer-Handbook.pdf`	All of the guides above in one file, with a table of contents
`SUPABASE.pdf`	docs/SUPABASE.md
`ARCHITECTURE.pdf`	docs/ARCHITECTURE.md
`LOCAL-DEVELOPMENT.pdf`	docs/LOCAL-DEVELOPMENT.md
`DEVOPS-02-CI.pdf`	docs/DEVOPS-02-CI.md
`CONTRIBUTING.pdf`	CONTRIBUTING.md

The Markdown files are the source of truth. The PDFs are generated from them by `scripts/docs-pdf.mjs`:

```
pnpm docs:pdf
```

The script needs Google Chrome (or Chromium) installed and network access, because it loads the Markdown and Mermaid renderers from jsDelivr. Set `CHROME_PATH` if your browser is not in a standard location. To check that every diagram renders without writing any PDFs:

```
pnpm docs:pdf --check
```

When a PR changes a doc, regenerate the PDFs in the same PR so they never drift from the Markdown.

Keeping Docs Current

Pull requests are squash merged. The squash commit on `main` keeps the PR title (or the commit title, for a single-commit PR) and the messages of the PR's commits. The PR description is not kept. Anything decided in a PR description or review thread is lost from the history unless someone writes it down.

So write rules and decisions into these docs, in the same PR that makes them:

- A new rule or convention goes into the doc that owns the topic (see the table above).
- A decision with a reason ("we drop the legacy tables instead of migrating them") goes into the doc or into the migration or code comment next to the change.
- A doc that no longer matches the code is a bug. Fix it in the PR that changes the code.

Local Development

Source: docs/LOCAL-DEVELOPMENT.md

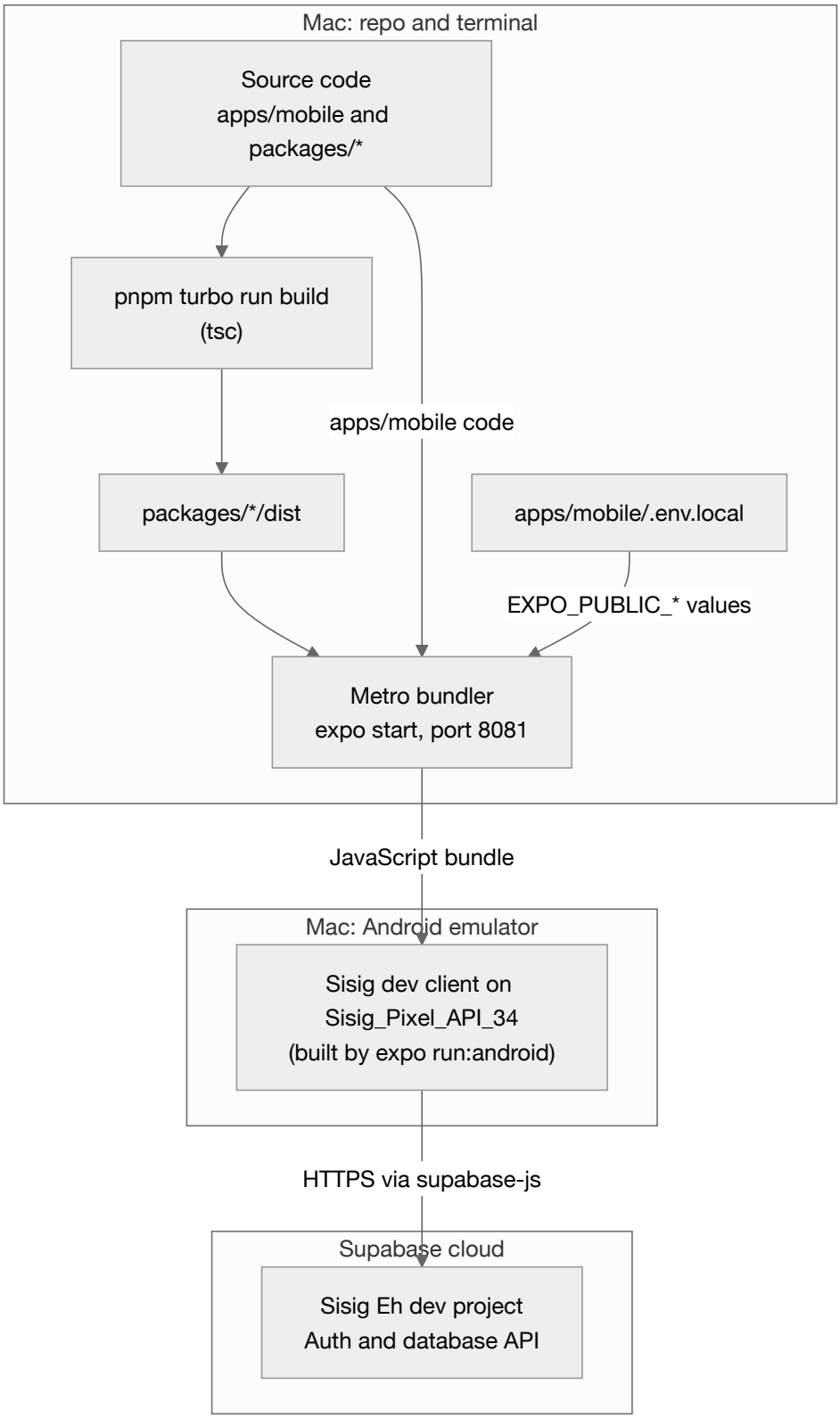
This guide takes a Mac from a clean clone to the Sisig Eh! mobile app running on an Android emulator, connected to the hosted Supabase dev project. It covers first-time setup, the daily routine, and fixes for failures the team has hit.

The steps were verified on 2026-11-06 on macOS 14 (Apple Silicon).

Scope:

- **Android only.** iOS needs the full Xcode app, which is not set up on the team's Mac yet. The Xcode Command Line Tools are not enough.
- **Expo Go is not supported.** The app runs in its own development build (the "dev client"), described below.
- **No local database.** The app talks to the hosted Supabase dev project. A local Supabase stack needs Docker; see docs/SUPABASE.md and supabase/README.md for database work.

What Runs Where



Metro bundles the app's JavaScript and serves it to the dev client on the emulator; both run on your Mac. The workspace packages reach Metro only through their compiled `dist/` folders, so they must be built first. The

app then talks straight to the hosted Supabase project over the internet.

Terms used in this guide:

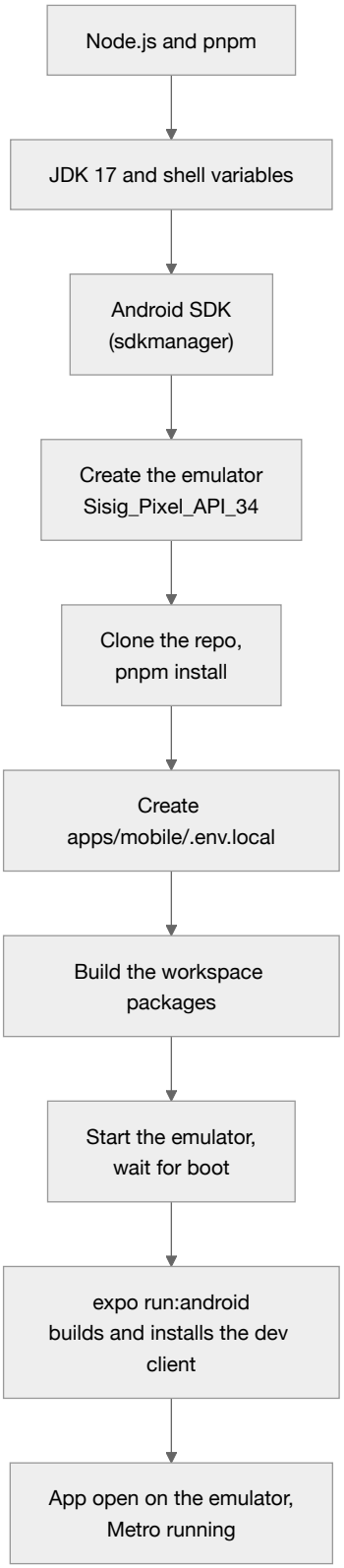
- **Dev client:** a debug build of this app with the native code it needs, plus the `expo-dev-client` launcher. You build it once and reinstall it only when native code changes.
- **Metro:** the JavaScript bundler that `expo start` runs. It serves the app's code on port 8081 and reloads the app when you save a file.
- **Workspace packages:** `@sisig/api`, `@sisig/ui` and `@sisig/claude` in `packages/`. Each `package.json` points `main` and `exports` at `dist/index.js`, so Metro cannot resolve them until `tsc` has built `dist/`.
- **AVD:** Android Virtual Device, the emulator's saved device profile.

Prerequisites

Tool	Version	Notes
macOS	14, Apple Silicon	The setup below uses arm64 images.
Node.js	18 or later	CI uses Node 18. The team's Mac runs Node 22.
pnpm	9	The root `package.json` sets `"packageManager": "pnpm@9.0.0"`.
JDK	17	Gradle needs it for the Android build. Any JDK 17 works.
Android SDK	API 34	Installed with the command-line tools (or with Android Studio).
Git	any	
Supabase access		Ask the project owner to add you to the "Sisig Eh" project in the Supabase dashboard. You need it to copy the API URL and the publishable key.

Disk space. Measured on the team's Mac: the API 34 system image is about 4.2 GB, the AVD about 4.3 GB, the NDK about 3.0 GB, and `~/.gradle` about 4.6 GB. A build also writes several GB to `apps/mobile/android/app/build`. Plan for roughly 20 GB free. Builds failed on that Mac when the disk ran low (see Troubleshooting).

First-Time Setup



The first nine boxes are the numbered steps below; the last box is the result. Steps 1 to 4 happen once per Mac, steps 5 to 7 and 9 once per clone, and step 8 (starting the emulator) every session.

1. Install Node.js and pnpm

Install Node.js 18 or later (for example from nodejs.org), then pnpm 9:

```
npm install -g pnpm@9
```

Check both:

```
node --version
pnpm --version
```

2. Install JDK 17 and set the shell variables

On the team's Mac, Homebrew's `openjdk@17` asked for the full Xcode app, so the Eclipse Temurin 17 tarball was used instead.

1. Download the Eclipse Temurin 17 JDK for macOS aarch64 as a `.tar.gz` from adoptium.net.
2. Extract it into your user Java folder. The file name depends on the version you downloaded:

   ```
   mkdir -p ~/Library/Java/JavaVirtualMachines
   ```

   ```
   tar -xzf ~/Downloads/OpenJDK17U-jdk_aarch64_mac_hotspot_<version>.tar.gz -C
   ~/Library/Java/JavaVirtualMachines
   ```

   This creates a folder such as `jdk-17.0.21+9`.
3. Add these lines to `~/.zshrc`, then open a new terminal:

   ```
   export JAVA_HOME="$(/usr/libexec/java_home -v 17)"
   export ANDROID_HOME="$HOME/Library/Android/sdk"
   export PATH="$JAVA_HOME/bin:$ANDROID_HOME/cmdline-tools/latest/bin:$ANDROID_HOME/platform-
   tools:$ANDROID_HOME/emulator:$PATH"
   ```

   `/usr/libexec/java_home -v 17` finds JDK 17 in `~/Library/Java/JavaVirtualMachines` as well as `/Library/Java/JavaVirtualMachines`. On the team's Mac it returns `~/Library/Java/JavaVirtualMachines/jdk-17.0.21+9/Contents/Home`.
4. Check that Java 17 is active:

   ```
   java -version
   ```

If you prefer not to edit `~/.zshrc`, put the variables in front of each command instead, for example `JAVA_HOME=... ANDROID_HOME=... pnpm -F @sisig/mobile exec expo run:android`. The rest of this guide assumes the variables are set.

3. Install the Android SDK

The team installed the SDK with the command-line tools' `sdkmanager`. Android Studio's SDK Manager installs the same packages into the same folder, if you prefer it.

1. Download "Command line tools only" for Mac from developer.android.com/studio.
2. Unzip it so that `sdkmanager` ends up in `$ANDROID_HOME/cmdline-tools/latest/bin`:

   ```
   mkdir -p "$ANDROID_HOME/cmdline-tools"
   ```

   ```
   unzip ~/Downloads/commandlinetools-mac-<version>_latest.zip -d "$ANDROID_HOME/cmdline-tools"
   ```

   ```
   mv "$ANDROID_HOME/cmdline-tools/cmdline-tools" "$ANDROID_HOME/cmdline-tools/latest"
   ```

3. Install the SDK packages:

   ```
   sdkmanager "platform-tools" "emulator" "platforms;android-34" "build-tools;34.0.0" "system-images;android-34;google_apis;arm64-v8a"
   ```

4. Accept the licences:

   ```
   yes | sdkmanager --licenses
   ```

   This can exit with code 141 (SIGPIPE) when your shell uses `set -o pipefail`. That is harmless: the licences are accepted.

The first Gradle build downloads the NDK and CMake by itself (on the team's Mac: NDK 26.1.10909125 and CMake 3.22.1). You do not install them here.

4. Create the emulator

Create an AVD named `Sisig_Pixel_API_34` (Pixel 8 profile, API 34):

```
avdmanager create avd -n Sisig_Pixel_API_34 -k "system-images;android-34;google_apis;arm64-v8a" -d pixel_8
```

If it asks whether to create a custom hardware profile, press Return to accept the default (no).

5. Clone the repo and install dependencies

```
git clone https://github.com/Alexd2004/Sisig.git
cd Sisig
```

```
pnpm install
```

Run every later command from the repo root.

6. Create `apps/mobile/.env.local`

The app reads two public values at bundle time. Create `apps/mobile/.env.local` with this content, then fill in the placeholders:

```
EXPO_PUBLIC_SUPABASE_URL=https://<project-ref>.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=<publishable key>
```

- `<project-ref>` is the ID of the "Sisig Eh" project. It is the last part of the project's dashboard URL (`supabase.com/dashboard/project/<project-ref>`).
- `<publishable key>` is on the dashboard under **Project Settings** → **API Keys**.

Rules:

- `.env.local` is git-ignored (`.gitignore` lists `.env.local`). Never commit it, and never paste its values into docs, issues or PRs.
- Use only the publishable key. Never put a secret key or a service-role key in the app: Expo copies every `EXPO_PUBLIC_*` value into the JavaScript bundle, where anyone with the app can read it. CI also fails if client code contains a service-role key (see DEVOPS-02-CI.md).
- If either value is missing, the app throws at launch: `EXPO_PUBLIC_SUPABASE_URL and EXPO_PUBLIC_SUPABASE_ANON_KEY must be set. See CONTRIBUTING.md.` The check is in `apps/mobile/src/lib/supabase.ts`.

7. Build the workspace packages

```
pnpm turbo run build --filter='@sisig/mobile^...'
```

The filter `@sisig/mobile^...` means "every workspace package the mobile app depends on, but not the app itself". Each package compiles with `tsc` into its `dist/` folder. Without this step Metro cannot resolve `@sisig/ui` and the bundle fails.

Do not run plain `pnpm build` for this. It also runs the mobile app's `build` script (`expo build`), which the local Expo CLI rejects.

8. Start the emulator and wait for it to boot

Start the emulator in its own terminal tab. It keeps running there:

```
"$ANDROID_HOME/emulator/emulator" -avd Sisig_Pixel_API_34
```

In your main terminal, wait until Android has finished booting:

```
adb wait-for-device; until adb shell getprop sys.boot_completed | grep -q 1; do sleep 3; done
```

Until boot completes, `adb devices` lists the emulator as `offline`, and a build or launch started then fails.

9. Build and install the dev client

```
pnpm -F @sisig/mobile exec expo run:android
```

The first run takes a while. It:

1. Runs `expo prebuild` for Android, because `apps/mobile/android/` does not exist yet. Prebuild generates that native project and adds `"android"` and `"ios"` scripts to `apps/mobile/package.json`.
2. Builds the debug app with Gradle (the first build downloads Gradle, the NDK and CMake).
3. Installs the dev client on the emulator, starts Metro on port 8081 and opens the app.

Do not commit the generated files. Prebuild output is not in `.gitignore` yet: `git status` shows `apps/mobile/android/` (and `apps/mobile/ios/`, if you ever ran `expo run:ios` or a plain `expo prebuild`) as untracked, and `apps/mobile/package.json` as modified. Leave them out of your commits: stage files by name instead of `git add -A`, and undo the script change with `git restore apps/mobile/package.json` when you need a clean tree.

When the app opens on the emulator, setup is done.

Daily Routine

1. Start the emulator in its own terminal tab, and wait for boot:

   ```
   "$ANDROID_HOME/emulator/emulator" -avd Sisig_Pixel_API_34
   ```

   ```
   adb wait-for-device; until adb shell getprop sys.boot_completed | grep -q 1; do sleep 3; done
   ```

2. If you pulled new commits, install dependencies and rebuild the packages:

   ```
   pnpm install
   ```

   ```
   pnpm turbo run build --filter='@sisig/mobile^...'
   ```

   Turborepo caches the build, so packages that did not change are restored from cache in a moment.

3. Start Metro and open the installed dev client:

   ```
   pnpm -F @sisig/mobile exec expo start --dev-client --android
   ```

If dependencies changed in step 2, add `--clear` to this command (see Troubleshooting).

4. Edit code. Metro reloads the app when you save a file under `apps/mobile`. Press `r` in the Metro terminal to reload by hand.
5. After you change code in `packages/*`, run the build command from step 2 again. Metro reads the packages from `dist/`, so it does not see a source change until the package is rebuilt.

When to rebuild the dev client. Only when native code changes: a dependency with native code is added, removed or upgraded, or `apps/mobile/app.json` changes (plugins, icons, package name). `expo run:android` runs prebuild only when `apps/mobile/android/` does not exist, so after an `app.json` change delete that generated folder first:

```
rm -rf apps/mobile/android
```

```
pnpm -F @sisig/mobile exec expo run:android
```

JavaScript-only changes never need a rebuild.

About `pnpm dev`. The root `pnpm dev` also starts Metro. It runs Turborepo, which runs the mobile app's `dev` script, `expo start --dev-client`. Turborepo runs the `build` tasks of the workspace packages before it (the `^build` dependency in `turbo.json`). The root scripts also pass the deprecated `--parallel` flag; in a timing check with Turborepo 2.11.10 and no cache, dependent tasks still started only after the builds finished. Ordering under `--parallel` is not guaranteed by Turborepo's docs, so building the packages yourself first (step 2) is the safe routine. It does not pass `--android`, so it does not open the app on the emulator. The direct command in step 3 is the routine verified on the team's Mac.

Database Work

The app uses the hosted Supabase dev project "Sisig Eh". It is the only hosted project. Schema changes go through migrations in pull requests (see CONTRIBUTING.md).

The Supabase CLI (2.122.0) is a root devDependency, so run it through `pnpm exec`. To connect your clone to the hosted project:

```
pnpm exec supabase login
```

```
pnpm exec supabase link --project-ref <project-ref>
```

`supabase link` stores the link in `supabase/.temp/`, which is git-ignored.

The local stack (`pnpm db:start`, `pnpm db:test`, `pnpm db:types`) needs Docker. Docker is not installed on the team's Mac, so the database tests run in CI. The full procedure for migrations, `supabase db push`, and destructive changes is in docs/SUPABASE.md.

Troubleshooting

Metro: "Unable to resolve" `@sisig/ui` or another `@sisig/*` package

The workspace packages are not built, so their `dist/` folders are missing. Build them, then restart Metro:

```
pnpm turbo run build --filter='@sisig/mobile^...'
```

Metro: "Unable to resolve " after pulling

New JavaScript dependencies arrived. Install them, rebuild the packages, and restart Metro with a clear cache:

```
pnpm install
```

```
pnpm turbo run build --filter='@sisig/mobile^...'
```

```
pnpm -F @sisig/mobile exec expo start --dev-client --android --clear
```

The app shows old code

Press `r` in the Metro terminal. If that does not help, force-stop the app and open it again from the emulator's launcher:

```
adb shell am force-stop com.sisig.mobile
```

`com.sisig.mobile` is the Android package name from `apps/mobile/app.json`.

The app crashes at launch: "EXPO_PUBLIC_SUPABASE_URL and EXPO_PUBLIC_SUPABASE_ANON_KEY must be set. See CONTRIBUTING.md."

`apps/mobile/.env.local` is missing or incomplete (see step 6). After you fix it, restart Metro with `--clear` so the new values are bundled.

The app crashes at launch: "Property 'jest' doesn't exist"

A test file is inside `apps/mobile/app/`. Expo Router turns every file in that folder into a route, so the test was bundled as a screen. Move the test out of `app/`, for example to `apps/mobile/src/`, and import the screen from there. `apps/mobile/src/navigation/index-redirect.test.tsx` is an example (PR #18).

`adb devices` shows the emulator as "offline"

The emulator is still booting. Wait with the boot loop from step 8 before you build or launch.

`adb`, `emulator`, `sdkmanager` or `avdmanager`: command not found

The PATH line from step 2 is missing or your terminal predates it. Open a new terminal, then check:

```
echo "$ANDROID_HOME"
```

Gradle fails because Java is missing or is not version 17

`java -version` must report 17, and `JAVA_HOME` must point at that JDK. `/usr/bin/java` on macOS is only a stub that looks for an installed JDK; it does not provide one.

`yes | sdkmanager --licenses` exits with code 141

Harmless. `yes` gets SIGPIPE when `sdkmanager` stops reading, and `set -o pipefail` reports that exit code. The licences are accepted.

Homebrew `openjdk@17` asks for Xcode

Use the Temurin 17 tarball from step 2 instead. Any JDK 17 works.

The build fails with disk errors, or the Mac runs out of space

An Android build needs several GB on top of the SDK and the AVD (see Prerequisites). To free space, stop the Gradle daemon and delete the build output:

```
(cd apps/mobile/android && ./gradlew --stop)
```

```
rm -rf apps/mobile/android/app/build
```

The next build recreates the folder and takes longer.

`expo-doctor` reports native project folders

After prebuild, `pnpm -F @sisig/mobile run doctor` also warns that `app.json` properties will not sync because `android/` and `ios/` exist. That warning comes from the generated folders on your machine. A clean clone has three other known failures, listed in the README.

iOS

Not set up. `pnpm -F @sisig/mobile exec expo run:ios` needs the full Xcode app and an iOS simulator, which the team's Mac does not have yet.

Sisig Platform Architecture Guide

Source: docs/ARCHITECTURE.md

Version 1.1 | **Updated:** November 2026 | **Describes:** `main` at `cd47f1e` (PR #19)

This guide explains how the Sisig Eh! rewards app fits together: the apps and packages in the monorepo, how they build, how navigation and sign-in gating work, how the session and data layer behave, and how the theme reaches components. Database detail lives in SUPABASE.md. Running the app on an Android emulator is covered in LOCAL-DEVELOPMENT.md.

Table of Contents

1. Overview
2. Monorepo and Build Order
3. DATA-01: Database Architecture
4. PLAT-02: Navigation Architecture
5. PLAT-03: Session and Data Layer
6. PLAT-06: Remote Config and Forced Update
7. THEME-01: Design System
8. Claude Integration
9. Integration Points
10. Development Workflow
11. Deployment Strategy
12. Summary of Key Decisions
13. Next Steps
14. Related Documents

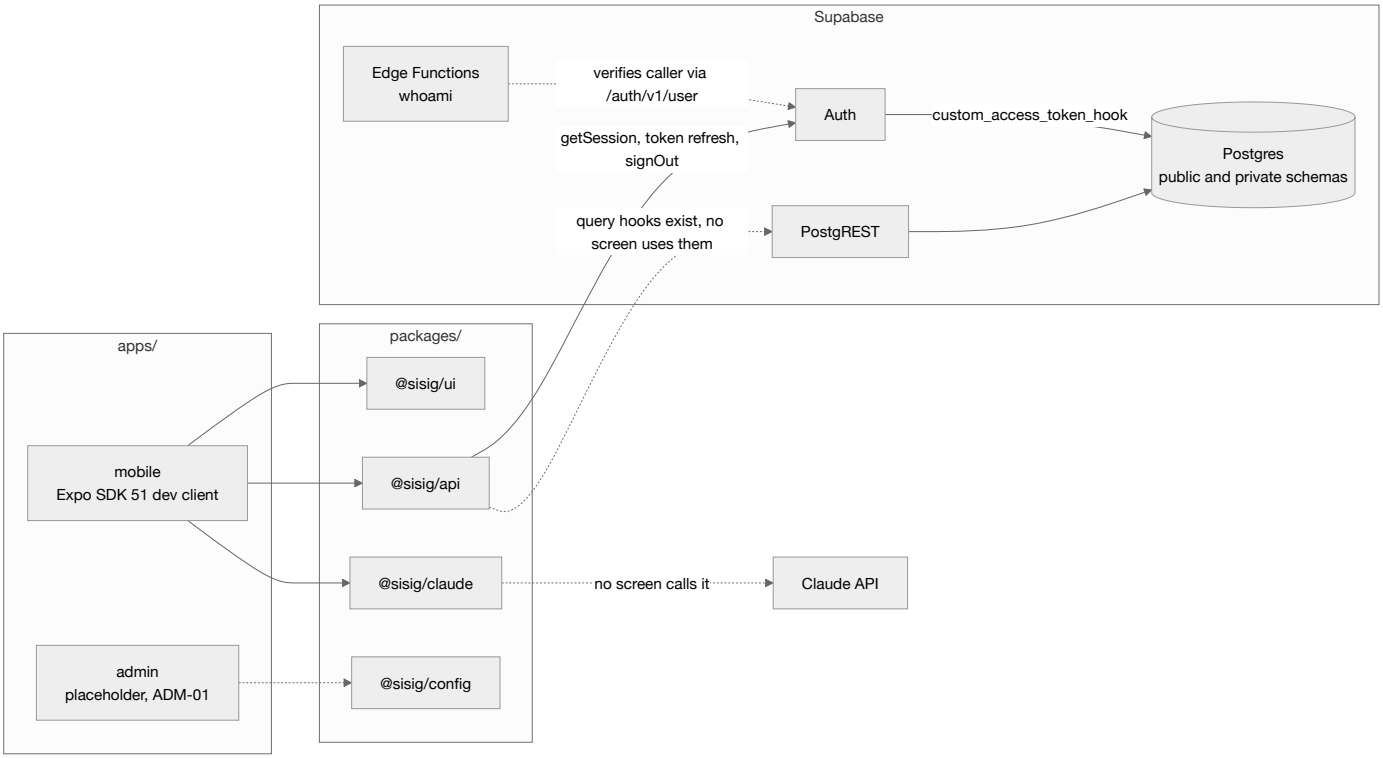
Overview

Sisig is a mobile-first rewards and ordering app. The repository is a pnpm workspace orchestrated by Turborepo.

Part	Path	What it is today
Mobile app	`apps/mobile`	Expo SDK 51 (React Native 0.74, React 18.2), Expo Router 3, TypeScript strict mode. Runs as a development build (a "dev client": a debug build of the app with Expo's developer tools). Expo Go is not supported.
Admin app	`apps/admin`	A `package.json` only. Every script echoes a placeholder message. The framework choice is planned (ADM-01).
`@sisig/ui`	`packages/ui`	Brand tokens, Tamagui config, theme hooks, and the `Button`, `Text` and `TextInput` components.

Part	Path	What it is today
`@sisig/api`	`packages/api`	Supabase client factory, error normalisation, QueryClient factory, query keys, persistence allowlist, menu query hooks, order and redeem mutations, generated database types.
`@sisig/claude`	`packages/claude`	A wrapper around `@anthropic-ai/sdk` plus helper functions. No screen calls it yet (see Claude Integration).
`@sisig/config`	`packages/config`	Shared `tsconfig.json`, ESLint and Prettier config.
Supabase	`supabase/`	Migrations, pgTAP tests, seed data, local stack config, and Edge Functions (`whoami` plus `_shared` helpers).

Most screens are still placeholders. Home, Order, Rewards and Code show hard-coded sample content, and the auth screens are placeholders until AUTH-01 to AUTH-03. No screen reads data from Supabase yet. Today the app talks to Supabase Auth only: it restores the session, refreshes tokens, and signs out.

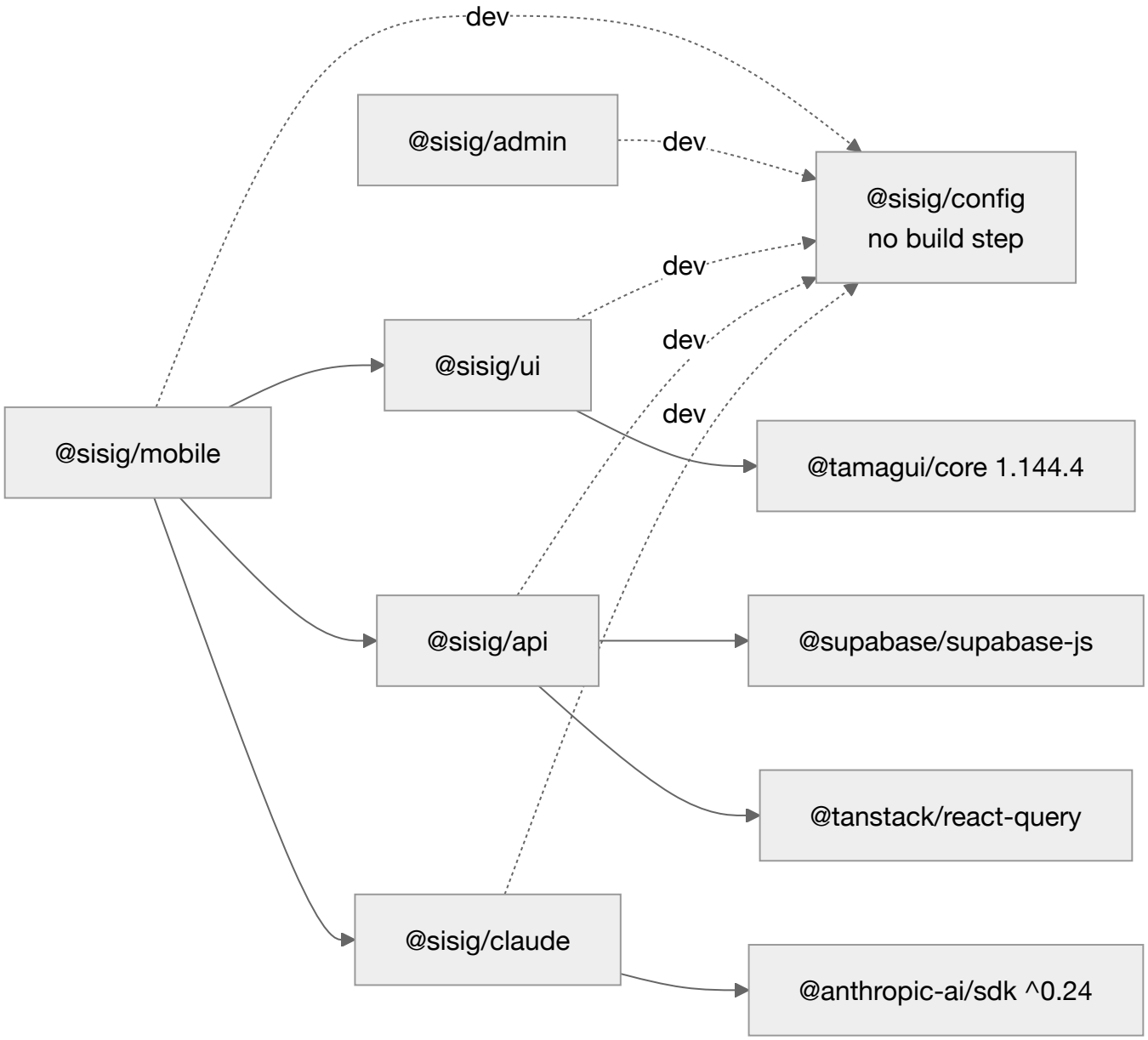


Solid arrows are calls the app makes on `main`. Dotted arrows are code that exists but that the app never runs, including the `whoami` Edge Function, which the app does not call yet.

Monorepo and Build Order

`pnpm-workspace.yaml` includes `apps/*` and `packages/*`. Each app depends on shared packages through `workspace:*` versions, so pnpm links them as symlinks into the app's `node_modules`.

Workspace	Build script	Workspace dependencies
`@sisig/mobile`	`expo build` (the local Expo CLI rejects it; release builds are planned, DEVOPS-04)	`@sisig/ui`, `@sisig/api`, `@sisig/claude`; `@sisig/config` (dev)
`@sisig/admin`	echo placeholder	`@sisig/config` (dev)
`@sisig/ui`	`tsc` to `dist/`	`@sisig/config` (dev)
`@sisig/api`	`tsc` to `dist/`	`@sisig/config` (dev)
`@sisig/claude`	`tsc` to `dist/`	`@sisig/config` (dev)
`@sisig/config`	none	none



Workspace dependencies come from each `package.json`. Solid arrows are runtime dependencies; dotted arrows are dev dependencies. The bottom row shows the main third-party library behind each package.

How Turborepo orders the work

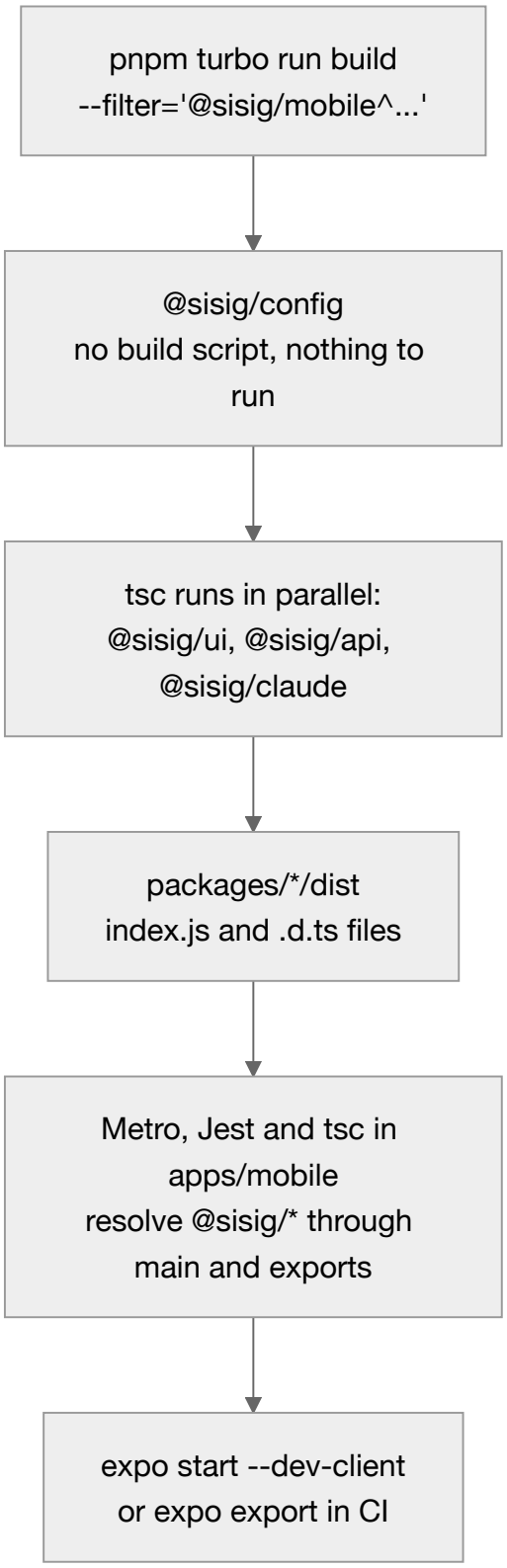
`turbo.json` gives `build`, `dev`, `typecheck` and `test` a `^build` dependency: before a task runs in a workspace, Turborepo runs `build` in every workspace it depends on. `build` caches `dist/**`, `.expo/**` and `build/**`.

How Metro finds the shared packages

There is no `metro.config.js`. Expo's default Metro config (from `expo/metro-config`) detects the workspace root. Each shared package's `package.json` points `main`, `types` and `exports` at `dist/index.js` and `dist/index.d.ts`. So Metro, Jest and `tsc` all read the built output, not `src/`. A fresh clone, or any change under `packages/*/src`, needs a build before Metro can serve it:

```
pnpm turbo run build --filter='@sisig/mobile^...'
```

The `^...` suffix selects every workspace the mobile app depends on, but not the app itself, whose `build` script would fail. After you pull new dependencies, restart Metro with `--clear`. CI runs the same build before it bundles the app with `expo export --platform android` (added in PR #16).



The three package builds only depend on `@sisig/config`, so they run side by side. The app reads their `dist/` output, so a stale `dist/` means stale code on the device.

DATA-01: Database Architecture

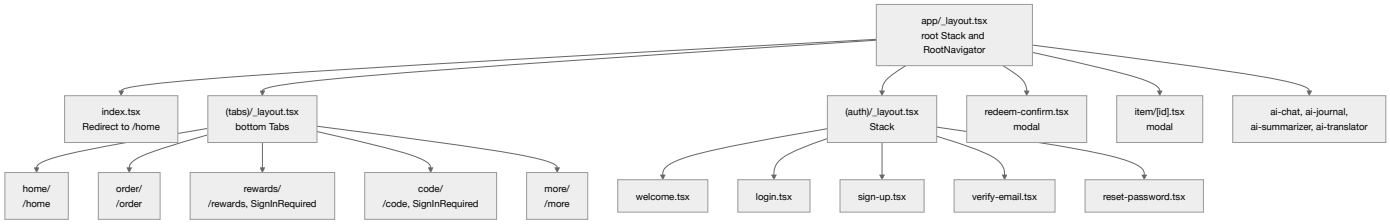
The full Supabase guide is SUPABASE.md. Conventions for migrations, policies and functions are in supabase/CONVENTIONS.md. In short:

- **Postgres on Supabase.** Forward-only migrations live in `supabase/migrations/`. Never edit an applied migration; add a new one.
- **Row-Level Security (RLS).** Every table created in the `public` schema enables RLS. The `private` schema holds internal functions (for example `custom_access_token_hook`) and is not exposed through PostgREST.
- **Current `public` tables** (from `packages/api/src/database.types.ts`): `profiles`, `user_roles`, `products`, `orders`, `order_items`, `locations`, `loyalty_settings`, `points_ledger`, `point_balances`, `rewards`, `reward_redemptions` and `app_config` (PLAT-06).
- **PR #19 (DATA-04)** dropped `public.users`, the `user_role` enum, and the legacy `loyalty_points` and `loyalty_transactions` tables. Identity now lives in `profiles` and `user_roles` (DATA-02). Loyalty now uses the ledger tables above.
- **Money is integer cents**, with non-negative checks.
- **Points change only through RPCs** (database functions called over PostgREST). The posting and redemption RPCs are planned (LOY-01, LOY-03, LOY-04).
- **Tests and types.** pgTAP tests live in `supabase/tests/database/` (`pnpm db:test`). `pnpm db:types` regenerates the TypeScript types from the local stack, and the CI `types-drift` job fails if the committed file differs.

PLAT-02: Navigation Architecture

The app uses Expo Router 3, which turns files under `apps/mobile/app/` into routes. The entry point is `expo-router/entry` (`main` in `apps/mobile/package.json`). Folders in parentheses, such as `(tabs)` and `(auth)`, are route groups: they share a layout but do not appear in the URL, so `app/(auth)/login.tsx` is `/login`.

Route tree



Each tab is a folder with its own `_layout.tsx` (a Stack) and an `index.tsx` screen. The Rewards and Code layouts wrap their Stack in `SignInRequired`. No screen links to `item/[id]` or to the four AI routes yet.

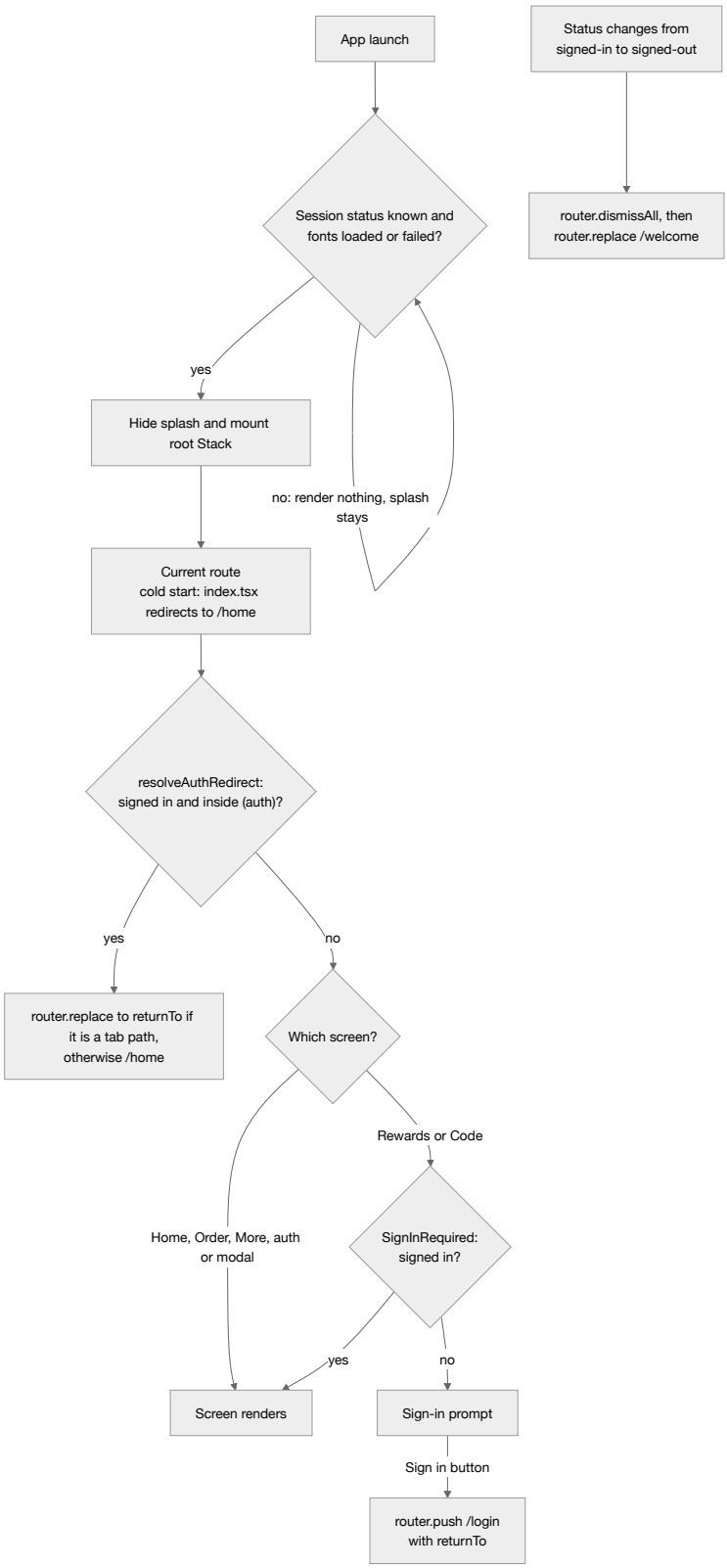
Root layout

`app/_layout.tsx` nests the providers in this order: `AppThemeProvider` (theme), `PersistQueryClientProvider` (query cache with disk persistence), `RemoteConfigProvider` (remote config and the update gate, PLAT-06), `SessionProvider` (session status), then `RootNavigator`.

At module level, `app/_layout.tsx` calls `SplashScreen.preventAutoHideAsync()`, so the splash screen stays up. `RootNavigator` then:

- Renders nothing until the session status is no longer `loading`, the Inter fonts have loaded or failed, and the remote config has read its disk cache (at most 2 seconds). A font failure falls back to the system font.
- Renders `ForcedUpdateScreen` instead of the Stack while the update gate is `blocked`, and skips its navigation effects then (see PLAT-06).
- Computes `resolveAuthRedirect` from the status, the route segments and a `returnTo` search parameter, and calls `router.replace` when it returns a path.
- Watches for a change from `signed-in` to `signed-out`. It then calls `router.dismissAll()` and `router.replace('/welcome')`.
- Declares `redeem-confirm` and `item/[id]` as modal screens.

Auth gating and redirects



Guests can browse Home, Order and More. Rewards and Code show a sign-in prompt instead of their stack. `sanitizeReturnTo` only accepts the five tab paths, so a `returnTo` parameter cannot send a user anywhere

else. The login screen is a placeholder until AUTH-01 to AUTH-03, so nobody can sign in from the app yet.

Tabs

- Five tabs: Home, Order, Rewards, Code, More. Each has a `tabBarAccessibilityLabel` (for example "Rewards tab") and a Material Community icon.
- The Tabs layout hides its own header. Each tab's Stack draws the header with colours from `useStackScreenOptions` (`src/navigation/stack-options.ts`).
- Tapping the active tab pops that tab's stack to its root. `popTabStackToRoot` (`src/navigation/pop-to-root.ts`) dispatches `POP_TO_TOP` to the nested stack only, so other tabs keep their stacks.
- The More tab shows Sign In (opens `/login` with `returnTo=/more`) or Sign Out (calls `useSession().signOut`). Its other menu items only log to the console for now.

What each screen shows today

Route	Content on `main`
`/home`	The soft update banner and the announcement banner when the remote config asks for them, then static welcome text, a sample special, "You have 0 points", and a list of quick links.
`/order`	Three hard-coded sample items with Add buttons that do nothing, and a $0.00 cart total. While `ordering_enabled` is false, an "Ordering is paused" card with the configured message instead.
`/rewards`	Points shown as 0, three sample rewards whose Redeem buttons open `/redeem-confirm`, and a "How it works" card.
`/code`	A placeholder QR area and a sample code behind a Show Code toggle. While the code is hidden, a leftover "Sign in to view your code" card with a Sign In button shows, even to signed-in users, and the button does nothing.
`/more`	Menu items and the Sign In or Sign Out item.
`(auth)` screens	`PlaceholderScreen` with a note saying the real screen is built in AUTH-01 to AUTH-03. Welcome has Log in, Sign up and Continue as guest buttons.
`/redeem-confirm`, `/item/[id]`	`PlaceholderScreen` with a note saying the real screen is built in a later ticket.

Keep tests out of `app/`

Expo Router treats every file under `app/` as a route, including test files. A test file there gets bundled as a screen and crashes the app on the device with "Property 'jest' doesn't exist". PR #18 moved the only such test to `src/navigation/index-redirect.test.tsx`. Put tests under `src/` and import screens with a relative path such as `../../app/index`.

PLAT-03: Session and Data Layer

The shared code is in `packages/api`. The mobile wiring is in `apps/mobile/src/lib` and `apps/mobile/src/session`. PR #12 isolated the logout purge steps and made key creation single-flight. PR #15

connected the Supabase session source and the purge on sign-out.

Supabase client

`src/lib/supabase.ts` creates the app's only Supabase client and QueryClient:

1. It reads `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_ANON_KEY`. Expo inlines `EXPO_PUBLIC_*` values into the bundle at build time, so only public values may use that prefix. The anon key is Supabase's publishable key. If either value is missing, the module throws when the app starts.
2. `createSupabaseClient(nativeSessionStorage, { url, anonKey })` (`@sisig/api`) calls `createClient` with the encrypted storage adapter, `autoRefreshToken: true`, `persistSession: true` and `detectSessionInUrl: false`.
3. `createQueryClient()` builds the QueryClient.
4. `attachSessionLifecycle(supabase, AppState)` runs token auto-refresh only while the app is in the foreground. It stops refresh when the app goes to the background or becomes inactive, and starts it again when the app is active.

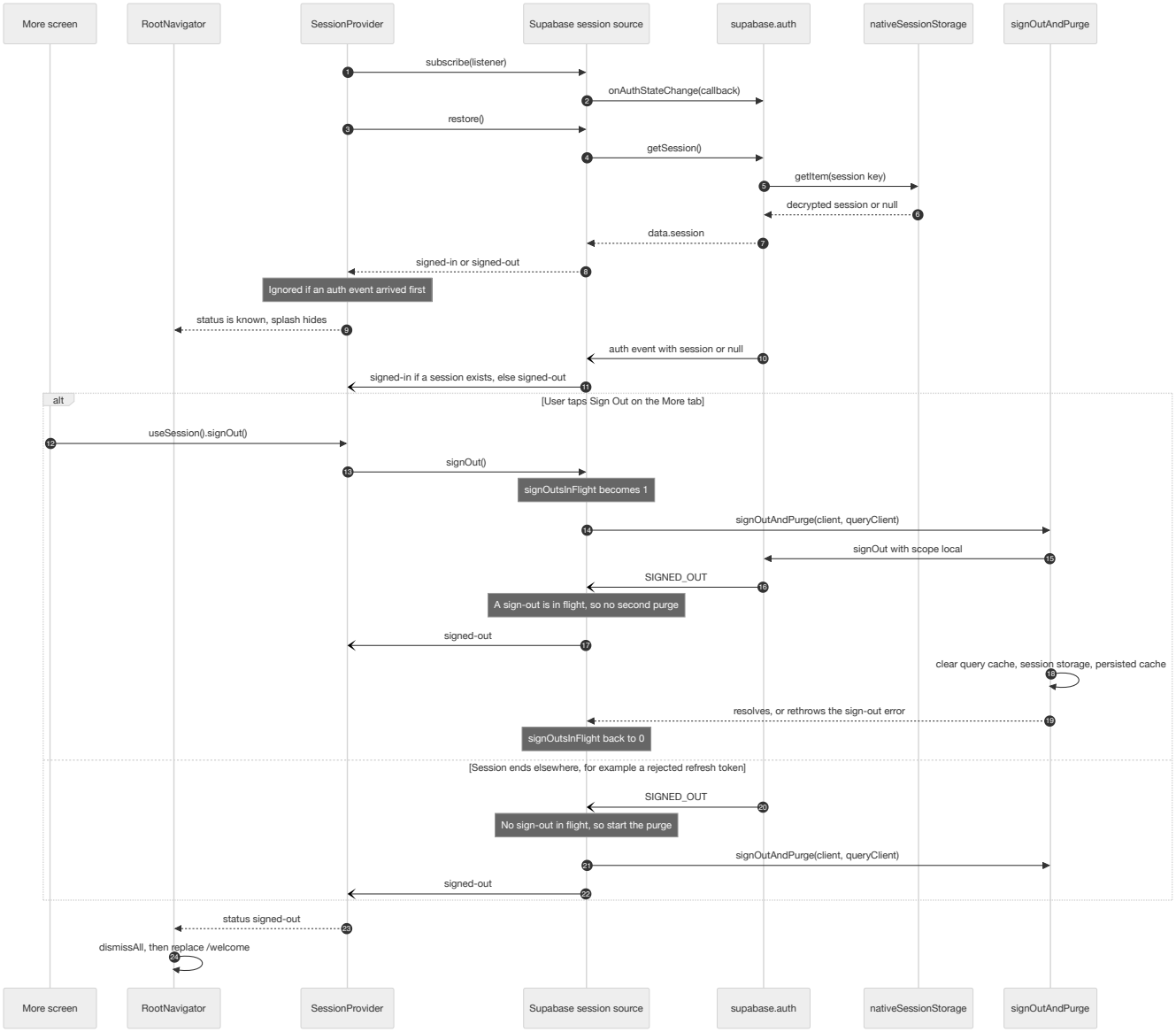
Encrypted session storage

`src/lib/session-storage.ts` is the `auth.storage` adapter that supabase-js uses for the session:

- A random 256-bit AES key (from `expo-crypto`) is stored in the device keystore through `expo-secure-store`, under `sisig_session_key`, with `AFTER_FIRST_UNLOCK_THIS_DEVICE_ONLY`.
- Every value is encrypted with AES-256-GCM (`@noble/ciphers`) under a fresh 96-bit nonce. The ciphertext goes to AsyncStorage under `sisig.secure.v1:<key>` as JSON with `v`, `iv` and `ct` fields. Tokens never reach AsyncStorage in plaintext.
- Only the key lives in SecureStore, because a Supabase session can be larger than the roughly 2 KB a SecureStore value is meant to hold.
- An entry that fails to decrypt (tampered data, or the key was reset) returns `null` and is deleted, so the user signs in again. A keystore read error returns `null` but keeps the entry.
- Concurrent first writes share one key-creation promise, so two writes cannot each create a key.
- `clearSessionStorage()` removes every sealed entry and two legacy keys, and deletes the AES key. Both steps run even if one fails.

Session lifecycle

`SessionProvider` (`src/session/session.tsx`) exposes `status` (`loading`, `signed-in` or `signed-out`) and `signOut` through `useSession()`. It takes a `SessionSource` with three methods: `restore`, `subscribe` and `signOut`. The app passes `createSupabaseSessionSource(supabase, queryClient)`, created once at module load in `app/_layout.tsx`.



The source maps every auth event to `signed-in` or `signed-out` by whether a session is present. It purges on every `SIGNED_OUT`, and the in-flight counter stops the `SIGNED_OUT` that its own `auth.signOut` call emits from starting a second purge.

Logout purge

`signOutAndPurge` (`src/lib/logout.ts`) runs these steps in order:

1. `queryClient.cancelQueries()`.
2. `client.auth.signOut({ scope: 'local' })`. auth-js removes the local session even when the network call fails, and reports that failure as `{ error }` instead of throwing.
3. `queryClient.clear()` (in-memory cache).
4. `clearSessionStorage()` (sealed entries and the AES key).
5. `clearPersistedQueries()` (the query cache on disk).

Steps 3 to 5 each run in their own `try` / `catch`, and a failure is logged, not thrown. A sign-out failure, thrown or returned, is rethrown after the purge, so a purge error never replaces it. Any new per-user store must be added to this function.

Queries, mutations and the persisted cache

- **QueryClient** (`packages/api/src/query-client.ts`): queries default to a 60-second `staleTime`. `shouldRetryQuery` retries only network and 5xx failures, up to 3 times. Mutations never retry.
- **Query keys** (`packages/api/src/query-keys.ts`): every key comes from the `keys` factory (`auth`, `menu`, `orders`, `loyalty`, `config`, `profile`). Invalidate by prefix, for example `keys.menu.all()`.
- **Query hooks** (`packages/api/src/hooks/useMenu.ts`): `useMenu` (5-minute `staleTime`), `menuCategoriesQueryOptions` and `useMenuCategories` (10 minutes), and `useProduct` (5 minutes). Each takes the Supabase client as an argument.
- **Mutations** (`packages/api/src/mutations.ts`): `useCreateOrder` inserts into `orders` and `order_items` and invalidates `keys.orders.all()`. `useRedeemPoints` always throws `redeem_loyalty_reward RPC is not implemented yet` until the redemption RPC exists (planned, LOY-03). Both keep their invalidation when a caller passes its own `onSuccess` (PR #11). `optimisticUpdate` applies an update and rolls it back on error.
- **Persisted cache** (`apps/mobile/src/lib/query-persistence.ts`): `PersistQueryClientProvider` writes the cache to AsyncStorage under `sisig.query-cache`, unencrypted, with a 24-hour `maxAge` and buster `v1`. Only successful queries whose key starts with an allowlisted prefix are written: `['menu']` and `['loyalty', 'rewards']`. Points, profile, orders and auth are never written to disk, and mutations are never persisted.

No screen calls these hooks or mutations yet.

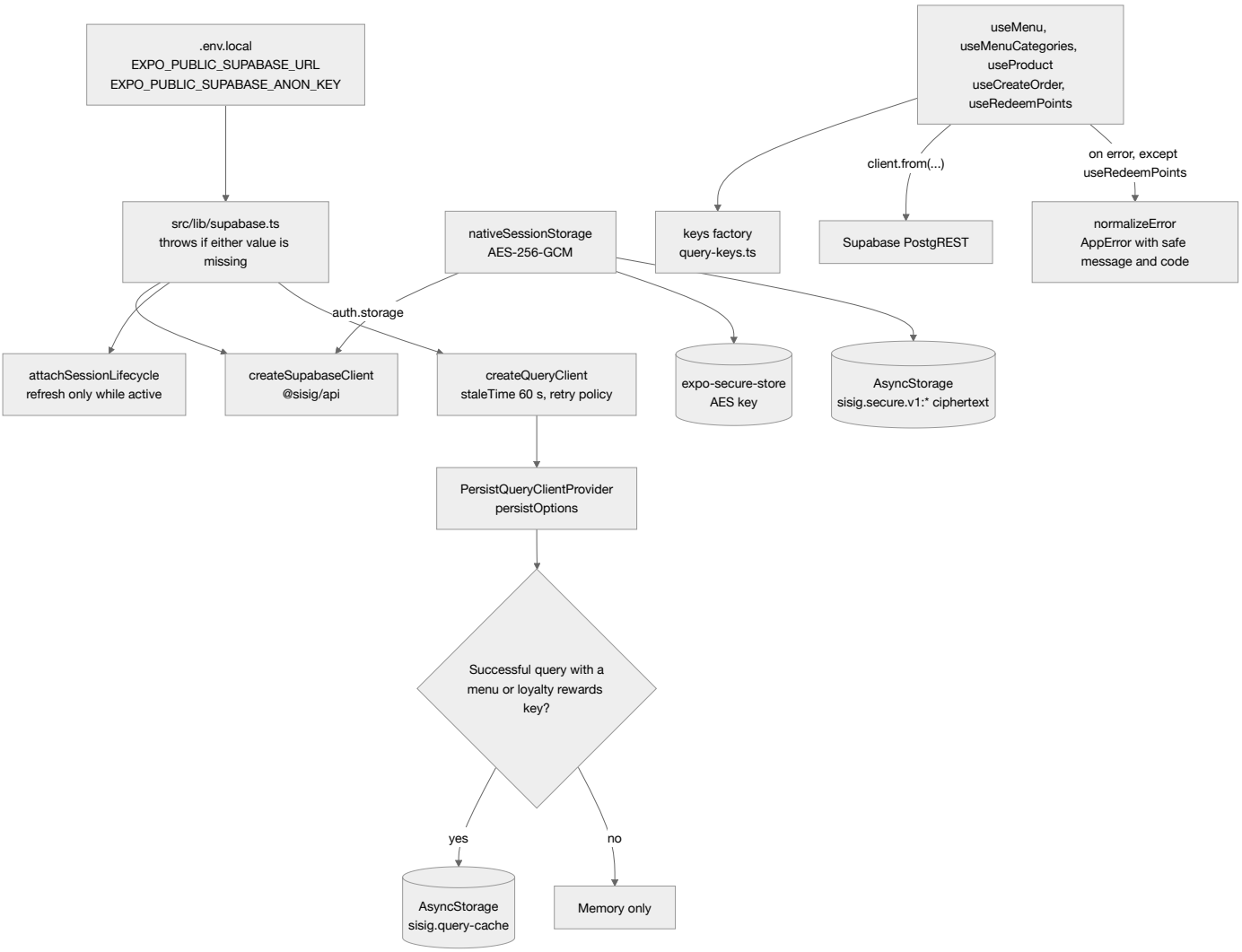
Error normalisation

Queries and mutations throw `normalizeError(error)` (`packages/api/src/client.ts`). The exception is `useRedeemPoints`, which makes no request and throws a plain `Error` until its RPC exists. It returns an `AppError` with a fixed, user-safe `message` and a stable `code`. The raw error stays in `details`.

Source	Condition	`code`
PostgREST	empty code (no HTTP response)	`NETWORK_ERROR`
PostgREST	`42501` or `PERM` (RLS denied)	`PERMISSION_DENIED`
PostgREST	`PGRST301` (JWT rejected)	`SESSION_EXPIRED`
PostgREST	`PGRST116` or `NTFND`	`NOT_FOUND`
PostgREST	`23505` (unique violation)	`DUPLICATE`
PostgREST	any other code	`DATABASE_ERROR`
Auth	retryable fetch error / session missing / `invalid_credentials`	`NETWORK_ERROR` / `UNAUTHORIZED` / `INVALID_CREDENTIALS`

Source	Condition	code
Auth	`session_not_found` or status 401 / status 500 or more / other	`SESSION_EXPIRED` / `SERVER_ERROR` / `AUTH_ERROR`
Edge Functions	HTTP 401 / 403 / 500 or more / other	`UNAUTHORIZED` / `FORBIDDEN` / `SERVER_ERROR` / `FUNCTION_ERROR`
Edge Functions	relay error / fetch error	`SERVER_ERROR` / `NETWORK_ERROR`
Anything else		`UNKNOWN_ERROR`

`isRetryableError` is true only for `NETWORK_ERROR` and `SERVER_ERROR`.



The session is encrypted on disk, while the persisted query cache is plain JSON, which is why only non-personal keys are on its allowlist. The hooks in the middle exist in `@sisig/api`, but no screen uses them yet.

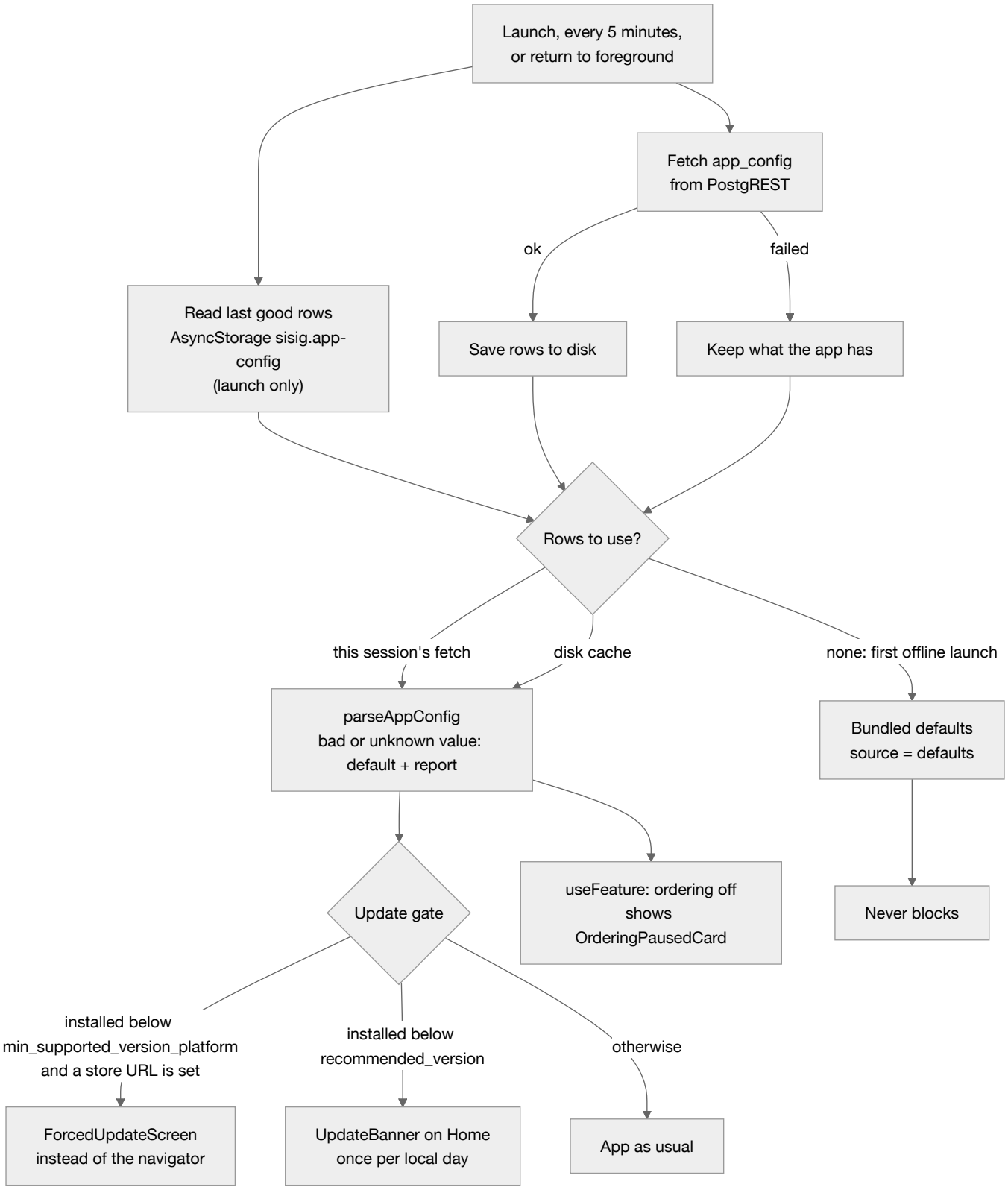
PLAT-06: Remote Config and Forced Update

The restaurant can switch off ordering or receipt scanning and force outdated apps to update without shipping a build. The values live in `public.app_config` (schema and RLS in SUPABASE.md). Until the admin team builds toggles, they are changed with SQL: see the remote config runbook.

Pieces

Piece	Where	Job
`appConfigQueryOptions`, `parseAppConfig`, `DEFAULT_APP_CONFIG`	`packages/api/src/remote-config.ts`	Fetch the raw `key` / `value` rows (key `keys.config.app()`, 5-minute `staleTime` and `refetchInterval`); validate each known key with zod; bundled fail-open defaults
`RemoteConfigProvider`, `useRemoteConfig`	`apps/mobile/src/config/remote-config.tsx`	Read the disk cache, fetch, refetch on foreground, save each good fetch, parse, compute the update gate
`useFeature('ordering' | 'receiptScan')`	`apps/mobile/src/config/useFeature.ts`	`{ enabled, message }` for feature screens. The Order tab uses it
`evaluateUpdateGate`	`apps/mobile/src/config/update-gate.ts`	`none`, `soft` or `blocked` from the installed version, the platform, the config and its source
`getInstalledVersion`	`apps/mobile/src/config/installed-version.ts`	The installed version, from `Constants.expoConfig.version`
`setConfigReporter`, `reportConfigIssue`	`apps/mobile/src/config/reporter.ts`	Where config problems go: `console.warn` in development, nothing in release builds until PLAT-04 plugs in Sentry
`ForcedUpdateScreen`, `UpdateBanner`, `AnnouncementBanner`, `OrderingPausedCard`	`apps/mobile/src/components/`	The UI, built from `@sisig/ui` `Text` and `Button`

Flow


Launch, every 5 minutes, or return to foreground
Fetch app_config from PostgREST
ok
failed
Read last good rows AsyncStorage sisig.app-config (launch only)
Save rows to disk
Keep what the app has
Rows to use?
this session's fetch
disk cache
none: first offline launch
parseAppConfig bad or unknown value: default + report
Bundled defaults source = defaults
Update gate
useFeature: ordering off shows OrderingPausedCard
Never blocks
installed below min_supported_version_platform and a store URL is set
installed below recommended_version
otherwise
ForcedUpdateScreen instead of the navigator
UpdateBanner on Home once per local day
App as usual

Rules

- **Fail-open.** A failed fetch never blocks: the app keeps the config it already has. The bundled defaults block nothing (every version `0.0.0`), and the gate ignores them anyway (`source === 'defaults'`). An unknown installed version, a platform other than iOS or Android, and a missing store URL also mean no block. A missing store URL is reported.
- **Blocking from the cache is allowed.** A cached config came from the server, so a cold start offline still shows the update screen if the last good config required it.
- **Per-key validation.** One malformed value falls back to its own default; the other keys still apply. Unknown keys are ignored. Each distinct problem goes to the reporter once per app session. Store links must be https on `apps.apple.com` or `play.google.com`, and the Uber Eats link on `ubereats.com`, because the blocking screen opens them with `Linking.openURL`.
- **Own disk cache.** The config is saved under `sisig.app-config`, not in the TanStack persisted cache, because that one expires after 24 hours and is cleared on logout. The config is not personal data, so `signOutAndPurge` leaves it. The once-a-day banner date (`sisig.update-banner.last-shown`) is device-wide too.
- **Within 5 minutes or on next foreground.** The query refetches every 5 minutes while the app runs, and `RemoteConfigProvider` refetches on every change to `active`. TanStack's `focusManager` is not wired to `AppState`, so the provider does this itself, for the config only.
- **Not blocking the navigator.** The forced-update screen replaces the Stack rather than sitting on top of it, so modal routes cannot cover it and there is no back gesture to dismiss it.
- **Banner day counted on first render.** `UpdateBanner` records the day when it first renders on Home, even if another tab is in front, so a customer can miss that day's banner. It still never shows more than once a day.
- **Hooks from `@sisig/api` and React copies.** `@sisig/api` resolves React 18.3 while the app uses 18.2, so the app calls its own `useQuery` with `appConfigQueryOptions` instead of a hook exported from the package.

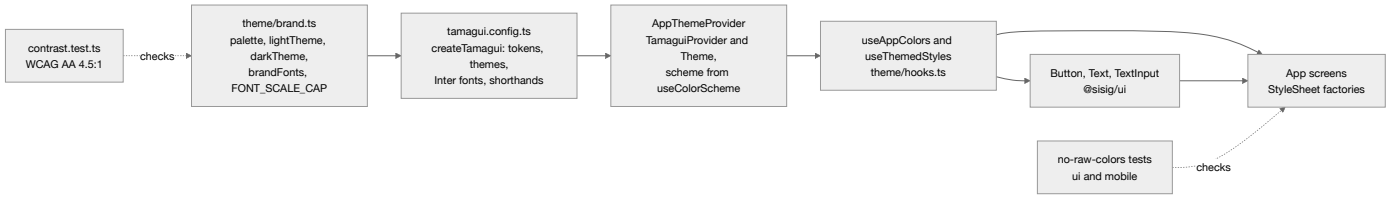
Limitations

- **Installed version.** `expo-application` is not installed, and adding a native module needs a dev client rebuild. `getInstalledVersion()` reads `Constants.expoConfig.version`, the `version` from `app.json` embedded at build time. That is the installed build's version while the app has no `expo-updates`. When DEVOPS-05 adds `expo-updates` (OTA updates and `runtimeVersion`), switch the seam to `Application.nativeApplicationVersion`. OTA JS compatibility is DEVOPS-05's job, not this gate's.
- **Sentry.** Problems are not sent to Sentry yet. PLAT-04 initialises Sentry and calls `setConfigReporter`.
- **Receipt scanning** has no screen yet, so `useFeature('receiptScan')` has no caller. `uber_eats_url` is validated but not shown.
- **Admin toggles** are not built (admin team). `is_admin()` reads the token, so a demoted admin keeps write access until their token expires.

THEME-01: Design System

Design Philosophy

Principle: semantic tokens define appearance. Components reference semantic names such as `primary` or `foregroundMuted`, never raw colours.



Brand values live only in `brand.ts`. The Tamagui config turns them into themes, the provider picks light or dark from the OS, and the hooks hand resolved colours to components and screens. Two tests guard the chain.

Tokens

`packages/ui/src/theme/brand.ts` has three layers:

- `palette`: the raw hex values. `tamagui.config.ts` registers them as Tamagui colour tokens, and the themes map semantic names to them. Components use the semantic names, not palette entries.
- `SEMANTIC_TOKENS`: 20 names (`background`, `surface`, `surfaceMuted`, `foreground`, `foregroundMuted`, `placeholder`, `border`, `primary`, `primaryPressed`, `primaryForeground`, `accent`, `accentForeground`, `success`, `successForeground`, `warning`, `warningForeground`, `danger`, `dangerForeground`, `bubble`, `bubbleForeground`).
- `lightTheme` and `darkTheme`: each maps every semantic name to a palette value.

All values are placeholders until the client's brand palette and fonts arrive.

Tamagui Configuration

- `packages/ui/src/tamagui.config.ts` calls `createTamagui` with size, space, radius and zIndex tokens, `palette` as the colour tokens, the `light` and `dark` themes, Tamagui shorthands, and `heading` and `body` fonts built with `createInterFont`. Settings include `allowedStyleValues: 'somewhat-strict'` and `disableRootThemeClass: true`.
- Tamagui is pinned at 1.144.4 to stay on React 18.2, which the app runs.
- The file declares `TamaguiCustomConfig` so Tamagui's types know the config:

```
declare module '@tamagui/core' {
  interface TamaguiCustomConfig extends Conf {}
}
```

Using colours in components

Components do not use Tamagui styled components today. They build React Native styles from resolved semantic colours:

```
import { StyleSheet } from 'react-native';
import { Text, useThemedStyles, type AppColors } from '@sisig/ui';

const createStyles = (c: AppColors) =>
  StyleSheet.create({
    title: { color: c.primary },
  });

export function Title(): JSX.Element {
  const styles = useThemedStyles(createStyles);
  return <Text style={styles.title}>Buy now</Text>;
}
```

`useAppColors()` reads the active Tamagui theme and returns every semantic colour. `useThemedStyles(factory)` rebuilds the styles only when the theme changes. Keep the factory at module level.

Dynamic Type Support

`Text` and `TextInput` from `@sisig/ui` set `maxFontSizeMultiplier` to `FONT_SCALE_CAP` (1.5) and clamp any larger value a caller passes. Text grows with the OS font size setting up to 1.5x, so layouts keep their shape. App code must use these wrappers, not the React Native components.

Light/Dark Theme Switching

`apps/mobile/src/components/AppThemeProvider.tsx` reads `useColorScheme()`, sets the Tamagui theme, and sets the status bar style. It switches while the app runs. There is no in-app override in v1. `app.json` sets `userInterfaceStyle: "automatic"`.

Fonts

The placeholder fonts are Inter 400 and 600 from `@expo-google-fonts/inter`. They are embedded at build time through the `expo-font` plugin in `apps/mobile/app.json` and loaded at runtime with `useFonts` in `app/_layout.tsx`. The font keys must match `brandFonts`.

Contrast and raw-colour tests

- `packages/ui/src/theme/contrast.test.ts` checks that both themes define every semantic token, and that 26 foreground/background pairs reach WCAG AA 4.5:1 in both themes. Only the decorative `border` token is not covered. It runs under `pnpm test`, which CI runs on every pull request.
- `packages/ui/src/no-raw-colors.test.ts` and `apps/mobile/src/no-raw-colors.test.ts` fail if a non-test `.ts` or `.tsx` file (other than `brand.ts`) contains a quoted hex colour such as `'#FFFFFF'`.

Rebrand

1. Change the values in `packages/ui/src/theme/brand.ts` (`palette`, `lightTheme`, `darkTheme`). Run `pnpm test`: the contrast test fails if any pair falls below AA.
2. Replace the Inter placeholders: update `brandFonts` in the same file, the font files listed under the `expo-font` plugin in `apps/mobile/app.json`, and the font imports in `apps/mobile/app/_layout.tsx`.
3. Rebuild and check:

```
pnpm -F @sisig/ui build
pnpm test
pnpm -F @sisig/mobile typecheck
```

Screens pick up the new colours with no component changes.

Claude Integration

`@sisig/claude` wraps `@anthropic-ai/sdk` (`^0.24`) in a `ClaudeClient` class and adds translation, code-analysis and document-generation helpers. In the app:

- `app/ai-chat.tsx`, `ai-journal.tsx`, `ai-summarizer.tsx` and `ai-translator.tsx` are UI only. They show fixed placeholder text and make no API call. `ai-chat` uses `ConversationManager` (local message history) and `ai-translator` uses `SUPPORTED_LANGUAGES`. No screen links to these routes.
- `src/hooks/useAI.ts` and `src/hooks/useImageAnalysis.ts` exist, but no screen uses them. `useImageAnalysis` never calls the API.

> **Warning:** `ClaudeClient` reads `ANTHROPIC_API_KEY` from `process.env` when no key is passed. Expo inlines only `EXPO_PUBLIC_*` variables, so that value is not in the app bundle, and `new ClaudeClient()` throws in the app. Do not work around this with an `EXPO_PUBLIC_` variable or by passing a key to `useAI`: anything in the bundle can be extracted from the installed app.

The package and its limits are documented in packages/claude/README.md and CLAUDE_SKILLS.md.

Integration Points

Database to frontend types

1. Migrations define the schema.
2. `pnpm db:types` generates `packages/api/src/database.types.ts` from the local Supabase stack.
3. Code imports the types with `import type { Database } from '@sisig/api'`, and the hooks use `SupabaseClient<Database>`.
4. The CI `types-drift` job regenerates the file and fails if it differs from the committed one.

Auth to database

The local stack config (`supabase/config.toml`) enables a custom access token hook, `private.custom_access_token_hook` , which adds an `app_role` claim to access tokens. See SUPABASE.md.

Edge Functions

`supabase/functions/whoami` answers `GET` with the caller's id, email and app role, after it verifies the caller's token against `/auth/v1/user` . Shared helpers for auth, CORS and error envelopes are in `supabase/functions/_shared` . The mobile app does not call any Edge Function yet. When it does, `normalizeError` already maps Functions errors.

Theme to components

See the diagram in THEME-01: Design System.

Development Workflow

Getting Started

The step-by-step setup, including the Android emulator, is in LOCAL-DEVELOPMENT.md. In short:

```
pnpm install

# Create apps/mobile/.env.local (gitignored) with
# EXPO_PUBLIC_SUPABASE_URL and EXPO_PUBLIC_SUPABASE_ANON_KEY

# Build the shared packages that Metro reads from dist/
pnpm turbo run build --filter='@sisig/mobile^...'

# First run: build and install the dev client, then start Metro
pnpm -F @sisig/mobile exec expo run:android

# Later runs: start Metro for the installed dev client
pnpm -F @sisig/mobile dev
```

The team runs the app on an Android emulator against the hosted Supabase dev project. iOS is not set up yet, because it needs a full Xcode install. A local Supabase stack (`pnpm db:start` , Docker required) is only needed for database work. See SUPABASE.md.

Creating a New Feature

1. **Schema (if needed).** Run `pnpm db:new <name>` , write the migration with RLS policies, then `pnpm db:reset` and `pnpm db:types` . Follow supabase/CONVENTIONS.md. Points, redemptions and orders change only through RPCs or Edge Functions.

2. **Data access.** Add keys to the `keys` factory. Add a `xxxQueryOptions(client)` function and a thin `useXxx` hook in `packages/api`, and throw `normalizeError(error)`. Rebuild the package.
3. **Screen.** A new tab is a folder `app/(tabs)/<name>/` with a `_layout.tsx` (a Stack using `useStackScreenOptions`) and an `index.tsx`, plus a `Tabs.Screen` in `app/(tabs)/_layout.tsx`. Add its path to `TAB_PATHS` in `src/navigation/auth-gate.ts` if it should be a `returnTo` target. Wrap the stack in `SignInRequired` if guests must not see it.
4. **Tests.** Put them under `src/`, never under `app/`.
5. **Checks.** Run `pnpm lint`, `pnpm typecheck`, `pnpm test` and, for schema changes, `pnpm db:test`.
6. **Branch and pull request.** Follow CONTRIBUTING.md.

Testing Strategy

Layer	Tool	Command
Database	pgTAP	`pnpm db:test`
Edge Functions	Deno test	`deno test --allow-all --no-config supabase/functions`
`@sisig/api`, `@sisig/claude`	Jest with ts-jest	`pnpm -F @sisig/api test`, `pnpm -F @sisig/claude test`
`@sisig/ui`, `@sisig/mobile`	Jest with jest-expo and React Native Testing Library	`pnpm -F @sisig/ui test`, `pnpm -F @sisig/mobile test`
Types	TypeScript	`pnpm typecheck`
Linting	ESLint	`pnpm lint`
Formatting	Prettier	`pnpm format:check`
App bundle	`expo export`	see the CI step in DEVOPS-02-CI.md

`pnpm test` runs every workspace's tests through Turborepo. The mobile and UI tests import `@sisig/*` from `dist/`, so build the packages first when you run a workspace's tests directly.

Deployment Strategy

Mobile App

1. **Local dev client.** `pnpm -F @sisig/mobile exec expo run:android` builds a debug app with a native project, installs it on the emulator or device, and starts Metro. `expo run:ios` does the same on iOS but needs Xcode, and has not been set up. Both commands generate `apps/mobile/android` and `apps/mobile/ios`. Those folders are local build output and are not committed.
2. **Release builds.** Not configured. The local Expo CLI rejects `expo build`, and there is no `eas.json`. App Store and Play Store builds through EAS (Expo Application Services) are planned (DEVOPS-04).

Database

Migrations apply when the local stack starts or resets (`pnpm db:start`, `pnpm db:reset`). The CI `database` job runs pgTAP tests and `pnpm db:lint` on every pull request. How changes reach the hosted project is described in SUPABASE.md.

Environment Management

Environment	Database	Used for
Local	`supabase start` (Docker)	Database work, pgTAP tests, type generation
Hosted dev project	Supabase cloud	Running the app on the Android emulator

No staging or production environment is defined in this repository.

Summary of Key Decisions

Decision	Rationale	Trade-off
RLS on every public table	Access control lives in the database	A missing policy means no access
Money in integer cents	No floating-point errors	Convert to dollars in the UI
Points change only through RPCs	One audited path for balance changes	Redeem waits on the LOY-03 RPC
Shared packages consumed from `dist/`	Metro, Jest and `tsc` read one built entry point	Build packages before starting Metro
Encrypted session, key in SecureStore	Tokens never sit in AsyncStorage as plaintext	More moving parts on sign-out
Purge on every `SIGNED_OUT`	A session that ends elsewhere also clears the device	Needs the in-flight guard to avoid a double purge
Persist only allowlisted query keys	No personal data in the unencrypted cache	New keys need a deliberate allowlist decision
Semantic tokens	Brand swaps without component changes	An extra layer of indirection
Expo Router file-based routes	Routes and deep links follow the file tree	Every file in `app/` becomes a route, tests included
Five-tab navigation with per-tab stacks	Fast access; each tab keeps its own history	Limited room for more top-level sections
Development build, not Expo Go	Native config and plugins are applied at build time	Each native change needs a new dev client build
Remote config is fail-open	A failed fetch or a bad value never locks customers out	A config the app cannot read cannot block an old version either

Next Steps

Planned work named in the code and docs:

- **AUTH-01 to AUTH-03:** the real welcome, login, sign-up, email verification and password reset screens.
- **AUTH-05:** member deletion and anonymisation (the ledger already allows it).
- **LOY-01, LOY-03, LOY-04:** RPCs that post points and handle redemptions. `useRedeemPoints` waits on LOY-03.
- **LOY-06:** client sign-off on loyalty rules; `loyalty_settings` values are placeholders until then.
- **ADM-01:** admin app framework. The admin team also builds the `app_config` toggles (PLAT-06).
- **PLAT-04:** Sentry setup, including `setConfigReporter` for remote-config problems.
- **DEVOPS-05:** OTA updates and `runtimeVersion`; switch `getInstalledVersion()` to `expo-application` then.
- **DEVOPS-04:** release builds through EAS.
- Client brand assets: swap the values in `packages/ui/src/theme/brand.ts`, then rebuild.

Related Documents

- SUPABASE.md: the Supabase guide (schema, migrations, RLS, Edge Functions).
- LOCAL-DEVELOPMENT.md: local setup and the Android emulator.
- DEVOPS-02-CI.md: the CI pipeline.
- supabase/README.md and supabase/CONVENTIONS.md: database commands and conventions.
- apps/mobile/README.md: the mobile app.
- CONTRIBUTING.md: branching, commits and pull requests.
- CLAUDE_SKILLS.md and packages/claude/README.md: the Claude package.

Document maintained by: Dev Team **Last updated:** November 2026

Supabase in Sisig

Source: docs/SUPABASE.md

This guide explains how Supabase works in the Sisig Eh! rewards app: auth and roles, row-level security, the schema, the loyalty data model, how migrations reach the hosted dev project, generated types, Edge Functions and tests. It describes `main` after PR #19 (DATA-04). Work that is planned but not built is marked with its Jira key, for example "planned, LOY-01".

Related documents:

- supabase/README.md: commands and quick reference for the `supabase/` folder.
- supabase/CONVENTIONS.md: the rules every migration, policy, function and test must follow.
- LOCAL-DEVELOPMENT.md: running the app on an Android emulator and connecting it to the hosted dev project.
- ARCHITECTURE.md: app architecture.
- DEVOPS-02-CI.md: the CI jobs.

Why this file exists: PRs are squash merged. The squash commit keeps the PR title and the commit messages, but not the PR description. Any rule or audit query that lived only in a PR description is gone from `main`, so it belongs here or in the migration itself.

Contents

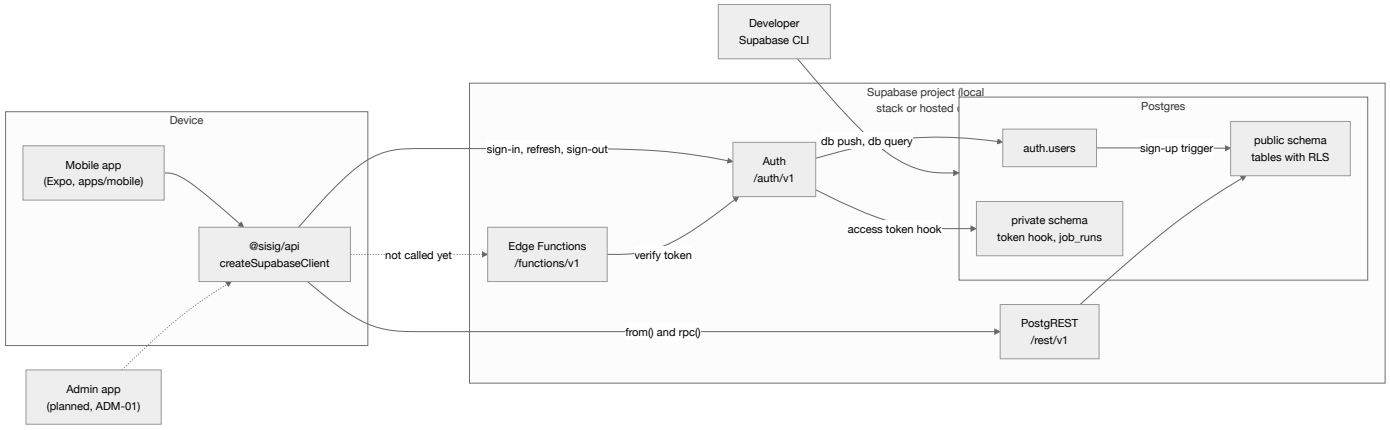
- What Supabase does for Sisig
- Environments
- Identity and roles
- Row-level security
- Schema reference
- Loyalty data model
- Remote config: `app_config`
- Rules for the LOY tickets
- Migration lifecycle
- Working with the hosted dev project
- Generated types
- Edge Functions
- Testing
- Troubleshooting
- Known issues

What Supabase does for Sisig

Supabase is a hosted Postgres database with services around it. Sisig uses these parts:

Part	What Sisig uses it for	State on `main`
Auth (Supabase's auth server, at `/auth/v1`)	Accounts, sessions, and access tokens (JWTs, signed JSON tokens) that carry an `app_role` claim	The app restores the session, refreshes tokens while in the foreground and purges the device on sign-out. The sign-in and sign-up screens are placeholders (planned, AUTH-01 to AUTH-03).
Postgres	Tables, constraints, triggers, functions and row-level security (RLS: per-row access rules enforced by the database)	12 tables in `public`, one in `private`. Extensions `uuid-ossp`, `pg_cron`, `pgtap`, `pgcrypto` and `pg_net` are enabled. No `pg_cron` job exists yet.
PostgREST (at `/rest/v1`)	Turns the `public` schema into a REST API. supabase-js `from('table')` and `rpc('function')` call it.	`@sisig/api` has `useMenu`, `useProduct`, `useMenuCategories` (products) and `useCreateOrder` (orders). No screen calls them yet. The app reads `app_config` on launch and foreground (PLAT-06).
Edge Functions (Deno, at `/functions/v1`)	Server-side code that needs a trusted runtime	One sample function, `whoami`. Not deployed anywhere.
Realtime, Storage, GraphQL	Nothing yet	Enabled by `supabase/config.toml` but unused. No storage bucket exists. CI starts the stack without Realtime.
Studio and the dashboard	Browsing data and settings	Local Studio at `http://127.0.0.1:54323`; the Supabase dashboard for the hosted project.

All client access goes through `createSupabaseClient` in `packages/api/src/client.ts`. The mobile app creates its single client in `apps/mobile/src/lib/supabase.ts` with the public URL and publishable key, and stores the session in encrypted storage.



The app talks to Supabase only through supabase-js. Auth writes `auth.users`, a trigger creates the member's rows in `public`, and the token hook in `private` adds the role claim. Dotted lines are not in use yet.

Environments

There are two environments today: the local stack on a developer machine and one hosted dev project. Staging and production do not exist yet (planned, DEVOPS-01).

	Local stack	Hosted dev project
What it is	Docker containers run by the Supabase CLI	Supabase cloud project "Sisig Eh", ref `dczjqhqpxbkikidtbntf`, Free plan, region us-east-1, created 2026-11-06
Start or reach it	`pnpm db:start` (needs Docker)	Link once with `pnpm exec supabase link`, then use `--linked` commands
API URL (Auth, REST, Functions)	`http://127.0.0.1:54321`	`https://dczjqhqpxbkikidtbntf.supabase.co`
Database	`127.0.0.1:54322`, Postgres 15 (`[db] major_version = 15`)	Postgres 17, reached with `db push --linked` and `db query --linked`
Dashboard	Studio, `http://127.0.0.1:54323`	Supabase dashboard
Auth emails	Mailpit (a local mail catcher), `http://127.0.0.1:54324`	The project's Auth email settings in the dashboard
How the schema arrives	`pnpm db:start` on a new database, or `pnpm db:reset`, applies every migration	`pnpm exec supabase db push --linked` after a merge
Seed data (`supabase/seed.sql`)	Applied after the migrations	Loaded once. Later changes to `seed.sql` are not applied (see Seed data).
Access token hook	Enabled by `[auth.hook.custom_access_token]` in `config.toml`	Must be enabled in the dashboard. Not confirmed yet.
Email confirmation before sign-in	Required (`enable_confirmations = true`)	Dashboard setting, not checked
Data	Disposable	Shared. Treat it as a shared environment: migrations applied there are never edited.

`config.toml` sets no ports, so the local stack uses the CLI defaults shown above. The CI `database` job prints the same ports.

History of the hosted dev project (observed on 2026-11-06):

1. It was first loaded from `main` with `supabase db push --include-seed`: the migrations up to `20261104180530` and the `seed.sql` of that time (the seven menu products).
2. The ten migrations from PR #19 were applied with `supabase db push --linked`, after a check that the legacy `users`, `loyalty_points` and `loyalty_transactions` tables held 0 rows.
3. The placeholder store row was inserted by hand with `supabase db query --linked`.

How the app chooses an environment

The mobile app reads two values from `apps/mobile/.env.local`. The file is gitignored. Never commit it.

```
# apps/mobile/.env.local
EXPO_PUBLIC_SUPABASE_URL=https://<ref>.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=<the project's publishable key, sb_publishable_...>
```

- For hosted dev, use the project URL and the publishable key from the dashboard.
- For the local stack, use `http://127.0.0.1:54321` and the publishable key that `pnpm db:start` or `pnpm exec supabase status` prints. An Android emulator cannot reach the host's `127.0.0.1`; see LOCAL-DEVELOPMENT.md.
- Only the publishable (anon) key goes here. The secret or service-role key must never reach the app. CI's `secrets` job runs gitleaks over the whole repo, then greps `apps/` and `packages/` for one fixed string, the HS256 JWT header (the legacy key format), and for the name `SUPABASE_SERVICE_ROLE_KEY`. The grep has no rule for `sb_secret_` keys, so do not rely on it to catch one.
- `apps/mobile/src/lib/supabase.ts` throws at launch if either value is missing.
- Expo inlines `EXPO_PUBLIC_*` values when it bundles the app, so restart the dev server after you change the file.

Identity and roles

Where identity lives

Object	Owner	Contents
`auth.users`	Supabase Auth	Credentials, email, confirmation state
`public.profiles`	Sisig	One row per user, same `id` as `auth.users`. Name, E.164 phone, birthday (month and day, no year), province, locale (`en-CA` or `fr-CA`), and `email`, a lower-cased copy of the auth email.
`public.user_roles`	Sisig	One row per user: `role`, `granted_by`, `granted_at`, `updated_at`. Kept apart from profiles so a role change never touches profile data.

Sign-up creates the profile and role

The trigger `on_auth_user_created` runs `public.handle_new_user()` after every insert into `auth.users`. The function is `SECURITY DEFINER` (it runs as its owner, not as the caller) and:

1. inserts `profiles (id, email)` with the email lower-cased;
2. inserts `user_roles (user_id, role)` with role `customer`.

Both inserts use `on conflict do nothing`. If the email is null, the profile insert fails (`23502`) and the whole sign-up fails, so no `auth.users` row is left behind. Sign-up metadata such as a first name is not copied into `profiles`.

The trigger `on_auth_email_changed` runs `public.sync_profile_email()` when `auth.users.email` changes, and copies the new address, lower-cased, into `profiles.email`.

Roles and the helper functions

The enum `public.app_role` has four labels. Each role includes the access of the ones before it.

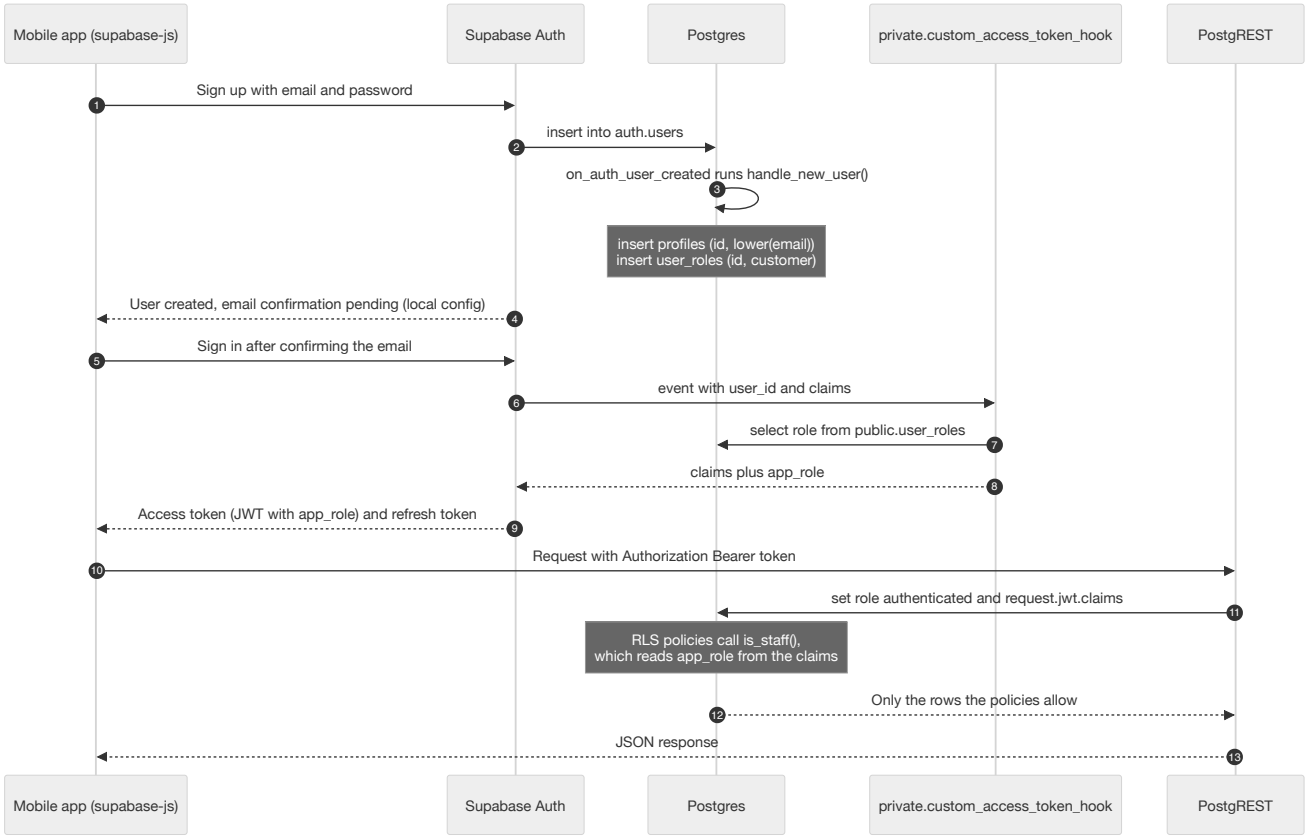
Role	What it adds	`is_staff()`	`is_admin()`	`is_owner()`
`customer`	Default for every sign-up. Own rows only.	false	false	false
`staff`	Reads every row that has a staff policy; inserts and updates products	true	false	false
`admin`	Can set `customer` or `staff` with `set_user_role`	true	true	false
`owner`	Can grant any role, including `admin` and `owner`	true	true	true
(no token, or no claim)	Anonymous request	false	false	false

`public.is_staff()`, `public.is_admin()` and `public.is_owner()` read the `app_role` claim from the request's JWT (`auth.jwt() ->> 'app_role'`). They do not read `user_roles`. They return `false`, never NULL, when the claim is missing.

The access token hook adds `app_role`

`private.custom_access_token_hook(event jsonb)` runs each time Auth issues or refreshes an access token. It reads the user's role from `public.user_roles` and adds it as `app_role` inside the event's `claims` object. A user without a `user_roles` row gets `app_role: null`.

- `supabase_auth_admin` (the role Auth uses) has `USAGE` on `private` and may execute it. `anon` and `authenticated` cannot call it.
- It is `SECURITY DEFINER`, so it reads `user_roles` as its owner and RLS does not filter the read.
- `config.toml` registers it for the local stack only. `db push` does not configure Auth. On each hosted project, enable it in the dashboard: **Authentication** → **Hooks** → **Customize Access Token** → `private.custom_access_token_hook`.
- Without the hook, nobody gets an `app_role` claim, so every staff, admin and owner is treated as a customer by RLS. Whether the hook is enabled on hosted dev has not been confirmed. Check it before you test anything that needs staff access.



Sign-up runs the trigger once. Every sign-in and every token refresh runs the hook again, so a role change reaches the token at the next refresh. The app screens for steps 1 and 5 are planned (AUTH-01 to AUTH-03); the database side exists today.

Changing a role: `set_user_role`

Clients cannot write `user_roles`: `anon` has no privileges on it, `authenticated` has no INSERT, UPDATE, DELETE or TRUNCATE privilege, and the table has no write policy. The only client path is the RPC `public.set_user_role(p_user_id uuid, p_role app_role)`. It is `SECURITY DEFINER`. `authenticated` may execute it; `anon` may not.

It checks the caller's role in the `user_roles` table, not in the JWT, so a demoted admin is refused at once, even while their token still says `admin`. The checks run in this order:

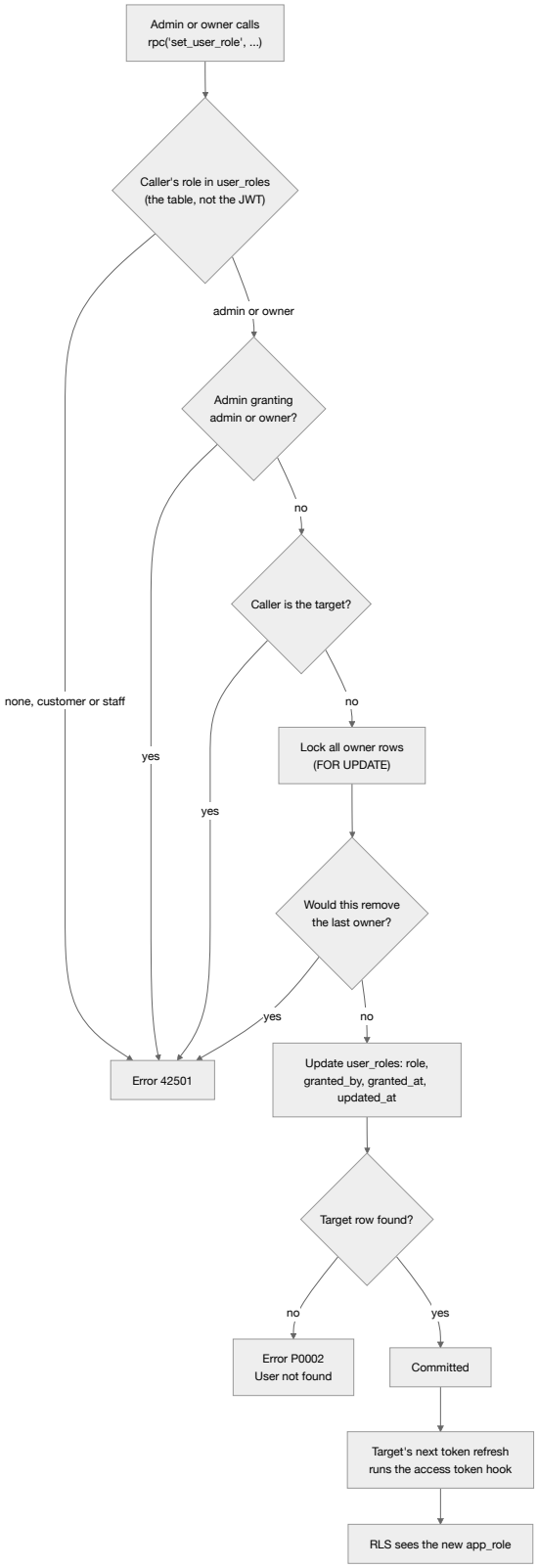
Check	Result when it fails
The caller has a `user_roles` row	`42501` "Caller has no role"
The caller is `admin` or `owner`	`42501` "Permission denied: insufficient role"
An admin is not granting `admin` or `owner`	`42501` "Admin cannot grant admin or owner role"
The caller is not changing their own role	`42501` "Cannot change your own role"

Check	Result when it fails
(locks every owner row with `FOR UPDATE`, so two demotions cannot run at once)	
The change does not remove the last owner	`42501` "Cannot demote the last remaining owner"
The target has a `user_roles` row	`P0002` "User not found"

On success it sets `role`, `granted_by` (the caller), `granted_at` and `updated_at`.

What this means in practice:

- An **owner** can give any role, including `owner`, to anyone else, and can demote another owner while at least two owners exist.
- An **admin** can set `customer` or `staff` on anyone else. The function does not stop an admin from demoting another admin, or an owner who is not the last owner.
- **Staff** and **customers** cannot change roles.



`set_user_role` takes effect in the table at once. RLS policies see it only when the target's token is refreshed, up to one token lifetime later. `config.toml` does not set `jwt_expiry`, so the local stack uses the CLI default of one hour.

The first owner

`set_user_role` needs an existing admin or owner, and no migration creates one. Set the first owner once per environment with SQL run as the database owner. This is also how the tests set up roles.

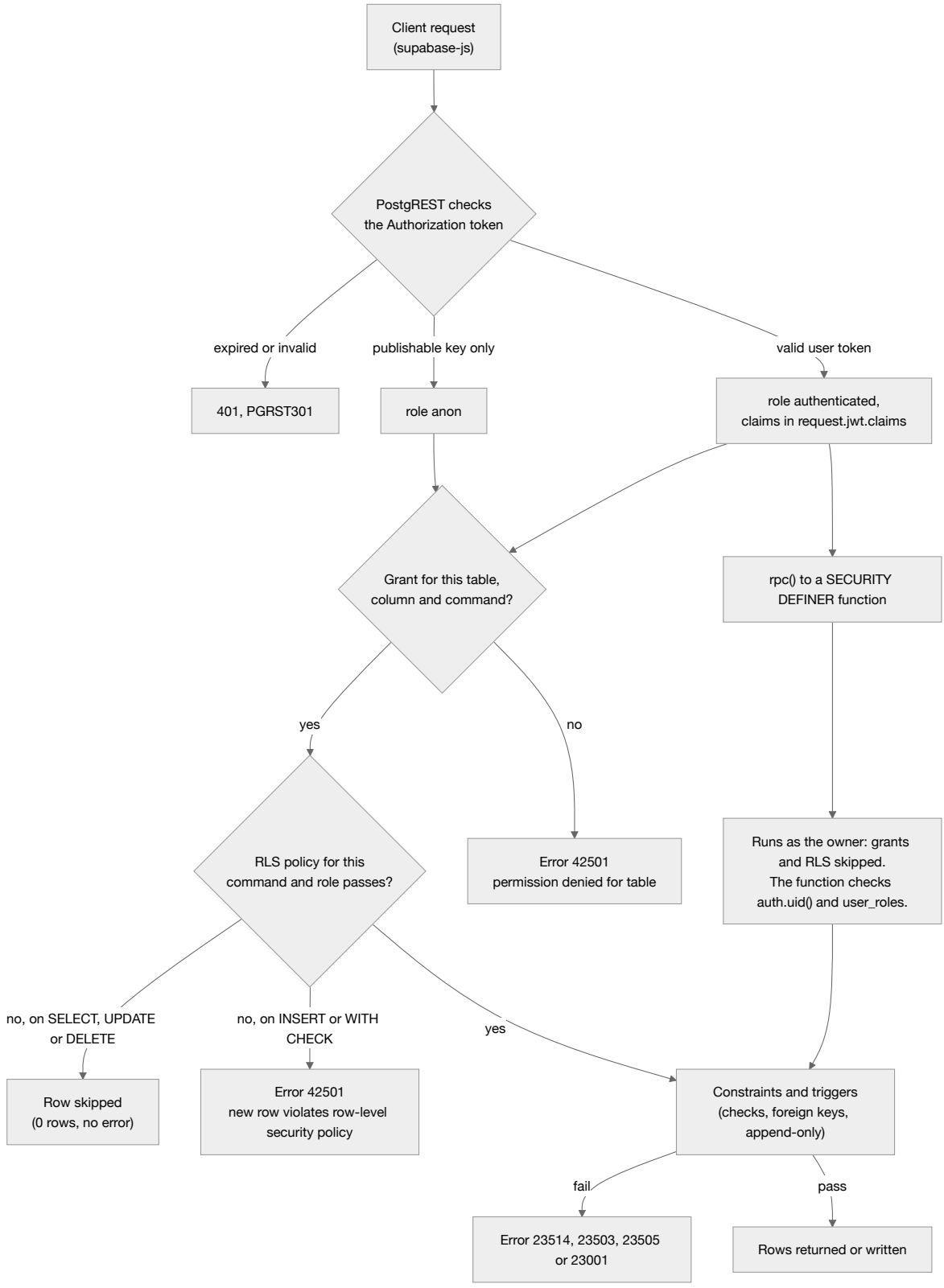
```
pnpm exec supabase db query --linked \
  "update public.user_roles set role = 'owner' where user_id = (select id from public.profiles where
email = '<email>') returning user_id, role"
```

For the local stack, use `--local` instead of `--linked`. The user then signs out and back in, so their next token carries `app_role: owner`. After that, use `set_user_role` for every other change.

Row-level security

Principles

1. **Every `public` table has RLS enabled.** A table with RLS and no matching policy returns no rows and accepts no writes.
2. **Two gates.** Postgres checks table and column privileges (grants) first, then RLS policies. A request must pass both.
3. **Supabase grants a lot by default.** New `public` tables get full privileges for `anon`, `authenticated` and `service_role`. Migrations revoke what they do not want.
4. **Staff checks read the JWT.** Policies call `is_staff()`, which is fast and avoids policy recursion, but can be stale for one token lifetime. Writes that need a privilege re-check `user_roles` inside a `SECURITY DEFINER` function, as `set_user_role` does. The one exception is `app_config`, whose admin write policies trust the token alone (see item 6).
5. **Tables added by DATA-04 and DATA-18 are read-only for every API role.** No API role can write `locations`, `loyalty_settings`, `points_ledger`, `point_balances`, `rewards` or `reward_redemptions`. `authenticated` and `service_role` hold SELECT on all six. `anon` holds SELECT on `locations`, `loyalty_settings` and `rewards` only, and nothing on the other three. Writes are meant to go through `SECURITY DEFINER` RPCs (planned, LOY-01/03/04).
6. `app_config` **is written directly by admins, through RLS.** `authenticated` holds INSERT, UPDATE and DELETE on it, and RLS lets the write through only when `(select public.is_admin())` is true (admin or owner, read from the token). `anon` and `service_role` can only read it. Because the check reads the token, a demoted admin can write until their token expires; the table holds no personal data, and the app only accepts store and Uber Eats links on their official hosts, so this is accepted for now. See Remote config.
7. `service_role` **skips RLS but not grants.** It is the role behind the secret (service-role) key. It keeps default privileges on the older tables (`profiles`, `user_roles`, `products`, `orders`, `order_items`), so it can write them directly. On the six tables in item 5 and on `app_config` it can only read.
8. **Roles:** `anon` is a request with only the publishable key. `authenticated` is a request with a signed-in user's token. Both reach Postgres through PostgREST.



A request passes the grant check, then RLS, then constraints and triggers. A `SECURITY DEFINER` function skips the first two gates, so it must check the caller itself.

Policy patterns in use

Pattern	Example	Used on
Own rows	`(select auth.uid()) = user_id`; older policies use `auth.uid() = id`	`profiles`, `user_roles`, `orders`, `points_ledger`, `point_balances`, `reward_redemptions`
Staff read everything	`(select public.is_staff())`; `profiles` and `user_roles` use `public.is_staff()`	All tables except `order_items`, `locations`, `loyalty_settings` and `app_config`
Public read	`to anon, authenticated using (true)`	`locations`, `loyalty_settings`, `app_config`
Admin write	`to authenticated` with `using` and `with check ((select public.is_admin()))`	`app_config`
Public read with a filter	`is_available = true`; active, not archived and inside the date window	`products`, `rewards`
Through the parent row	`exists (select 1 from public.orders where id = order_items.order_id and user_id = auth.uid())`	`order_items`
Own insert	`with check (auth.uid() = user_id)`	`orders`
No write policy	Writes only through a `SECURITY DEFINER` function	`user_roles` and the six DATA-04/DATA-18 tables

Wrapping a function in `(select ...)` makes Postgres evaluate it once per statement instead of once per row. The newer policies do this.

Who can read and write each table

"Member" means a signed-in user acting on their own rows, whatever their role. "Staff" means `is_staff()` is true: staff, admin or owner, according to the token.

Table	anon reads	Member reads	Staff reads	Client writes allowed today	Intended write path
`profiles`	nothing	own row	all	UPDATE of own row, only `first_name`, `last_name`, `phone_e164`, `birthday_month`, `birthday_day`, `province`, `locale`	Sign-up trigger creates the row; email trigger keeps `email` in sync; `birthday_locked_at` is server-owned (no writer yet)
`user_roles`	nothing (no grants)	own row	all	none	`set_user_role` RPC; sign-up trigger inserts `customer`

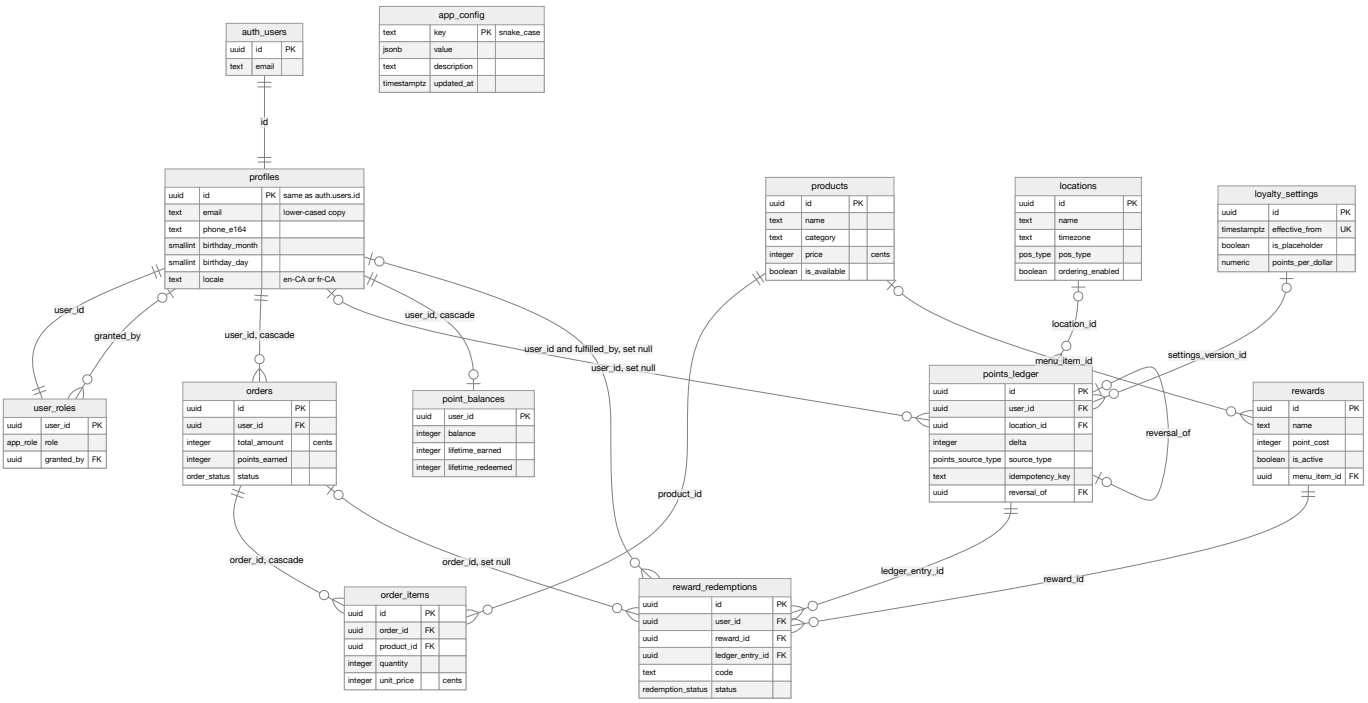
Table	anon reads	Member reads	Staff reads	Client writes allowed today	Intended write path
`products`	available rows	available rows	all	staff INSERT and UPDATE; no DELETE policy	Staff through PostgREST
`orders`	nothing	own orders	all	member INSERT of own order, with any `status`, `total_amount` and `points_earned` (only non-negative checks apply); no UPDATE or DELETE policy	Not settled (planned, DATA-06)
`order_items`	nothing	items of own orders	items of own orders only (no staff policy)	none: no INSERT policy	Not settled (planned, DATA-06)
`locations`	all	all	all	none (SELECT only)	Migrations; admin RPCs later
`loyalty_settings`	all	all	all	none (SELECT only)	A new migration per version (planned, LOY-06)
`points_ledger`	nothing (no grants)	own entries	all	none (SELECT only)	`SECURITY DEFINER` RPCs (planned, LOY-01/03/04)
`point_balances`	nothing (no grants)	own row	all	none (SELECT only)	Same RPCs, in the same transaction as the ledger insert
`rewards`	live rewards	live rewards	all	none (SELECT only)	Migrations or admin RPCs (planned, ADM work)
`reward_redemptions`	nothing (no grants)	own redemptions	all	none (SELECT only)	Redemption RPC (planned, LOY-03)
`app_config`	all	all	all	INSERT, UPDATE and DELETE by admins and owners only (`is_admin()`); members and staff are refused by RLS, `anon` by grants	SQL in the dashboard for now (runbook); admin toggles later (admin team)

Functions in `public` and `private`

Function	Runs as	Who may execute	Purpose
`public.is_staff()`, `is_admin()`, `is_owner()`	definer	any role (execute not revoked)	Read the `app_role` claim
`public.set_user_role(p_user_id, p_role)`	definer	`authenticated` (not `anon`)	Change a role (rules above)
`public.current_loyalty_settings(p_at)`	invoker (the caller)	`anon`, `authenticated`, `service_role`	Loyalty rules in effect at a time
`public.handle_new_user()`, `public.sync_profile_email()`	definer	triggers on `auth.users`	Sign-up and email sync
`private.custom_access_token_hook(event)`	definer	`supabase_auth_admin`	Add `app_role` to tokens
`private.enforce_points_ledger_append_only()`	invoker	trigger on `points_ledger`	Block updates and deletes

Every function sets `search_path = ''` and schema-qualifies what it touches. The generated types list the five functions clients can call: the three helpers, `set_user_role` and `current_loyalty_settings`.

Schema reference



The current `public` schema and its link to `auth.users`. Labels name the foreign key column and its delete action. `app_config` stands alone: it has no foreign keys. `private.job_runs` has no foreign keys and is not shown.

Tables

Table	Purpose	Notable rules	Added by
`profiles`	Member details	E.164 phone check; birthday month and day both set or both null, and the day must exist in the month (29 February allowed); unique lower-cased email	DATA-02
`user_roles`	One role per member	Default `customer`; clients write it only through the sign-up trigger and `set_user_role` (`service_role` keeps default write privileges)	DATA-02
`products`	Menu items	`price` in cents, `price >= 0`	DATA-01
`orders`	Customer orders	`user_id` references `profiles` (on delete cascade); `total_amount >= 0`, `points_earned >= 0`	DATA-01, DATA-04
`order_items`	Order lines	`quantity > 0`, `unit_price >= 0`, deleted with the order	DATA-01
`locations`	Stores	E.164 phone; `timezone` defaults to `America/Edmonton`; `hours` as JSON in the store's time zone; ordering off by default	DATA-18 (slim)
`loyalty_settings`	Versioned programme rules	Unique `effective_from`; one `-infinity` placeholder baseline	DATA-04
`points_ledger`	Every change to a member's points	Append-only; see below	DATA-04
`point_balances`	One chain-wide balance per member	All three counters `>= 0`; no location column	DATA-04
`rewards`	Reward catalogue	`point_cost > 0`; visible to everyone only while active, not archived and inside `starts_at` / `ends_at`	DATA-04
`reward_redemptions`	Issued reward codes	Code unique among issued redemptions; `points_spent > 0`	DATA-04
`app_config`	Remote config: kill switches, update gate, links	One row per `key` (snake_case check `app_config_key_format`); `value` is JSON; ten fail-open baseline rows	PLAT-06
`private.job_runs`	Log for scheduled jobs	Status `running`, `success` or `error`; no job writes it yet	DATA-01

The money and points checks on `products`, `orders` and `order_items` were added `NOT VALID`: Postgres checks new and updated rows, but not rows that existed before. A later migration will validate them (see Audit queries).

No table has a trigger that maintains `updated_at`. Only `set_user_role` sets it, on `user_roles`.

Enums

Enum	Labels
`app_role`	`customer`, `staff`, `admin`, `owner`
`order_status`	`pending`, `confirmed`, `preparing`, `ready`, `completed`, `cancelled`
`pos_type`	`square`, `toast`
`loyalty_rounding_mode`	`down`, `nearest`, `up`
`loyalty_earn_base`	`pre_tax_subtotal`, `post_tax_total`
`loyalty_expiry_policy`	`none`, `inactivity`
`points_source_type`	`pos_purchase`, `qr_counter`, `receipt`, `admin_adjust`, `reward_redeem`, `redemption_refund`, `offer_bonus`, `expiry`, `reversal`, `signup_bonus`
`redemption_status`	`issued`, `fulfilled`, `cancelled`, `expired`
`redemption_channel`	`counter`, `pickup`

Removed objects

PR #19 dropped the legacy `public.users` table, the `public.user_role` enum, and the `public.loyalty_points` and `public.loyalty_transactions` tables. The loyalty tables were dropped by product-owner decision, without migrating their rows. The `users` rows were not migrated either; roles are granted again with `set_user_role`. The old migrations stay in `supabase/migrations/` as history.

Loyalty data model

DATA-04 added the loyalty tables, constraints, indexes, read policies and grants. It added no posting, redemption or reversal logic. That logic is planned for LOY-01, LOY-03 and LOY-04, as `SECURITY DEFINER` RPCs. Until then, `useRedeemPoints` in `packages/api/src/mutations.ts` throws on purpose.

`points_ledger`: append-only

Each change to a member's points is one row. Rows are never updated or deleted. A mistake is fixed by adding a new row.

Column	Meaning
`user_id`	The member. Becomes NULL when the member is deleted.
`location_id`	Store where the points were earned. Required for in-store sources.
`delta`	Points added (positive) or removed (negative). Never 0; between -1,000,000 and 1,000,000.
`source_type`, `source_id`	What caused the entry, and the id of that record (POS transaction, receipt, offer).

Column	Meaning
`idempotency_key`	Unique per `source_type`. Stops the same event being posted twice.
`reversal_of`	For `reversal` rows only: the entry being reversed.
`reason`, `metadata`	Free text and JSON. No personal data.
`settings_version_id`	The `loyalty_settings` version used.
`created_by`	Staff or admin actor, set only for their actions. No foreign key, so the row outlives the actor.

Sources fall into two groups:

- **In-store**, `location_id` required: `pos_purchase`, `qr_counter`, `receipt`.
- **Chain-level**, `location_id` optional: `signup_bonus`, `offer_bonus`, `expiry`, `admin_adjust`, `reward_redeem`, `redemption_refund`, `reversal`.

The index on `(user_id, created_at desc, id desc)` serves member history, newest first, with a stable order for keyset pagination (paging by the last row seen).

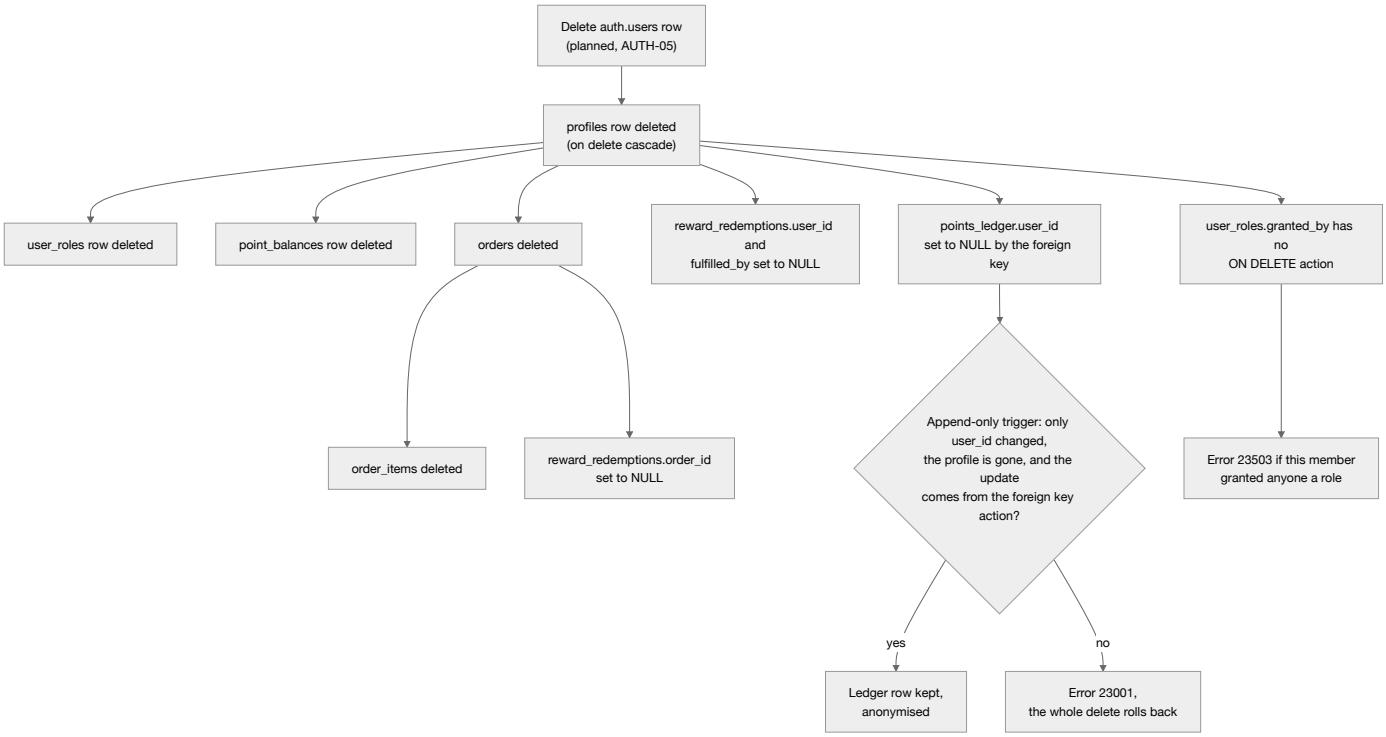
The trigger `points_ledger_append_only` runs `private.enforce_points_ledger_append_only()` before every UPDATE or DELETE and raises `23001` ("points_ledger is append-only: UPDATE is not allowed"), whoever runs the statement, the table owner `postgres` included. The API roles hold at most SELECT (`authenticated` and `service_role` hold SELECT, `anon` nothing), so they fail earlier with `42501`. TRUNCATE is revoked from the API roles because it skips row triggers. The table owner can still truncate.

Idempotency keys and reversals

- `unique (source_type, idempotency_key)` on `points_ledger`: a second insert with the same key and source type fails with `23505`. The same key string under another source type is allowed.
- `unique (user_id, idempotency_key)` on `reward_redemptions`: a retried redemption request cannot issue twice.
- A `reversal` row must set `reversal_of`, and only a `reversal` row may set it.
- `unique (reversal_of)`: an entry can be reversed at most once.
- The database does not check that a reversal's `delta` cancels the original, or that each source type has the right sign. Those rules belong to the RPCs (planned, LOY-01/03/04).

Member deletion and anonymisation

Deleting a member (planned, AUTH-05) deletes the `auth.users` row. Nothing in the app does this yet; the tests do it directly. The foreign keys then decide what happens to each table.



A member's balance, role and orders are deleted. Their ledger entries and redemptions are kept with `user_id` set to NULL. The `granted_by` branch is a known gap that AUTH-05 must handle.

The trigger lets an update through only when all of these hold:

1. it is an UPDATE that changes `user_id` from a value to NULL;
2. every other column is unchanged;
3. the profile `user_id` pointed at no longer exists;
4. it runs inside a trigger (`pg_trigger_depth() > 1`), as a foreign key action does.

A direct `update ... set user_id = null` fails, and so does a trigger that clears `user_id` while the profile still exists.

Anonymisation clears only `user_id` . Any member identifier stored elsewhere in the row (email, phone, user id inside a key) would survive. That is why the column comments on `source_id` , `idempotency_key` , `reason` and `metadata` forbid personal data.

`point_balances`

One row per member, chain-wide: points earned at one store can be spent at another. `balance` , `lifetime_earned` and `lifetime_redeemed` are all `>= 0` . The LOY RPCs will maintain the row from the ledger; no trigger does. No row is created at sign-up, so a member has no balance row until an RPC writes one.

`rewards`

A reward costs `point_cost` points (`> 0`). Everyone sees a reward while `is_active` is true, `archived_at` is null and now is inside the optional `starts_at` / `ends_at` window. Staff see every reward. `is_active` defaults to

false. `stock` and `per_user_limit` are stored but not enforced; the redemption RPC will enforce them (planned, LOY-03). `menu_item_id` optionally links the reward to the product it gives away.

`reward_redemptions`: one-time codes

A redemption is issued with a short `code` that staff look up at the counter or at pickup. It spends points through exactly one ledger entry (`ledger_entry_id` , required).

- `status` is `issued` , `fulfilled` , `cancelled` or `expired` . The database adds no state rules; LOY-03 will.
- A partial unique index makes `code` unique among `issued` rows only. Once a redemption is fulfilled, cancelled or expired, its code can be issued again. A separate index serves code lookups.
- `expires_at` is required. Nothing moves an expired code to `expired` automatically.
- `channel` is `counter` or `pickup` . `order_id` links a pickup redemption to its order.
- `order_id` and `fulfilled_by` are indexed because every member deletion follows them.

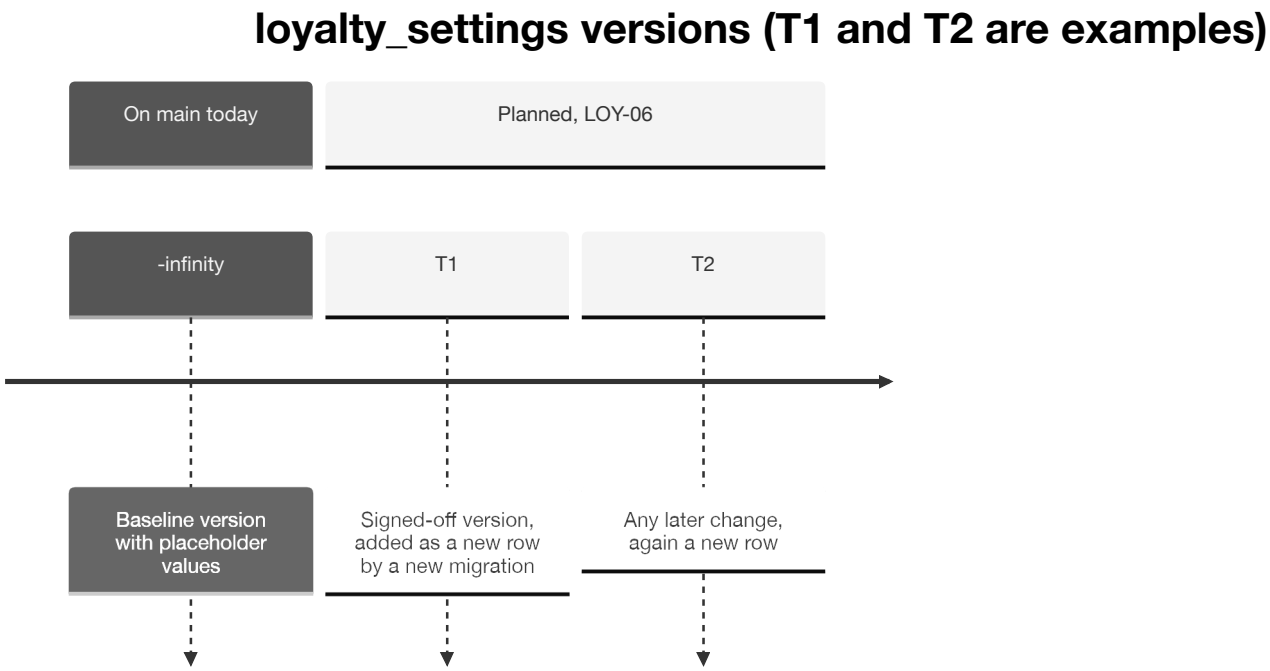
`loyalty_settings` and `current_loyalty_settings(at)`

Each row is one version of the programme rules. A version is in effect from its `effective_from` (inclusive) until the next version starts. A new version is added by a new migration; rows are not edited. Nothing in the database blocks an edit by the table owner, so this is a convention.

The baseline row starts at `-infinity` , so every timestamp has a version. Its values are placeholders until the client signs off the rules (planned, LOY-06). `is_placeholder` is true and the label starts with `PLACEHOLDER` . The migration inserts it, so it exists on every environment, hosted dev included.

Setting	Baseline (placeholder) value
`points_per_dollar` , `rounding_mode` , `earn_base`	1, `down` , `pre_tax_subtotal`
`min_spend_cents`	0
`daily_points_cap`	200
`max_points_per_transaction` , `daily_receipt_claims_cap`	none
`purchase_age_limit_days`	14
`signup_bonus_points`	25
`expiry_policy` , `inactivity_months` , `expiry_warning_days`	`none` , none, empty
`redemption_code_ttl_minutes`	15
`multipliers`	`{}`

`public.current_loyalty_settings(p_at timestamptz default now())` returns the version with the latest `effective_from` at or before `p_at` . The unique `effective_from` and the baseline mean exactly one row comes back for every non-null time. The function is `STRICT` , so a NULL argument returns NULL. It runs as the caller, and `anon` , `authenticated` and `service_role` may call it.



`current_loyalty_settings(at)` returns the baseline for any `at` before T1, the T1 version for T1 up to (not including) T2, and the T2 version from T2 on.

PostgREST sends the baseline's `effective_from` as the string `"-infinity"`. The generated type is `string`, and `new Date("-infinity")` is an invalid date. Clients must handle that value explicitly.

The placeholder location

DATA-18 was done "slim": Sisig has one store, so there is one `locations` row.

- Locally, `supabase/seed.sql` inserts `PLACEHOLDER Sisig Edmonton` (id `10ca7100-0000-4000-8000-000000000001`, Edmonton coordinates, `America/Edmonton`, empty hours, ordering off). The seed comment says not to add a second store.
- On hosted dev the same row was inserted by hand, because the seed is not re-run there.
- Real store data comes from the client.
- Deferred until a second store exists: per-location staff RLS (DATA-18 AC4) and per-location settings with a chain fallback (DATA-18 AC5). `location_id` on orders and receipts belongs to DATA-06 and DATA-07.

Remote config: `app_config`

PLAT-06 added `public.app_config` so the restaurant can switch off ordering or receipt scanning and force outdated apps to update without shipping a build. The app side is in ARCHITECTURE.md. The procedure for changing values is in the remote config runbook.

- **Shape.** One row per setting: `key` (primary key, snake_case), `value` (`jsonb`, not null), `description` (not null, shown in Studio) and `updated_at`. No trigger maintains `updated_at`, so every UPDATE sets it.
- **Open key set.** The table does not restrict which keys exist, so a newer app can read a key that older apps ignore. The app validates every known key with zod and falls back to its bundled default when a value is malformed. It ignores unknown keys. Both cases are reported.
- **Baseline in the migration, not `seed.sql`.** Every environment needs the rows, and CONVENTIONS.md puts such data in a migration. The values are fail-open and match the defaults bundled in the app.
- **Access.** Everyone, `anon` included, can read it: the app needs it before sign-in. Only admins and owners can write it through the API, through RLS with `(select public.is_admin())`. `service_role` can only read.
- **Stale role claim.** `is_admin()` reads the token, so an admin who was demoted keeps write access until their token expires (up to one token lifetime). The table holds no personal data, so this is accepted. The dashboard SQL editor runs as `postgres` and does not depend on the token.

Key	JSON type	Baseline	Effect in the app
`ordering_enabled`	boolean	`true`	`false` hides the Order tab's menu and checkout and shows the paused message
`ordering_paused_message`	string (1 to 280 characters) or null	`null`	Message while ordering is off; null uses the app's default text
`receipt_scan_enabled`	boolean	`true`	Kill switch for receipt scanning; read by `useFeature('receiptScan')`. No receipt-scan screen exists yet.
`uber_eats_url`	https link on `ubereats.com` or null	`null`	Validated and available to screens; no screen shows it yet
`min_supported_version_ios`	`"x.y.z"`	`"0.0.0"`	iOS installs below it get the blocking update screen
`min_supported_version_android`	`"x.y.z"`	`"0.0.0"`	Android installs below it get the blocking update screen
`recommended_version`	`"x.y.z"`	`"0.0.0"`	Installs below it get a dismissible banner on Home, at most once a day
`store_url_ios`	https link on `apps.apple.com` or null	`null`	Link on the update screen and banner. Without it, iOS is never blocked
`store_url_android`	https link on `play.google.com` or null	Google Play listing for `com.sisig.mobile`	Same, for Android
`announcement_message`	string (1 to 280 characters) or null	`null`	Banner on the Home tab

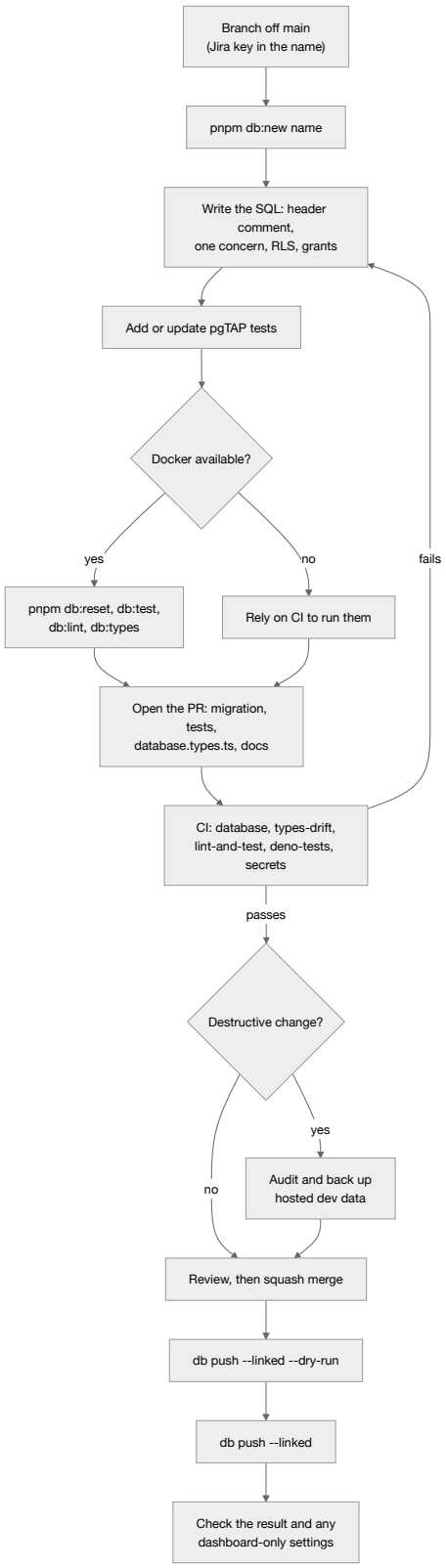
`public.locations` also has `ordering_enabled` and `uber_eats_url` (DATA-18). The app reads neither today. `app_config.ordering_enabled` is the chain-wide kill switch. How it combines with a per-store setting is for DATA-06 to decide.

Rules for the LOY tickets

These rules apply to LOY-01 (earning), LOY-03 (redemption), LOY-04 (reversals and adjustments), DATA-06 (orders) and any other work that writes loyalty data.

1. **Write only through** `SECURITY DEFINER` **RPCs** added in migrations. No API role has a write grant on `locations`, `loyalty_settings`, `points_ledger`, `point_balances`, `rewards` or `reward_redemptions`: `authenticated` and `service_role` hold SELECT only, and `anon` holds SELECT on `locations`, `loyalty_settings` and `rewards` and nothing on the rest. Do not add write grants or write policies so that a client or an Edge Function can write these tables directly. Follow the function rules in CONVENTIONS.md.
2. **Re-check roles in** `user_roles` for staff and admin actions. The `app_role` claim can be one token lifetime out of date. Set `created_by` only for staff and admin actions.
3. **Keep member identifiers out of identifier columns.** `points_ledger.source_id`, `points_ledger.idempotency_key`, `points_ledger.reason`, `points_ledger.metadata` and `reward_redemptions.idempotency_key` must not contain a user id, email, phone or any other member identifier. Anonymisation clears only `user_id`, so anything else in the row outlives the member.
4. **Never trust client-written order fields.** Members can insert `orders` rows directly today (the "Users can create orders" policy), with any `status`, `total_amount` and `points_earned` the client sends; only the non-negative checks apply. LOY-01 and DATA-06 must never compute points from, or award points because of, client-written `orders.status`, `orders.total_amount` or `orders.points_earned`.
5. **Skip zero-point postings.** `delta <> 0` rejects them. An earn that rounds to 0 points must not insert a row.
6. **Write the ledger row and the balance in one function call**, so they commit together. The `balance >= 0` check rejects a spend that would make the balance negative.
7. **Use the settings in effect at the time of the event.** Call `current_loyalty_settings(<event time>)` rather than relying on `now()`, and store the version's id in `settings_version_id`. The baseline is a placeholder until LOY-06.
8. **Handle** `"-infinity"`**.** The baseline `effective_from` reaches JavaScript as the string `"-infinity"`, which `Date` cannot parse.
9. **Reverse once, and negate correctly.** The database allows one reversal per entry but does not check its sign or size.
10. **Check** `expires_at` **on every code lookup.** Nothing expires codes automatically, and an `issued` row keeps its code reserved until its status changes.
11. **Do not hard-code the placeholder store's id.** Real stores will be new rows.
12. **Add pgTAP tests for each RPC** that call it as `anon`, as a member and as staff, and check both the success path and each rejection.

Migration lifecycle



A migration goes from a branch through CI and a squash merge, then someone pushes it to the hosted dev project by hand. Nothing deploys migrations automatically.

Steps

1. Create a branch off `main`, named as CONTRIBUTING.md describes.
2. Run `pnpm db:new <snake_case_name>`. It creates `supabase/migrations/<timestamp>_<name>.sql`. Never pick a timestamp by hand.
3. Write the SQL. Start with a header comment: the Jira key, the one concern, why, any data impact, and "Forward-only: no existing migration is edited." Follow CONVENTIONS.md.
4. Add or update pgTAP tests in `supabase/tests/database/`, and update each `plan(n)`.
5. If you have Docker, run `pnpm db:reset`, `pnpm db:test`, `pnpm db:lint` and `pnpm db:types`. Without Docker, push the branch and let CI run them.
6. Commit the migration, the tests, the regenerated `packages/api/src/database.types.ts` and doc updates (this guide, and the schema list in supabase/README.md) in one PR.
7. Fill in the PR template's **Migration Impact** section. Put audit queries and data-loss notes in the migration header or in this guide as well, because the squash merge drops the PR description.
8. CI must pass: `database` (pgTAP tests and schema lint), `types-drift`, `lint-and-test`, `deno-tests` and `secrets`. See DEVOPS-02-CI.md.
9. For a destructive change, follow the rules below before you merge.
10. Squash merge to `main`.
11. From an up-to-date `main`, push to hosted dev: Apply migrations.

Destructive migrations

A migration is destructive when it can lose or reject existing data: dropping a table, column or type, changing a column type, adding NOT NULL, adding a constraint or foreign key that checks existing rows, or rewriting data.

1. **Audit hosted dev before you merge.** Run the queries that count the rows the migration would drop or reject, with `pnpm exec supabase db query --linked "<sql>"`. Record the result in the PR.
2. **Back up the affected data before you merge.** Export the rows to a file outside the repo. `pnpm exec supabase db dump --linked --data-only -f <file>` is one option. If it does not work on your machine, save the output of `pnpm exec supabase db query --linked "select * from public.<table>"`. The file holds personal data: keep it out of git and delete it when it is no longer needed.
3. **Put any backfill in the same PR, before the destructive migration.** After the merge there is no slot for it: migrations are forward-only, and `db push` stops when a local migration sorts before the last one applied on the remote (only `--include-all` overrides that, and it should not be used to rewrite history on a shared database).
4. **Fail loudly instead of deleting quietly.** For example, `20261106230040_retire_legacy_users.sql` adds a validated foreign key, so it fails with `23503` if an order has no profile, rather than deleting the order.
5. **Add checks that old rows might fail as** `NOT VALID`, and validate them in a later migration once the data is clean. `20261104180526_money_points_checks.sql` adds its checks this way; the validating migration does not exist yet.
6. **Get product-owner sign-off for data loss.** The legacy table drops in PR #19 record that decision in their migration headers.

Audit queries for the open NOT VALID checks

`20261104180526_money_points_checks.sql` says to run "the audit queries in the PR description" before applying it, and to validate the checks later. Those queries are not on `main`. These queries count the rows each check would reject. Run them before the migration that validates the checks (planned; no Jira key is recorded in the code):

```
select 'products.price' as rule, count(*) from public.products where price < 0
union all
select 'orders.total_amount', count(*) from public.orders where total_amount < 0
union all
select 'orders.points_earned', count(*) from public.orders where points_earned < 0
union all
select 'order_items.unit_price', count(*) from public.order_items where unit_price < 0;
```

Every count must be 0 before `alter table ... validate constraint ...` runs.

Working with the hosted dev project

The Supabase CLI 2.122.0 is a root devDependency (locked in `pnpm-lock.yaml`), so run it as `pnpm exec supabase ...`. CI installs the same version.

One-time setup per machine

1. Install dependencies from the repo root:

   ```
   pnpm install
   ```

2. Sign the CLI in to your Supabase account. Your account needs access to the project.

   ```
   pnpm exec supabase login
   ```

3. Link the repo to the hosted dev project. The CLI stores the link in `supabase/.temp/`, which is gitignored.

   ```
   pnpm exec supabase link --project-ref dczjqhqpxbkikidtbntf
   ```

4. Check that the project is listed and linked:

   ```
   pnpm exec supabase projects list
   ```

While you are logged in, `db push` and `db query --linked` print "Initialising login role..." and connect with a temporary role through the Supabase Management API. You do not need the database password.

Apply migrations after a merge

1. Update your local `main`, so the migration files match `main`.
2. List the pending migrations:

   ```
   pnpm exec supabase db push --linked --dry-run
   ```

3. Check that the list holds only the migrations you expect, in order.
4. Apply them:

   ```
   pnpm exec supabase db push --linked
   ```

5. Run the dry run again. It should list nothing.
6. If the change needs a dashboard-only setting (such as the access token hook), set it by hand.

`pnpm exec supabase migration list --linked` compares the local files with the migrations recorded on the hosted project.

Run SQL

```
pnpm exec supabase db query --linked "select count(*) from public.profiles"
```

The command prints the result rows as JSON. `-f <file.sql>` runs a file instead. The temporary role can write tables that are read-only for the API roles (the placeholder store was inserted this way), so grants and RLS do not protect you here. Run a SELECT first, and make schema changes only through migrations.

Seed data on hosted dev

`seed.sql` is local test data. On a push with `--include-seed`, the CLI records each seed file's hash in `supabase_migrations.seed_files` and does not run a changed `seed.sql` again on a later push. So:

- Do not run `db push --include-seed` on hosted dev again.
- Insert any row hosted dev needs by hand with `db query --linked`. For example, the placeholder store, copied from `seed.sql`:

  ```
  pnpm exec supabase db query --linked \
    "insert into public.locations (id, name, address, latitude, longitude, timezone, hours,
  ordering_enabled) values ('10ca7100-0000-4000-8000-000000000001', 'PLACEHOLDER Sisig Edmonton',
  'PLACEHOLDER address, Edmonton, AB', 53.546100, -113.493800, 'America/Edmonton', '{}', false)
  on conflict do nothing"
  ```

Check the access token hook

1. In the dashboard, open **Authentication** → **Hooks**.
2. Check that **Customize Access Token** is enabled and points to `private.custom_access_token_hook`.
3. To confirm it works, sign in as a test user and decode the payload of the access token (`session.access_token` from `supabase.auth.getSession()`). It must contain `app_role`. Until the sign-in screens exist (planned, AUTH-01 to AUTH-03), create test users in the dashboard.

Connect the app

Set `apps/mobile/.env.local` as in How the app chooses an environment. LOCAL-DEVELOPMENT.md covers running the app against hosted dev.

Do not

- Run `supabase db reset --linked`. It rebuilds the hosted database and deletes its data.
- Edit a migration that hosted dev has applied. Write a new one.
- Commit `supabase/.temp/`, `.env.local`, keys or backup files.

Generated types

`packages/api/src/database.types.ts` holds TypeScript types for the `public` schema: tables (Row, Insert, Update), enums and the RPCs. `createSupabaseClient` returns a `SupabaseClient<Database>`, so `from()` and `rpc()` calls are type-checked.

```
pnpm db:start
pnpm db:types
```

`pnpm db:types` runs `supabase gen types typescript --local --schema public`. It writes to a temporary file and replaces the committed file only if the output is not empty. If the stack is not running, it exits 1 and leaves the committed file alone. `private` is not typed, because clients cannot reach it.

The CI `types-drift` job starts a fresh stack from the migrations, runs `pnpm db:types`, and fails if the file differs from the committed copy. It prints the diff. Without Docker, you can apply that diff by hand. Commit the regenerated file in the same PR as the schema change.

Do not edit or format the file. `pnpm format` runs `prettier --write src` in `packages/api`, which would rewrite it and fail `types-drift`.

Things to know about the types:

- Timestamps are `string`. `loyalty_settings.effective_from` can be `"-infinity"`.
- Columns with a default are optional in `Insert`.
- `Functions` lists `is_staff`, `is_admin`, `is_owner`, `set_user_role` and `current_loyalty_settings`.

Edge Functions

What exists

Path	What it is
`supabase/functions/whoami/`	Sample function. `GET` returns `{ user: { id, email, appRole } }`. `OPTIONS` returns 204. Other methods return 405. A missing or rejected token returns 401.
`supabase/functions/_shared/auth.ts`	`getBearerToken`, `authenticate`, `decodeJwtPayload`, `requireRole`, `authConfigFromEnv`
`supabase/functions/_shared/cors.ts`	CORS headers, preflight and JSON responses
`supabase/functions/_shared/errors.ts`	`AppError` and the error envelope `{ "error": { "code", "message" } }`
`supabase/functions/_shared/import_map.json`	An import map. `config.toml` does not reference it.

Each function keeps `index.ts` to one `Deno.serve(...)` call and puts its logic in `handler.ts`, so tests can import the handler without a server.

`authenticate` sends the caller's token to `GET <SUPABASE_URL>/auth/v1/user`, which rejects expired, revoked or forged tokens. Only then does it read the `app_role` claim from the token. An unknown or missing claim becomes `appRole: null`. `requireRole` returns 403 for a role that is not allowed. `authConfigFromEnv` reads `SUPABASE_URL` and `SUPABASE_ANON_KEY`, which Supabase sets for every function, and returns 500 "Server misconfigured" if either is missing.

CORS allows any origin (`*`). `cors.ts` notes that it must be tightened before production.

How functions are tested

The Deno tests (`_shared/auth_test.ts`, `_shared/errors_test.ts`, `_shared/import_map_test.ts`, `whoami/handler_test.ts`) inject a fake `fetch`, so they need no running stack. The `deno-tests` job in `.github/workflows/edge-functions.yml` runs on every PR to `main`:

```
deno test --allow-all --no-config supabase/functions
deno fmt --check --no-config supabase/functions
deno lint --no-config supabase/functions
```

To call a function locally, start the stack and run `pnpm exec supabase functions serve` (see supabase/README.md).

Deployment is not set up

No workflow deploys Edge Functions, and this guide has no record of a manual deploy to hosted dev. The mobile app does not call any function.

Testing

pgTAP database tests

pgTAP is a unit-test framework for Postgres. The tests live in `supabase/tests/database/` and run in file-name order.

File	What it covers
`01_init.sql`	`public` and `private` schemas; `uuid-ossp`, `pg_cron` and `pgcrypto`; `job_runs`, `products`, `orders`, `order_items`; key columns
`02_identity.sql`	Structure of `profiles` and `user_roles`; `app_role` labels; policy names; the helpers and `set_user_role` exist; phone and birthday checks; defaults
`03_identity_behaviour.sql`	DATA-02 behaviour: sign-up creates one profile and one `customer` role; lower-cased email and email sync; sign-up without email fails; hook output and grants; stale JWT versus the `set_user_role` re-check; every role rule; direct writes to `user_roles` blocked by grants and by RLS; read isolation; helper truth table, including `anon`; phone and birthday rules
`04_set_user_role_lock.sql`	The `FOR UPDATE` owner lock comes before the last-owner count; a role change sets `updated_at`
`05_money_points_checks.sql`	Negative `price`, `total_amount`, `points_earned` and `unit_price` are rejected; the four checks exist and are `NOT VALID`
`06_legacy_schema_cleanup.sql`	Legacy tables and enum are gone; `orders.user_id` references `profiles`; staff policies are exactly `(select is_staff())`; staff versus customer reads; deleting a member deletes their orders
`07_locations.sql`	Columns, time zone default, `pos_type`, one read policy, one seeded `PLACEHOLDER` store, RLS on, SELECT-only for all three API roles
`08_loyalty_settings.sql`	One `-infinity` placeholder baseline; `current_loyalty_settings` at, before and after version boundaries; unique `effective_from`; SELECT-only grants; RLS and policy
`09_points_ledger_append_only.sql`	`postgres` cannot update or delete (23001); `service_role` cannot either (42501); privileges; nested-trigger updates rejected; deleting a member keeps the rows with `user_id` NULL and nothing else changed
`10_points_ledger_integrity.sql`	History index; idempotency per source type; one reversal per entry; `reversal_of` rule; location rule for in-store sources; chain-level sources; `delta` non-zero and bounded; member and staff reads
`11_balances_and_redemptions.sql`	Privileges on balances, redemptions and rewards; RLS on; code index; chain-wide balance; policy sets; member and staff reads; the public catalogue window; code unique among issued and reusable after; what a member deletion does to balances, orders and redemptions
`12_app_config.sql`	Columns, key check, RLS and policy set, grants; the ten fail-open baseline rows; `anon` and members read; `anon` writes refused by grants; member and staff writes refused or filtered by RLS; admin and owner insert, update and delete

How the tests work:

- Each file runs as `postgres` inside one transaction and rolls back at the end.
- Fixtures insert into `auth.users`, so the real sign-up trigger creates profiles and roles.
- Files switch role with `set local role authenticated` (or `anon`) and fake the token with a `pg_temp.act_as(uid, app_role)` helper that sets `request.jwt.claims`. They call the hook function directly. They cannot show that the hook is enabled on a hosted project.
- Each file uses its own UUID prefix for fixtures (for example `09000000-...`; 03 and 04 both use `00000000-...`) and avoids data that a later migration might add.

Run them with `pnpm db:test` (needs the local stack, so Docker). The CI `database` job runs the same command after `supabase start`, which applies every migration and then `seed.sql` (`07_locations.sql` reads the seeded store). Docker is not available on the team's main Mac, so these tests have been verified in CI, not locally.

`pnpm db:lint` runs `supabase db lint` on `public` and `private` and fails on errors. CI runs it in the `database` job.

Troubleshooting

Local-stack problems (Docker, `db:reset`, stale types, function 500s) are in supabase/README.md.

Symptom	Cause	Fix
App throws at launch: "EXPO_PUBLIC_SUPABASE_URL and EXPO_PUBLIC_SUPABASE_ANON_KEY must be set"	`apps/mobile/.env.local` is missing or incomplete	Create it (Environments) and restart the dev server
A staff, admin or owner account sees only its own rows	No `app_role` claim: the hook is off on that project, or the token was issued before the role change	Check the hook (steps); sign out and back in
`permission denied for table ...` (42501)	The role has no grant for that command, for example any write to the DATA-04 tables, or `anon` reading `points_ledger`	Expected. Writes go through RPCs.
`new row violates row-level security policy for table ...`	No INSERT policy, or WITH CHECK failed (for example `order_items`)	Expected today; see Known issues
An UPDATE or DELETE returns no error but changes nothing	No policy for that command, so RLS hides every row	Expected; use the RPC for that change
`infinite recursion detected in policy for relation ...`	A policy reads its own table	Use a helper function (CONVENTIONS.md)
`points_ledger is append-only: UPDATE is not allowed` (23001)	Any update or delete on the ledger	Add a correcting or `reversal` row

Symptom	Cause	Fix
`duplicate key value violates unique constraint "points_ledger_source_type_idempotency_key_key"`	The same event was posted twice	The first posting stands; treat the retry as done
`Invalid Date` from `loyalty_settings.effective_from`	The baseline is `"-infinity"`	Handle the value explicitly
A change to `seed.sql` never reaches hosted dev	The CLI records seed files by hash	Insert the rows with `db query --linked`
`db push` lists migrations you did not expect, or refuses one	Your `main` is behind, or a timestamp sorts before the last applied migration	Update `main`; rename a clashing migration (README)
`types-drift` fails	The committed types do not match the migrations	Run `pnpm db:types`, or apply the diff CI prints
`set_user_role` with an unknown user shows "A database error occurred"	It raises `P0002` "User not found", which `normalizeError` in `packages/api/src/client.ts` does not map (permission denials, `42501`, map to `PERMISSION_DENIED`)	Known issue; read `error.details`

Known issues

1. **The hook on hosted dev is unconfirmed.** Until someone checks the dashboard, assume staff access may not work there.
2. **Creating an order with items fails.** `useCreateOrder` in `packages/api/src/mutations.ts` inserts the order, then its items. `order_items` has no INSERT policy, so the second insert fails and the order row is left without items. No screen calls the hook yet. Orders belong to DATA-06 (planned).
3. **Orders trust the client.** Members can insert orders with any `status`, `total_amount` and `points_earned`. No role can update or delete orders through the API.
4. **Staff cannot read other members' order items.** `order_items` has no staff policy.
5. **Deleting a user who granted a role fails.** `user_roles.granted_by` references `profiles` with no ON DELETE action, so the delete fails with `23503`. AUTH-05 (planned) must handle it.
6. **Sign-up needs an email.** `profiles.email` is NOT NULL, so a sign-up without an email fails. Sign-up metadata is not copied into `profiles`.
7. **An admin can demote another admin, or an owner who is not the last.** `set_user_role` only stops admins from granting `admin` or `owner`. No test covers these cases.
8. **Money and points checks are `NOT VALID`.** Old rows are not checked until a validating migration runs.
9. `updated_at` **is not maintained.** No table has an `updated_at` trigger.
10. **Error mapping is out of date.** `normalizeError` still maps the old `PERM` and `NTFND` codes, which the database no longer raises, and does not map `P0002`.

11. **Postgres versions differ.** The local stack and CI run Postgres 15; hosted dev runs Postgres 17.
12. **Some `config.toml` keys are not CLI settings.** `[project]`, `[db] seed_file_path`, `[auth] email_autoconfirm` and `[auth] other_providers` do not appear in the template that `supabase init` writes for CLI 2.122.0. The seed still runs, from the CLI's default path. GitHub and Google sign-in are not configured (that needs `[auth.external.*]` blocks), and auth redirect URLs are not set (planned, AUTH-01).
13. **Redemption codes do not expire on their own.** No job moves `issued` rows to `expired`.
14. **Edge Functions are not deployed**, and their CORS setting allows any origin.
15. **The table owner can still truncate `points_ledger`**, which skips the append-only trigger. Only the API roles lost TRUNCATE.
16. **`pnpm db:reset` does not start the stack.** Run `pnpm db:start` first.
17. **`db:reset` timing is not measured.** The 3-minute target in supabase/README.md has not been checked.

Runbook: Remote Config, Kill Switches and Forced Update

Source: docs/runbooks/remote-config.md

Use this runbook to switch off ordering (for example during a POS outage), switch off receipt scanning, show an announcement, or make outdated apps update. The values live in `public.app_config` (PLAT-06). The admin team will build toggles for them. Until then, change them with SQL as described here.

- How the table works: SUPABASE.md.
- How the app uses it: ARCHITECTURE.md.

Before you start

- **Environment.** Only the hosted dev project exists today. There is no staging or production project yet. Use the same steps there once they exist.
- **Where to run SQL.** Use either:
  - the Supabase dashboard: **SQL Editor**, then run the statement; or
  - the CLI from the repo root, once the project is linked (SUPABASE.md):

    ```
    pnpm exec supabase db query --linked "select key, value from public.app_config order by key"
    ```

  Both run as a privileged role, so RLS does not stop a mistake. Run the `select` first. Studio's **Table Editor** works too: edit the `value` cell as JSON and set `updated_at`.
- **Values are JSON.** Booleans are `'true'` or `'false'`. Strings need JSON quotes inside the SQL quotes: `'"1.4.0"'`. Use `to_jsonb('text'::text)` for a message, so quotes in the text are escaped for you. `'null'` clears a value.
- **Set** `updated_at = now()` in every UPDATE. No trigger does it for you.
- **When it takes effect.** Open apps fetch the config every 5 minutes and every time they come back to the foreground. A fresh launch fetches it straight away.
- **A bad value is ignored.** If a value does not match its type (for example `'"1.4"'` for a version), the app uses its bundled default for that key and reports it. Check your value against the table in SUPABASE.md.

Check the current values:

```
select key, value, updated_at from public.app_config order by key;
```

Switch off ordering

```
update public.app_config
   set value = 'false', updated_at = now()
 where key = 'ordering_enabled';

update public.app_config
   set value = to_jsonb('Online ordering is paused while we fix our till. Please order at the
counter.'::text),
       updated_at = now()
 where key = 'ordering_paused_message';
```

The Order tab hides its menu and checkout and shows "Ordering is paused" with your message (at most 280 characters). With `ordering_paused_message` set to `null`, the app shows its own default text.

To confirm: open the app, send it to the background and bring it back, then open the Order tab.

Switch ordering back on

```
update public.app_config
   set value = 'true', updated_at = now()
 where key = 'ordering_enabled';

update public.app_config
   set value = 'null', updated_at = now()
 where key = 'ordering_paused_message';
```

Switch receipt scanning off or on

```
update public.app_config
   set value = 'false', updated_at = now()   -- 'true' to switch it back on
 where key = 'receipt_scan_enabled';
```

No receipt-scan screen exists yet. The receipt-scanning feature reads this switch through `useFeature('receiptScan')` when it is built.

Force an update

Do these checks first. A mistake here locks customers out of the app.

1. **The new version is live in the store** for that platform, and your version number is not higher than the version in the store. Otherwise customers are asked to install an update that does not exist.
2. **The store link is set and works** for that platform (`store_url_ios` or `store_url_android`). Without a link, the app does not block (it fails open) and reports the problem, so the update would not be forced. The Android link is set by the baseline. The iOS link is `null` until the app has an App Store listing.
3. **Use the app's `version` (x.y.z)** from `apps/mobile/app.json` of the release you require, not a build number.

Store links must be https on `apps.apple.com` (iOS) or `play.google.com` (Android). The app rejects any other value and reports it, then uses its bundled default for that key: for iOS that is no link, so iOS is not blocked; for Android it is the Play listing for `com.sisig.mobile` , so Android is still blocked with that link.

To see the screen without touching customers, test on Android (iOS has no store link yet, so iOS fails open and shows nothing). In your local checkout, set `version` in `apps/mobile/app.json` to `0.0.1` and restart Metro (the dev client reads the version from Metro, so no rebuild is needed), then set `min_supported_version_android` to `'"0.0.2"'` . Customers on real releases are not affected. Afterwards, set the minimum back to `'"0.0.0"'` and do not commit the lowered `app.json` version. Do not test by setting the minimum just above the current release: there is no staging project, so the minimum applies to every customer at once.

Then set the minimum for one platform. For iOS, replace `REPLACE_WITH_APP_STORE_URL` with the real listing link before you run it; as written, the app rejects it and iOS is not blocked:

```
-- Set the App Store link once the listing exists.
update public.app_config
   set value = '"REPLACE_WITH_APP_STORE_URL"', updated_at = now()
 where key = 'store_url_ios';

update public.app_config
   set value = '"1.4.0"', updated_at = now()
 where key = 'min_supported_version_ios';      -- or min_supported_version_android
```

Installs below that version see a full-screen "Update required" screen with an "Update now" button that opens the store. It cannot be dismissed. It appears on launch and on the next foreground. An app that is offline at launch still shows it if its last fetched config required it, but the app never blocks because of a failed fetch.

To undo, set the minimum back to `'"0.0.0"'` .

Recommend an update

```
update public.app_config
   set value = '"1.4.0"', updated_at = now()
 where key = 'recommended_version';
```

Installs below that version, but not below the minimum, see a dismissible "A new version is available" banner at the top of the Home tab, at most once per day. It uses the same store links. To stop it, set the value back to `'"0.0.0"'`.

Show or clear an announcement

```
update public.app_config
   set value = to_jsonb('Closed Monday for the holiday.'::text), updated_at = now()
 where key = 'announcement_message';

-- Clear it:
update public.app_config
   set value = 'null', updated_at = now()
 where key = 'announcement_message';
```

The announcement shows at the top of the Home tab until you clear it (at most 280 characters).

Set the Uber Eats link

```
update public.app_config
   set value = '"https://www.ubereats.com/ca/store/..."', updated_at = now()
 where key = 'uber_eats_url';
```

The app validates it (https on `ubereats.com` only). No screen shows it yet.

Do not

- Delete the baseline rows. The app falls back to its defaults, which is safe, but the `description` column that explains each key is lost.
- Add new keys for features that do not read them. Unknown keys are ignored and reported.
- Raise a minimum version above the latest store release.

Supabase database (DATA-01)

Source: supabase/README.md

PostgreSQL with versioned migrations, row-level security, generated TypeScript types and Edge Functions. This file is the quick reference for the `supabase/` folder.

- How auth, roles, RLS, the loyalty schema and the hosted dev project work: docs/SUPABASE.md.
- Rules for migrations, schemas, policies, functions and tests: CONVENTIONS.md.

Prerequisites

- Docker running. The local Supabase stack runs in containers. Without Docker you cannot run the local stack; CI's `database` job runs the tests instead.
- Node 18+ and pnpm 9. The Supabase CLI is a root devDependency (2.122.0, locked in `pnpm-lock.yaml`), so the `db:*` scripts use it after `pnpm install`. Run other CLI commands as `pnpm exec supabase ...`. CI installs the same version.
- Deno 2 for the Edge Function tests (CI uses Deno 2.x).

Clean clone: create the local database

```
pnpm install
pnpm db:start   # starts the local stack; the first run pulls Docker images
pnpm db:reset   # rebuilds the database: every migration, then supabase/seed.sql
```

On a new database, `pnpm db:start` already applies every migration and then `seed.sql`. `pnpm db:reset` does not start the stack, so run `pnpm db:start` first.

Timing target: `pnpm db:reset` under 3 minutes with images already pulled. This has not been measured yet. To measure it, run `time pnpm db:reset`.

Commands

Command	What it does
`pnpm db:start`	Start the local stack (`supabase start`)
`pnpm db:stop`	Stop the local stack
`pnpm db:reset`	Rebuild the local database from migrations and seed (`supabase db reset`)
`pnpm db:types`	Regenerate `packages/api/src/database.types.ts` from the local `public` schema
`pnpm db:test`	Run the pgTAP tests (`supabase test db`); exits non-zero on failure
`pnpm db:lint`	Lint the `public` and `private` schemas (`supabase db lint --local`); fails on errors

Command	What it does
`pnpm db:new <name>`	Create a new migration file

Local stack URLs

`config.toml` sets no ports, so the CLI defaults apply.

Service	URL
API (Auth, REST, Edge Functions)	`http://127.0.0.1:54321`
Database	`127.0.0.1:54322`
Studio	`http://127.0.0.1:54323`
Mailpit (auth emails)	`http://127.0.0.1:54324`

`pnpm exec supabase status` prints the URLs and the local publishable key.

Hosted dev project

The only hosted project is "Sisig Eh" (ref `dczjqhqpxbkikidtbntf`). Link it once per machine, then use `--linked` commands. Step-by-step instructions, including the seed and access token hook caveats, are in docs/SUPABASE.md.

Command	What it does
`pnpm exec supabase login`	Sign the CLI in (once per machine)
`pnpm exec supabase link --project-ref dczjqhqpxbkikidtbntf`	Link this repo to hosted dev (once per machine)
`pnpm exec supabase db push --linked --dry-run`	List the migrations hosted dev has not applied
`pnpm exec supabase db push --linked`	Apply them (only from an up-to-date `main`)
`pnpm exec supabase db query --linked "<sql>"`	Run SQL on hosted dev; prints JSON rows
`pnpm exec supabase migration list --linked`	Compare local migration files with hosted dev's history

Layout

```
supabase/
├── config.toml       Local stack settings (exposed API schemas, auth, token hook)
├── migrations/       Forward-only SQL migrations, applied in timestamp order
├── seed.sql          Local seed data, applied after the migrations
├── tests/database/   pgTAP tests
```

```
├── functions/        Edge Functions; _shared/ holds common helpers
└── .temp/            CLI link state (gitignored)
```

Extensions (`uuid-ossp`, `pg_cron`, `pgtap`, `pgcrypto`, `pg_net`) are enabled in `migrations/20261028000000_init_schemas.sql`, not in `config.toml`. Auth redirect URLs are not templated in `config.toml` yet; AUTH-01 owns that.

Schema (from the migrations on main)

- `public` **tables**: `profiles`, `user_roles` (roles `customer`, `staff`, `admin`, `owner`), `products`, `orders`, `order_items`, `locations` (one store for now), `loyalty_settings` (versioned), `points_ledger` (append-only), `point_balances`, `rewards`, `reward_redemptions` and `app_config` (remote config, PLAT-06: public read, admin-only write).
- `public` **functions**: `is_staff()`, `is_admin()`, `is_owner()`, `set_user_role(p_user_id, p_role)`, `current_loyalty_settings(p_at)`, and the sign-up triggers' `handle_new_user()` and `sync_profile_email()`.
- `private`: `job_runs` (scheduled job log; no job exists yet) and the functions `custom_access_token_hook` and `enforce_points_ledger_append_only` (the ledger's append-only trigger).
- **Enums**: `app_role`, `order_status`, `pos_type`, `loyalty_rounding_mode`, `loyalty_earn_base`, `loyalty_expiry_policy`, `points_source_type`, `redemption_status`, `redemption_channel`.

The ER diagram, the access table and the loyalty model are in docs/SUPABASE.md.

Migrations

- Create one with `pnpm db:new <snake_case_name>`. Never edit one that has been applied to a shared environment (hosted dev counts). Rules are in CONVENTIONS.md.
- Rebuild the local database with `pnpm db:reset`. It applies every migration, then `seed.sql`.
- Check history with `pnpm exec supabase migration list --local` (the stack must be running) or `--linked` (hosted dev).
- After the PR is squash merged, push the migration to hosted dev. The full lifecycle and the rules for destructive migrations are in docs/SUPABASE.md.

Timestamp clashes

Two branches created migrations with the same or interleaved timestamps. Rebase onto `origin/main`, rename your migration to a timestamp after the latest one on main, then run `pnpm db:reset` and `pnpm db:types`.

NOT NULL column with seed rows

Seeding fails if a NOT NULL column is added without a default. Add the column with a default, fill the existing rows, and drop the default in the same migration:

```
alter table public.products add column sku text not null default 'UNKNOWN';
-- Fill real values for the existing rows here, for example:
-- update public.products set sku = '<value>' where sku = 'UNKNOWN';
alter table public.products alter column sku drop default;
```

Database types

```
pnpm db:types   # writes packages/api/src/database.types.ts (needs the stack running)
```

CI's `types-drift` job regenerates the file and fails if it differs from the committed copy. Commit the regenerated file in the same PR as the schema change. Do not edit or format it by hand.

Import the types from the API package:

```
import type { Database } from "@sisig/api";

type Profile = Database["public"]["Tables"]["profiles"]["Row"];
```

Testing

```
pnpm db:test                                          # pgTAP suites in tests/database
pnpm db:lint                                          # schema lint
deno test --allow-all --no-config supabase/functions  # Edge Function tests
deno fmt --check --no-config supabase/functions
deno lint --no-config supabase/functions
```

`pnpm db:test` is the same command CI's `database` job runs after `supabase start`, which applies the migrations and the seed from scratch. What each test file covers is listed in docs/SUPABASE.md.

Edge Functions

Each function is a folder with two files:

- `index.ts` calls `Deno.serve(...)` and nothing else.
- `handler.ts` exports `handler(req, deps)`, which holds the logic.

Shared code is in `_shared/`: `cors.ts` (CORS and JSON responses), `errors.ts` (`AppError` and the error envelope) and `auth.ts` (bearer token, verification with Supabase Auth, role check).

The sample function is `whoami`. `GET` returns the caller's `id`, `email` and `appRole`. A missing or invalid token gets a 401 in the error envelope. No function is deployed to hosted dev, and no workflow deploys them.

Run it locally:

```
pnpm db:start
pnpm exec supabase functions serve
curl -H "Authorization: Bearer <access_token>" http://127.0.0.1:54321/functions/v1/whoami
```

`authConfigFromEnv()` reads `SUPABASE_URL` and `SUPABASE_ANON_KEY`, which Supabase sets for functions. Never put the service-role key in a function that a client can reach, and never in the mobile or admin apps. CI's `secrets` job runs gitleaks over the whole repo, then greps `apps/` and `packages/` for one fixed string, the HS256 JWT header (the legacy key format), and for the name `SUPABASE_SERVICE_ROLE_KEY`. The grep has no rule for `sb_secret_` keys, so do not rely on it to catch one.

Troubleshooting

- **Docker is not running.** Start Docker Desktop, then run `pnpm db:start`.
- `db:reset` **says the stack is not running.** Run `pnpm db:start` first.
- `pnpm db:types` **exits 1 and the file is unchanged.** The stack is not running. Start it and run the command again.
- **Types look stale.** Run `pnpm db:types` and review `git diff packages/api/src/database.types.ts`.
- **A function returns 500 "Server misconfigured".** `SUPABASE_URL` or `SUPABASE_ANON_KEY` is missing from the function's environment.

Errors from RLS, the ledger and the hosted project are covered in docs/SUPABASE.md.

Known issues (open)

1. `db:reset` **does not start the stack.** See Clean clone above.
2. **The access token hook must be enabled per hosted project.** `config.toml` enables it for the local stack only. On hosted dev it is a dashboard setting that has not been confirmed. Without it, no token carries `app_role`.
3. **Local Postgres is 15, hosted dev is 17.** `config.toml` sets `major_version = 15`.
4. **Timing not measured.** The 3-minute `db:reset` target has not been checked.

Fixed: the old known issue 1, "RLS recursion" (staff policies on `products` and `orders` read `public.users`), which the header of `20261106230039_staff_policies_use_role_helpers.sql` refers to. That migration fixed it, and the numbering above no longer matches.

The full list is in docs/SUPABASE.md.

Resources

- Supabase CLI reference
- Row-Level Security

- pgTAP
- pg_cron
- Deno std

Database conventions

Source: supabase/CONVENTIONS.md

These rules apply to everything under `supabase/`. Schema changes go through versioned SQL migrations, and `packages/api/src/database.types.ts` must match them. Setup and commands are in README.md. How the pieces fit together (auth, roles, RLS, the loyalty model, the hosted dev project) is in docs/SUPABASE.md.

Migrations

- **Create them with the CLI.** Run `pnpm db:new <snake_case_name>`, which wraps `supabase migration new`. Never pick a timestamp by hand. Files go in `supabase/migrations/<YYYYMMDDHHMMSS>_<name>.sql`.
- **One concern per migration:** one table group, one function, or one policy set per file. Several early files bundle more than one concern (`20261028000000_init_schemas.sql`, `20261028000003_init_orders_and_rewards.sql`, `20261028000004_init_identity.sql`, `20261028000006_identity_hardening.sql`). They are not split.
- **Start with a header comment.** Name the Jira key, the one concern, why the change is needed and any data impact, and end with "Forward-only: no existing migration is edited." The DATA-04 and DATA-18 migrations are examples.
- **Forward-only.** Once a migration has been applied to any shared environment (the hosted dev project, later staging and production, or another developer's database), never edit it. Fix the problem with a new migration.
- **Timestamp clashes.** If your migration has the same timestamp as one on `origin/main`, or sorts before it, rebase onto `origin/main`. Rename your file to a timestamp after the latest migration on main, keep its contents, then run `pnpm db:reset` and `pnpm db:types`.
- **Checks that existing rows might fail are added `NOT VALID`.** Postgres then checks new and updated rows only. A later migration runs `VALIDATE CONSTRAINT` once the data is clean. `20261104180526_money_points_checks.sql` adds its checks this way; the migration that validates them does not exist yet.
- **Fail loudly.** When a migration depends on the data being in a certain state, it fails with an error instead of deleting or rewriting rows to make it fit (`20261106230040_retire_legacy_users.sql` fails with 23503 if an order has no profile).
- **Destructive changes are audited first.** Before a migration that can lose or reject data reaches a shared environment, follow the audit and backup steps in docs/SUPABASE.md.
- **Seed data is for the local stack.** Seed inserts end with `on conflict do nothing`. The clause only guards rows that have a fixed id or a unique key (the placeholder store). The products insert has neither, so running it twice duplicates the rows. Data that every environment needs goes in a migration, not in `seed.sql` (the loyalty settings baseline is in `20261106230042_create_loyalty_settings.sql`). Adding a NOT NULL column to a table that has rows needs a default in the same migration, and the default is dropped once the rows are filled.

Naming

- Tables are plural snake_case (`products` , `order_items`). Columns are snake_case. Timestamps are `created_at` and `updated_at` , typed `timestamp with time zone` .
- Money is an `integer` in cents (`price` , `total_amount` , `unit_price` , `min_spend_cents`).
- Indexes are `idx_<table>_<columns>` . A partial index adds its condition (`idx_reward_redemptions_code_issued`).
- New constraints are named explicitly, because pgTAP tests match the name: checks are `<table>_<column>_<rule>` or `<table>_<rule>` (`products_price_non_negative` , `points_ledger_in_store_requires_location`), and unique constraints are `<table>_<columns>_key` (`points_ledger_source_type_idempotency_key_key`).
- Policies have a short descriptive name in double quotes, unique per table. pgTAP `policies_are` tests check the names.
- Functions are snake_case verbs (`set_user_role` , `enforce_points_ledger_append_only`). Predicates return boolean and are named `is_*` (`is_staff`). Parameters start with `p_` and PL/pgSQL variables with `v_` (`p_user_id` , `v_caller_role`).
- Enum types are snake_case (`app_role`) with lowercase labels.

Schemas: public vs private

- `public` holds only what clients may reach through PostgREST or RPC. Every public table enables row-level security and has explicit policies. No policy means no access.
- `private` holds internal tables and functions: the job log (`private.job_runs`), the access-token hook (`private.custom_access_token_hook`) and the ledger's append-only trigger function (`private.enforce_points_ledger_append_only`). It is not in `[api] schemas` in `supabase/config.toml` , and the `20261028000000_init_schemas.sql` migration revokes it from `public` .
- A role that must reach a private object gets an explicit grant in a migration (`supabase_auth_admin` has `USAGE` on `private` and `EXECUTE` on the hook). Do not widen `private` to `anon` or `authenticated` .

Grants

- Supabase gives `anon` , `authenticated` and `service_role` full privileges on every new `public` table. A migration that creates a table therefore states its grants: `revoke all ... from anon, authenticated, service_role` , then `grant select` to the roles that read it. Every table added by DATA-04 and DATA-18 does this.
- A new table that is written by RPCs gets no INSERT, UPDATE or DELETE grant for any API role, `service_role` included, and no write policy. Writes go through a `SECURITY DEFINER` function: the LOY RPCs for the six DATA-04 and DATA-18 tables (planned, LOY-01/03/04).
- Exception: the older tables predate this rule. `user_roles` revokes writes from `anon` and `authenticated` only (clients write it through `set_user_role`), so `service_role` keeps its default write privileges there. `profiles` , `products` , `orders` and `order_items` keep the default grants for every API role and rely on policies. No migration changes this yet.

- To let clients change only some columns, revoke UPDATE and grant it per column (`profiles` : `first_name` , `last_name` , `phone_e164` , `birthday_month` , `birthday_day` , `province` , `locale`).
- A function that clients may call revokes `EXECUTE` from `public` (and `anon` where needed) and grants it to the roles that call it (`set_user_role` , `current_loyalty_settings`).

Policies

Policies must not query their own table

A policy whose subquery reads the table it protects recurses, directly or through another table's policy. Postgres fails the query with `infinite recursion detected in policy for relation ...` . Move the check into a helper instead:

```
create policy "Staff can view all orders"
  on public.orders for select
  using ((select public.is_staff()));
```

Wrap helper calls in a scalar subquery

New policies write `(select public.is_staff())` and `(select auth.uid())` , not the bare call, so Postgres evaluates the function once per statement instead of once per row. The policies on `profiles` and `user_roles` and the own-row policies on `orders` and `order_items` predate this rule.

Name the roles a policy applies to

New policies say which roles they cover: `to authenticated` , or `to anon, authenticated` for data anyone may read. A policy without `to` applies to every role.

Staff checks read the JWT

`is_staff()` , `is_admin()` and `is_owner()` read the `app_role` JWT claim that the access-token hook adds. `config.toml` registers the hook for the local stack. Each hosted project must enable it in the dashboard (see docs/SUPABASE.md). The claim can be stale for up to one token lifetime, so privileged writes re-check `public.user_roles` in SQL (next section).

SECURITY DEFINER functions

- Every function, security definer or invoker, sets `search_path = ''` and schema-qualifies each object it touches (`public.user_roles` , `auth.uid()`). `pg_catalog` functions such as `now()` need no prefix.

```
create or replace function public.handle_new_user()
returns trigger as $$
begin
  insert into public.profiles (id, email) values (new.id, new.email);
```

```
  return new;
end;
$$ language plpgsql security definer set search_path = '';
```

- Check the rule against the catalog. Every row must show `search_path=""`:

```
select n.nspname, p.proname, p.prosecdef, p.proconfig
from pg_proc p join pg_namespace n on n.oid = p.pronamespace
where n.nspname in ('public', 'private');
```

- A `security definer` function runs as its owner, so grants and RLS do not apply inside it. It checks the caller itself. For a privileged action it reads the caller's role from `public.user_roles`, not from the JWT, so a demoted user is refused at once (`set_user_role` does this).
- Raise standard SQLSTATEs: `42501` (insufficient_privilege) for a permission denial, `P0002` (no_data_found) when the target row does not exist, `23001` for an append-only violation. Never invent codes: `'PERM'` is not a valid SQLSTATE and fails with 42704.

Append-only tables

- An append-only table has a `BEFORE UPDATE OR DELETE` trigger. Its function lives in `private` and raises `23001` with `<table> is append-only: <operation> is not allowed`. Example: `points_ledger_append_only` runs `private.enforce_points_ledger_append_only()`.
- API roles get at most SELECT (`anon` gets nothing on `points_ledger`). `revoke all` also removes TRUNCATE, which skips row triggers.
- Corrections are new rows (a `reversal` row in `points_ledger`), never updates.
- Any exception is as narrow as possible, and each of its guards has its own pgTAP test. The only one today is the AUTH-05 anonymisation path in `points_ledger`.

Member data and deletion

- A foreign key to `public.profiles` in a new table states its `ON DELETE` action: `cascade` for data that belongs only to the member (`point_balances`), `set null` for records that must outlive the member (`points_ledger.user_id`, `reward_redemptions.user_id`). `user_roles.granted_by` predates this rule and has no action.
- A row that outlives its member must not hold member identifiers (user id, email, phone) anywhere except the column that is set to NULL. Say so in a column comment, as `points_ledger.source_id`, `points_ledger.idempotency_key`, `points_ledger.reason`, `points_ledger.metadata` and `reward_redemptions.idempotency_key` do.
- Index the foreign key columns that a profile deletion follows, so the delete does not scan the table (`idx_reward_redemptions_order_id`, `idx_reward_redemptions_fulfilled_by`).

Scheduled jobs and job_runs logging

No `pg_cron` job exists yet. These rules apply to the first one.

- Every pg_cron job writes to `private.job_runs`. It inserts a `running` row at start, then sets `ended_at` and `status` to `success` (with `rows_processed`) or `error` (with `error_message`).
- A job that catches an error must record it and return normally. If it re-raises, the transaction rolls back and the error row is lost.
- The start and finish helpers are not in a migration yet. The first scheduled job that needs them adds them, and later jobs reuse them.

Edge Functions

- Layout: `supabase/functions/<name>/index.ts` only calls `Deno.serve(...)`. The logic lives in `handler.ts` as `handler(req, deps)`, so tests can import it without starting a server. Example: `supabase/functions/whoami/`.
- Shared code is in `supabase/functions/_shared/`:
  - `cors.ts`: CORS headers, preflight response, JSON responses.
  - `errors.ts`: `AppError` and `errorEnvelope`.
  - `auth.ts`: bearer token, `authenticate` (verifies with Supabase Auth), `requireRole`.
- Errors use one envelope, `{ "error": { "code": "...", "message": "..." } }`. Anything that is not an `AppError` becomes a 500 `internal_error` with a generic message.
- Tokens are verified with Supabase Auth before any claim is read. The `app_role` claim can be stale for up to one token lifetime. Sensitive writes must re-check `public.user_roles` in SQL.
- Tests run from the repo root with `--no-config`, so the TypeScript config of the app packages is not picked up: `deno test --allow-all --no-config supabase/functions`, then `deno fmt --check --no-config` and `deno lint --no-config`.

Database tests

- pgTAP files live in `supabase/tests/database/` and are named `NN_<topic>.sql` so they run in order. Each file wraps its tests in `begin; select plan(n);` and `select * from finish(); rollback;`.
- `plan(n)` must equal the number of assertions. A mismatch fails the run.
- Test behaviour, not only structure. Each new policy, grant and constraint gets a test that exercises it, as `anon`, a member and staff where they apply.
- A rejected write asserts both the SQLSTATE and the message, so a different failure fails the test: `throws_ok(sql, '23514', 'new row for relation "..." violates check constraint "..."', description)`.
- Switch roles with `set local role authenticated` (or `anon`), and fake the token with a `pg_temp.act_as(uid, app_role)` helper that sets `request.jwt.claims`. Run `reset role` before writing fixtures as `postgres`.
- Fixtures insert into `auth.users`, so the sign-up trigger creates the profile and role. Each file uses its own UUID prefix (`09000000-...` in `09_points_ledger_append_only.sql`). The exception is 03 and 04, which

both use `00000000-...` (with different suffixes). Tests do not depend on rows that a later migration or real data could add: look up seeded rows by their `PLACEHOLDER` label, and use far-future dates (2999) for test settings versions.

- Run them with `pnpm db:test`. CI runs the same command.

DEVOPS-02: GitHub Actions CI Pipeline

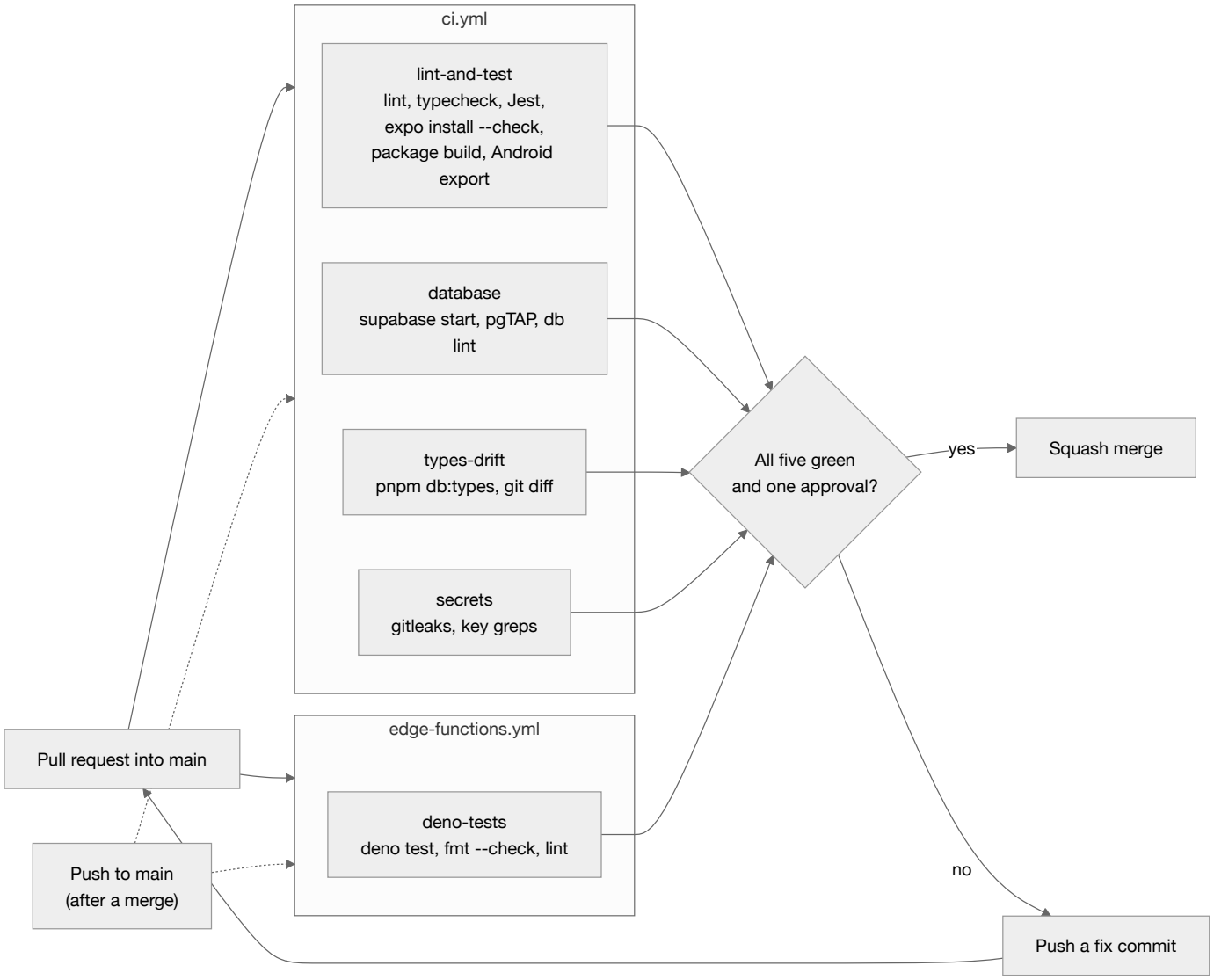
Source: docs/DEVOPS-02-CI.md

Plan ID: DEVOPS-02 | **Epic:** DEVOPS | **Owner:** DEV-C | **Status:** Code complete. Required checks are not yet enforced on `main` (manual step below).

Overview

Every pull request into `main` runs automated checks for code quality, the mobile bundle, database integrity, and leaked secrets. Every push to `main` runs them again. A failing check blocks a merge only after branch protection on `main` requires it (see Required status checks), and that is not configured yet.

Architecture



Both workflows start on every pull request into `main` and on every push to `main`. The five jobs run in parallel, with no dependencies between them. The merge gate is a team rule until branch protection requires the five checks; the dotted push run re-checks `main` after a merge and gates nothing.

Workflow	File	Triggers	Jobs
CI	`.github/workflows/ci.yml`	`pull_request` to `main`, `push` to `main`	`lint-and-test`, `database`, `types-drift`, `secrets`
Edge Functions	`.github/workflows/edge-functions.yml`	`pull_request` to `main`, `push` to `main`	`deno-tests`

Neither workflow has a path filter, `workflow_dispatch` or a `concurrency` group. Every PR runs every job, and a new push to a PR does not cancel the run for the previous push.

Workflows

1. CI (`.github/workflows/ci.yml`)

Triggers: `pull_request` and `push` to `main`. No path filter, so every PR runs every job.

All four jobs run on `ubuntu-latest`. Durations below are from run 42338504203 (the push to `main` for PR #19, 2026-11-07 04:27 UTC).

Job 1: `lint-and-test`

- **Timeout:** 15 minutes. **Measured duration:** 73 seconds.
- **Steps, in order:**
  1. `actions/checkout@v4`
  2. `pnpm/action-setup@v2` with pnpm 9
  3. `actions/setup-node@v4` with Node 18 and `cache: pnpm` (the pnpm store)
  4. **Restore Turborepo cache:** `actions/cache@v4` on `.turbo/cache` (see Caching)
  5. **Install dependencies:** `pnpm install --frozen-lockfile`
  6. **Lint:** `pnpm lint` (ESLint in every workspace that has the script; `apps/admin` is a placeholder)
  7. **Type check:** `pnpm typecheck` (`tsc --noEmit`; Turborepo runs the `build` tasks of the workspace packages before it (the `^build` dependency in `turbo.json`). The root scripts also pass the deprecated `--parallel` flag; in a timing check with Turborepo 2.11.10 and no cache, dependent tasks still started only after the builds finished)
  8. **Unit tests:** `pnpm test` (Jest in every workspace that has the script)
  9. **Check Expo dependencies:** `pnpm -F @sisig/mobile exec expo install --check`. Fails if an installed package version does not match what Expo SDK 51 expects. It compares versions only.
  10. **Build workspace dependencies of the mobile app:** `pnpm turbo run build --filter='@sisig/mobile^...'`. Builds the `dist/` entry points of the `@sisig/*` packages the app imports. Step 7 already built them; this step keeps step 11 working if it ever runs on its own.
  11. **Bundle mobile app (Android export):** `pnpm -F @sisig/mobile exec expo export --platform android --output-dir ../../.expo-export-check`. A full Metro bundle of the app. It catches what

step 9 cannot: a missing entry point, an undeclared or mis-pinned package, an import Metro cannot resolve. The output goes to `.expo-export-check/` at the repo root, which is git-ignored and not kept.

Step 9 has been in the job since the first DEVOPS-02 pipeline. PR #16 added steps 10 and 11 (and the Anthropic key check in `secrets`), after the app failed to bundle on a clean clone while step 9 stayed green.

Job 2: `database`

- **Timeout:** 15 minutes. **Measured duration:** 134 seconds.
- **Steps:** checkout; `npm install -g supabase@2.122.0`; **Start Supabase:** `supabase start -x realtime` (the Realtime container is skipped), retried only on Docker registry rate limits (see Troubleshooting); **Wait for database:** `sleep 5` then `supabase status`; pnpm 9; Node 18 with the pnpm cache; `pnpm install --frozen-lockfile`; **Run pgTAP tests:** `pnpm db:test` (`supabase test db`, runs `supabase/tests/database/*.sql`); **Lint database schema:** `pnpm db:lint` (`supabase db lint --local --schema public,private --fail-on error`). The `extensions` schema is not linted, because pgTAP's own functions there fail lint.
- **Database:** the Supabase CLI starts its own Postgres container in Docker. The job does not use a separate PostgreSQL service. On a fresh database, `supabase start` applies every file in `supabase/migrations/` and then `supabase/seed.sql` (the log shows "Seeding data from supabase/seed.sql..."). The pgTAP tests run against that database, and `07_locations.sql` asserts the seeded placeholder store, so a broken seed fails this job.

Job 3: `types-drift`

- **Timeout:** 15 minutes. **Measured duration:** 128 seconds.
- **Steps:** the same Supabase CLI install, start and wait as `database`; pnpm and Node setup; `pnpm install --frozen-lockfile`; **Generate types:** `pnpm db:types` (generates `packages/api/src/database.types.ts` from the local `public` schema); **Check for uncommitted type changes:** `git diff --quiet packages/api/src/database.types.ts`. On a difference it prints the diff and "Run 'pnpm db:types' and commit the changes", then fails.

Job 4: `secrets`

- **Timeout:** 5 minutes. **Measured duration:** 7 seconds.
- **Checks, in workflow order:**
  1. **gitleaks scan:** `gitleaks/gitleaks-action@v2` with default rules. It scans the commits of the push or pull request, not the whole history: run 42338504203 used `--log-opts=-1` and logged "1 commits scanned". `fetch-depth: 0` only makes the history available. Fails on a detected key or token.
  2. **Service-role key or JWT pattern:** `git grep -F` for the HS256 JWT header `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9` in `apps/` and `packages/`.
  3. **Anthropic API key pattern:** `git grep -E` for `sk-ant-` followed by 20 or more key characters in `apps/` and `packages/`. Placeholders such as `sk-ant-...` in the READMEs do not match. `ANTHROPIC_API_KEY` itself is not checked, because client code references it legitimately.

4. `SUPABASE_SERVICE_ROLE_KEY` **references:** `git grep -F` for that name in `apps/` and `packages/`.

Checks 2 to 4 pass only when `git grep` exits 1 (no match). A match (exit 0) and any other exit code (a grep error) both fail. The JWT header in check 2 matches every HS256 JWT, including Supabase's legacy anon key, so Supabase keys must come from environment configuration (`apps/mobile/.env.local`), never from source.

2. Edge Functions (`.github/workflows/edge-functions.yml`)

Triggers: `pull_request` and `push` to `main`, with no path filter on purpose. `deno-tests` is meant to be a required check, and a path-filtered workflow reports no status on PRs outside its paths, which would leave a required check pending forever.

Job: `deno-tests`

- **Timeout:** 10 minutes. **Measured duration:** about 7 seconds (run 42338504195, same push).
- **Steps:** checkout; `denoland/setup-deno@v1` with Deno `v2.x`; **Run Deno tests:** `deno test --allow-all --no-config supabase/functions`; **Format check:** `deno fmt --check --no-config supabase/functions`; **Lint:** `deno lint --no-config supabase/functions`.

Removed: `required-status-checks.yml`

This placeholder read the conclusion of other workflow runs on `main`, and enforced nothing. Results of `workflow_run` runs do not attach to pull request checks, so it never blocked a merge. It was deleted in DEVOPS-02.

Required status checks (manual GitHub setting)

Not configured yet. Checked again on 2026-11-06: `gh api repos/Alexd2004/Sisig/branches/main/protection` returns 404 "Branch not protected", and `gh api repos/Alexd2004/Sisig/rulesets` returns an empty list. A repository admin must set this in **Settings > Branches > Branch protection rule** for `main`:

- Require status checks to pass before merging, and select `lint-and-test`, `database`, `types-drift`, `secrets` and `deno-tests`.
- Require a pull request with at least one approval, and block force pushes. These belong to DEVOPS-03 (see CONTRIBUTING.md).

The check names are the job IDs above. GitHub offers a check as selectable only after it has run at least once on the repository, which has happened for all five.

Local Development

Running CI checks locally

The database jobs need Docker, and the first `pnpm db:start` pulls the Supabase images. Docker is not installed on the team's Mac, so those jobs are checked in CI. The other jobs run without Docker.

1. `lint-and-test`:

   ```
   pnpm install --frozen-lockfile
   pnpm lint
   pnpm typecheck
   pnpm test
   ```

   ```
   pnpm -F @sisig/mobile exec expo install --check
   ```

   ```
   pnpm turbo run build --filter='@sisig/mobile^...'
   pnpm -F @sisig/mobile exec expo export --platform android --output-dir ../../.expo-export-check
   ```

2. `database` (Docker required):

   ```
   pnpm db:start
   pnpm db:reset
   pnpm db:test
   pnpm db:lint
   ```

3. `types-drift` (Docker required). `git diff --exit-code` exits 1 when the regenerated file differs; commit it if so:

   ```
   pnpm db:types
   git diff --exit-code packages/api/src/database.types.ts
   ```

   If the local stack is not running, `pnpm db:types` exits 1 and leaves the committed file unchanged.

4. `deno-tests` (install Deno 2 first):

   ```
   deno test --allow-all --no-config supabase/functions
   deno fmt --check --no-config supabase/functions
   deno lint --no-config supabase/functions
   ```

5. `secrets` (install gitleaks first: `brew install gitleaks`):

```
gitleaks detect
```

```
git grep -n -I -F -e "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9" -e "SUPABASE_SERVICE_ROLE_KEY" --
apps packages
git grep -n -I -E -e "sk-ant-[A-Za-z0-9_-]{20,}" -- apps packages
```

`git grep` searches tracked files only, so stage new files before you run it. No output means clean.

Common Failures & Fixes

Type drift detected

The migrations changed the `public` schema, but the committed types were not regenerated. With the local stack running (Docker):

```
pnpm db:start
pnpm db:types
git diff packages/api/src/database.types.ts
```

Commit the regenerated file on your branch with a Jira-scoped message, for example `chore(DATA-04): regenerate database types`.

Migration or pgTAP test fails

Read the test output. If the migration is already on `main` or applied to the hosted project, fix it with a new migration; never edit an applied migration (see supabase/CONVENTIONS.md). Then run `pnpm db:reset && pnpm db:test`.

Schema lint fails

`supabase db lint --schema public,private --fail-on error` reported an error. Fix the reported object, then run `pnpm db:reset && pnpm db:lint`. Check that the error is in our schemas, not pgTAP internals (the `extensions` schema is not linted).

`expo install --check` fails

A dependency in `apps/mobile/package.json` is outside the range Expo SDK 51 expects. The step's log names the package and the expected version. Align the version, run `pnpm install`, and commit `pnpm-lock.yaml` with it.

Android export fails

Metro could not bundle the app. Typical causes: an import of a package that `apps/mobile/package.json` does not declare (pnpm does not expose undeclared packages), a version pinned outside the SDK 51 line, or a test or helper file under `apps/mobile/app/` (Expo Router bundles every file there as a route). Reproduce with the export commands in Running CI checks locally.

Lint error

Fix the ESLint findings from `pnpm lint`. `pnpm format` applies Prettier formatting only.

Secret detected

A secret that reached a commit stays in git history. Rotate it first, then remove it from the code. Deleting it in a later commit is not enough.

Service-role key or reference found

Move the value to a server-side Edge Function or to environment configuration. `SUPABASE_SERVICE_ROLE_KEY` must not appear in `apps/` or `packages/`.

Performance

Caching

- **pnpm store:** `actions/setup-node` with `cache: pnpm`, in every job that installs packages (`lint-and-test`, `database`, `types-drift`).
- **Turborepo:** only `lint-and-test` restores `.turbo/cache` (Turborepo's local cache directory) with `actions/cache@v4`. The key is `turbo-<os>-<run_id>`, so each run saves a new entry; the restore key `turbo-<os>-` makes the next run restore the most recent one. Turborepo then replays cached output for tasks whose inputs did not change. In run 42338504203, 4 of 5 lint tasks and 5 of 8 typecheck tasks were cache hits.
- **Not cached:** the Supabase Docker images (pulled on every `database` and `types-drift` run) and the Deno toolchain.
- CI does not filter to affected packages. Every task runs on every PR, and unchanged tasks replay from cache.

Measured durations

Run 42338504203 (push to `main` for PR #19): `lint-and-test` 73 seconds, `database` 134 seconds, `types-drift` 128 seconds, `secrets` 7 seconds; `deno-tests` about 7 seconds in the Edge Functions run. Jobs run in parallel, so the wall clock was about 2.3 minutes.

Median over the last 10 CI runs up to that one, including one failed run (`gh run list --workflow CI`): about 2.4 minutes. Target: under 10 minutes.

Security

gitleaks

Detects leaked secrets with its default rules (AWS keys, GitHub tokens, Slack webhooks and others) in the commits of each push or pull request. It does not rescan the whole history. Fails the PR when it finds one. There is no `.gitleaksignore` yet.

Service-role key and JWT check

See `secrets` checks 2 to 4 above. Client code (`apps/` , `packages/`) must never contain the service-role key. `supabase/` is excluded on purpose, because Edge Functions run server-side and may use it.

Monitoring & Alerts

GitHub Actions shows each job's duration, its failures and their logs. There are no alerts beyond GitHub's own notifications.

Re-running CI

If a job fails for a reason unrelated to the change (rare flakiness), open the PR, click "Details" on the failed check and choose "Re-run jobs". Investigate any repeated failure.

Expansion (Future)

DEVOPS-04: Build & Deploy

- `pnpm build` for the mobile and admin apps (today the mobile `build` script is `expo build` , which the local Expo CLI rejects)
- `eas build` cloud builds (EAS CLI; not configured yet, there is no `eas.json`)
- Deploy to Vercel (admin) and to the App Store and Play Store (mobile)

Data Migration Testing

Once DATA-14 (audit logging) lands: test that audit trails are created and that historical data stays accessible.

Migrations to the hosted project

CI tests migrations on a throwaway local database only. Nothing applies them to the hosted Supabase dev project; that is a manual `supabase db push` after merge (see CONTRIBUTING.md and docs/SUPABASE.md).

Known Gaps

- **Branch protection is not configured.** Until it is, none of these checks blocks a merge.
- **Merge methods are not restricted.** The repository still allows merge commits and rebase merges next to squash merges, and does not delete head branches after merge.
- `pnpm format:check` **is not run in CI.** It fails on `main` today. Checked on 2026-11-06, Prettier reports 14 files in `packages/api` , 10 in `packages/claude` , 13 in `packages/ui` and 53 in `apps/mobile` . Prettier finds no config file for those sources (`prettier --find-config-path` reports none; the shared `packages/config/prettier.config.js` is not referenced), so it checks them against its defaults. CONTRIBUTING.md does not list it as a required check.
- **Behavioural RLS coverage is partial.** `supabase/tests/database/` has 11 files. `01_init.sql` and `02_identity.sql` are structural: they check that tables, columns, functions, constraints and policy names

exist. The other nine also run statements and assert the results, and five of them (`03`, `06`, `09`, `10`, `11`) switch roles with `set local role` to test RLS as `anon`, `authenticated` or `service_role`. Behavioural RLS tests for every table belong to DATA-08.

- **The `db lint` gate found real problems on its first run.** On commit 45398aa the `database` job failed on `public.set_user_role`, which raised the invalid error code `PERM` (fixed by forward-only migration `20261029120000_fix_set_user_role_error_codes.sql`). The same run reported nine findings from pgTAP's own functions in the `extensions` schema, so the lint covers `public,private` only.
- **No affected-only runs.** Turborepo runs every task on every PR.
- **No `concurrency` group.** Pushing twice to a PR runs both CI runs to the end.
- **CI runs Node 18,** which reached end of life in April 2025. Upgrading is a separate change.
- **Action versions are behind.** GitHub warns that `actions/checkout@v4`, `actions/setup-node@v4` and `pnpm/action-setup@v2` target Node.js 20, which is deprecated on its runners, and runs them on Node.js 24 instead.

Troubleshooting

CI flaky or timing out

1. The `database` and `types-drift` jobs retry `supabase start` only when the log contains "Data limit exceeded" (the public.ecr.aws registry rate limit on shared runners): three attempts, 90 seconds apart, with `supabase stop --no-backup` between them. Any other start failure fails the job at once.
2. If `supabase start` is slow, run it locally to confirm the failure is real.
3. Job timeouts are set with `timeout-minutes` in `.github/workflows/` (15 minutes for the CI jobs except `secrets` at 5, and 10 for `deno-tests`).

gitleaks false positive

Some strings trigger gitleaks, for example a fake token in a test fixture. Confirm it is not a real secret, then either:

- add a `gitleaks:allow` comment at the end of that line, or
- add the finding's fingerprint (shown in the gitleaks report as `Fingerprint`) to a `.gitleaksignore` file at the repo root, one per line.

Types drift after merge

If `main` itself has drifted, do not push to `main`. Open a PR with the fix:

1. Create a branch from an up-to-date `main`, for example `fix/<JIRA-KEY>-sync-database-types`.
2. With the local stack running, run `pnpm db:types`.
3. Commit `packages/api/src/database.types.ts` with a message such as `fix(<JIRA-KEY>): sync database types with migrations`, push, and open a PR.

Related Documentation

- .github/workflows/ci.yml: CI workflow definition

- .github/workflows/edge-functions.yml: Deno tests
- CONTRIBUTING.md: branching, commits, PR process, required checks
- docs/LOCAL-DEVELOPMENT.md: running the app locally
- docs/SUPABASE.md: the hosted project and applying migrations
- supabase/README.md: local database commands
- supabase/CONVENTIONS.md: database conventions
- README.md: repo front page

Owner: DEV-C (DevOps engineer) **Last updated:** 2026-11-06

Contributing to Sisig

Source: CONTRIBUTING.md

Branching Strategy

We follow **trunk-based development**: `main` is the only long-lived branch and is always releasable. Work lands through short-lived branches, each merged to `main` by pull request. The only other long-lived branches are `release/x.y` lines, and only when a hotfix is needed (see Release Strategy).

Branch Naming

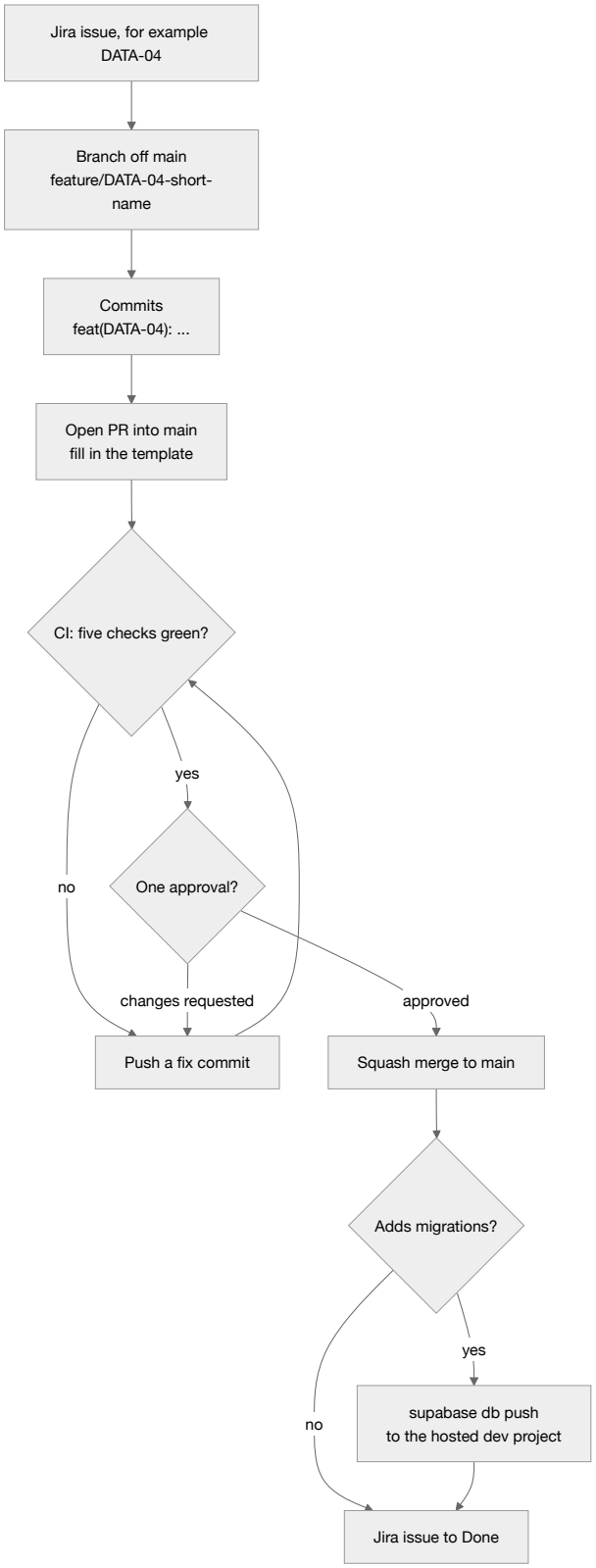
- Feature: `feature/JIRA-KEY-description`
- Bugfix: `fix/JIRA-KEY-description`
- Chore, config or docs: `chore/JIRA-KEY-description`
- Hotfix: `hotfix/JIRA-KEY-description`
- Release line (hotfix support only): `release/x.y`

Example: `feature/DATA-04-loyalty-schema-locations`

Some early branches did not follow this pattern (for example `feat/theme-01-tamagui-theme` and `fix/mobile-placeholder-assets`). New branches use it.

Workflow

1. Create a short-lived branch off `main`
2. Make commits in Conventional Commits format with the Jira key as the scope (see Commit Messages)
3. Open a PR into `main`. The PR title uses the same format as a commit, for example `feat(DATA-04): add loyalty ledger schema and slim locations`
4. Wait for CI. All five checks must pass (see Required Checks)
5. Get one approval from another developer; CODEOWNERS (`.github/CODEOWNERS`) routes review requests by area
6. Squash merge to `main`
7. If the PR adds migrations, apply them to the hosted dev project (see Database Changes)
8. Move the Jira issue to Done, and delete the branch (GitHub does not delete it automatically)



A change moves from a Jira issue to a merged squash commit on `main`. When the PR includes migrations, they reach the hosted Supabase project only after someone runs `supabase db push`, so the issue is not done until then.

What the squash commit keeps

The repository's squash settings build the commit on `main` like this:

- **Title:** the PR title, or the commit's own title when the PR has a single commit. Either way it must carry the Jira key, and GitHub appends the PR number, as in `feat(DATA-04): add loyalty ledger schema and slim locations (#19)`.
- **Body:** the messages of every commit in the PR.
- **Not kept:** the PR description and review comments.

So write useful commit messages, and put rules and decisions into the docs or code comments, not only into the PR description. See docs/README.md.

Release Strategy

There are no releases yet: the repo has no tags, and release builds are not set up. The process below is the agreed plan.

Standard Release

Normal releases are tags on `main`. Do not create a release branch for them.

1. Make sure `main` contains everything for the release and all checks are green
2. Open a PR with the version bump and changelog update (`chore(JIRA-KEY): release x.y.z`); it goes through the normal PR flow
3. Tag the merge commit on `main` as `vx.y.z`
4. Build and deploy by hand for now. No workflow in `.github/workflows` builds or deploys from tags, and release builds need EAS, which is not configured (no `eas.json`). A build and deploy pipeline is planned for DEVOPS-04.

Hotfix

Use a `release/x.y` branch only to ship a hotfix to an already-released line.

1. If `release/x.y` does not exist yet, create it from the `vx.y.z` tag of the affected release
2. Create `hotfix/JIRA-KEY-description` off `release/x.y`
3. Open a PR into `release/x.y` with the fix; the same review and squash rules apply
4. After merge, tag the new patch version `vx.y.(z+1)` on `release/x.y` and deploy it by hand (no tag-based deploy workflow exists yet)
5. Bring the fix to `main` with its own PR (cherry-pick the squashed commit); never push to `main` directly

Commit Messages

Follow Conventional Commits:

```
type(scope): description

[optional body]

[optional footer]
```

Types:

- `feat`: New feature
- `fix`: Bug fix
- `refactor`: Code refactor (no behavior change)
- `test`: Test additions/changes
- `docs`: Documentation
- `chore`: Build, dependency updates, config changes

Scope is the Jira key (for example `PLAT-01`, `DEVOPS-03`). Every commit and PR title must carry one so Jira can link the work. Area names (`mobile`, `admin`, `ui`, `api`, `supabase`) go in the body or PR description, not in place of the key. Some earlier PRs used an area scope, such as `fix(mobile): ...` in #15, #16 and #17; do not copy them. Nothing in CI checks the format, so reviewers do.

Examples:

- `feat(PLAT-01): scaffold pnpm workspaces with Turborepo`
- `fix(DEVOPS-02): generate database types for the public schema only`
- `test(PLAT-01): add Button component tests`

Pull Request Process

1. Keep the PR title concise (under 70 characters), in Conventional Commits format with the Jira key
2. Fill out the PR template (`.github/pull_request_template.md`) completely: Jira key and link, changes, testing, screenshots in light and dark mode for UI changes, migration and RLS impact, analytics events, the accessibility checklist, the privacy inventory check and the REL-07 question
3. Request review from the CODEOWNER(s) for the changed areas; one approval from another developer is required
4. Address feedback in new commits (don't force-push)
5. Squash merge once CI is green and the PR is approved

Code Quality

Required Checks

These five CI jobs run on every PR into `main` and must all pass before merge:

Check (job)	Workflow	What it runs
`lint-and-test`	`ci.yml`	`pnpm lint`, `pnpm typecheck`, `pnpm test`, `expo install --check`, package build, Android `expo export`
`database`	`ci.yml`	Local Supabase (migrations and seed), `pnpm db:test` (pgTAP), `pnpm db:lint`
`types-drift`	`ci.yml`	`pnpm db:types`; fails if `packages/api/src/database.types.ts` changes
`secrets`	`ci.yml`	gitleaks, plus greps for JWTs, service-role key references and Anthropic keys in `apps/` and `packages/`
`deno-tests`	`edge-functions.yml`	`deno test`, `deno fmt --check`, `deno lint` on `supabase/functions`

Branch protection does not enforce them yet (see Branch Protection), so check the PR's status before you merge. Details: docs/DEVOPS-02-CI.md.

`pnpm format:check` is not one of the checks. It fails on `main` today (see docs/DEVOPS-02-CI.md).

Running Locally

Run the `lint-and-test` steps before you push:

```
pnpm lint
pnpm typecheck
pnpm test
```

The database checks need Docker. If you do not have it, CI runs them for you. The full list of local commands for every job is in docs/DEVOPS-02-CI.md.

Mobile Development

The app runs in a development build (dev client), not Expo Go. Setup, the daily routine and troubleshooting are in docs/LOCAL-DEVELOPMENT.md.

Two rules that affect PRs:

- `expo run:android` generates `apps/mobile/android/` and adds `android` and `ios` scripts to `apps/mobile/package.json`. These are not git-ignored yet. Do not commit them.
- Keep test files out of `apps/mobile/app/`. Expo Router turns every file there into a route, and a test bundled as a screen crashes the app.

Data Layer

The shared data layer lives in `packages/api` (`@sisig/api`). The mobile wiring is in `apps/mobile/src/lib`. docs/ARCHITECTURE.md explains the design. The rules reviewers check:

- **Config:** `apps/mobile/.env.local` sets `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_ANON_KEY` (the publishable key). Never commit it. Expo inlines `EXPO_PUBLIC_*` values into the bundle, so only public values may use that prefix. A service-role key must never reach the app; CI scans for it.
- **Client:** create Supabase clients only with `createSupabaseClient` (`@sisig/api`). The app's single client is `apps/mobile/src/lib/supabase.ts`.
- **Sessions:** stored only through `nativeSessionStorage` (AES-256-GCM, key in SecureStore). Never write tokens to AsyncStorage directly.
- **Logout:** `signOutAndPurge` (`apps/mobile/src/lib/logout.ts`) clears every per-user store. Add any new per-user store to it.
- **Query keys:** add every key to the `keys` factory in `packages/api/src/query-keys.ts`. Never inline key arrays.
- **Queries:** expose each query as `xxxQueryOptions(client)` plus a thin `useXxx` wrapper, as `useMenuCategories` does in `packages/api/src/hooks/useMenu.ts`. Use `createQueryClient()` for the app's QueryClient. In `apps/mobile`, pass the options to the app's own `useQuery` instead of calling a hook from `@sisig/api`: the package resolves a different React copy (18.3, the app uses 18.2), so its hooks fail there with an invalid hook call. `appConfigQueryOptions` (PLAT-06) is options-only for this reason.
- **Errors:** throw `normalizeError(error)` from queries and mutations, and show users only `error.message` from the `AppError` it returns.
- **Writes:** points, redemptions and orders change only through RPCs or Edge Functions, never by direct table writes. `useCreateOrder` in `packages/api/src/mutations.ts` predates this rule; do not copy its pattern.
- **Offline cache:** only keys on the `persistedQueryKeyPrefixes` allowlist (`packages/api/src/query-keys.ts`) are written to disk. Add a key only if its data is not personal. Mutations are never persisted.

Database Changes

Every schema change is a migration in `supabase/migrations/`, made with `pnpm db:new <name>` and merged through a normal PR. The rules for writing migrations are in supabase/CONVENTIONS.md.

1. **In the PR:** add the migration, pgTAP tests in `supabase/tests/database/` for what it changes, and the regenerated `packages/api/src/database.types.ts` (`pnpm db:types`, which needs the local stack and Docker). Fill in the template's Migration Impact section. CI's `database` and `types-drift` jobs apply every migration to a fresh local database and fail on test, lint or type differences.
2. **Destructive migrations** (dropping a table or column, deleting or rewriting rows): before merge, audit the data in the hosted dev project (for example, row counts for every affected table), back it up, and record the result in the PR. `supabase/migrations/20261106230037_retire_legacy_loyalty_tables.sql` (DATA-04) is an example.
3. **After merge:** CI does not deploy migrations. Someone with access to the hosted dev project runs `supabase db push` from an up-to-date `main` (preview first with `--dry-run`). Until then the hosted database is behind `main`.

The full procedure (login, link, dry run, push, backups) is in docs/SUPABASE.md.

TypeScript Strict Mode

Every TypeScript workspace extends `packages/config/tsconfig.json`, which sets `strict: true`. No `any` types: use proper typing.

Branch Protection

These are GitHub repository settings, not files in this repo. **None of them is configured yet:** as of 2026-11-06, `gh api repos/Alexd2004/Sisig/branches/main/protection` returns 404 "Branch not protected", and the repo has no rulesets. Until a repo admin sets them, these rules are team discipline, not enforcement.

The intended settings, under Settings > Branches for `main` (and for `release/*` lines once they exist):

- PR required for all changes
- At least one approval from another developer; `.github/CODEOWNERS` routes review requests by area
- The five required checks above must pass (see docs/DEVOPS-02-CI.md)
- No direct pushes
- No force pushes

Squash-only merging is a separate setting under Settings > General > Pull Requests. Today merge commits and rebase merging are still allowed there, so pick **Squash and merge** by hand. "Automatically delete head branches" is off, so delete your branch after merge.

Onboarding

New developers should:

1. Follow docs/LOCAL-DEVELOPMENT.md to set up a Mac and run the app on an Android emulator
2. Read this file
3. Skim docs/README.md for the other guides, and read the ones for your area

Questions?

Start at docs/README.md, which lists every guide. `apps/admin` has no README yet; it is planned with ADM-01.

Sisig Mobile App

Source: apps/mobile/README.md

The Sisig Eh! customer app. It uses Expo SDK 51 (React Native 0.74, React 18.2), Expo Router 3 for file-based routing, Tamagui themes through `@sisig/ui`, Supabase through `@sisig/api`, and TypeScript strict mode.

The app runs as a **development build** (a "dev client": a debug build of the app that loads JavaScript from Metro, the React Native bundler). **Expo Go is not supported.**

For how the pieces fit together, with diagrams, see docs/ARCHITECTURE.md.

Quick Start

The team runs the app on an Android emulator (AVD `Sisig_Pixel_API_34`, API 34) against the hosted Supabase dev project. The full setup, including Android Studio and the emulator, is in docs/LOCAL-DEVELOPMENT.md. In short, from the repository root:

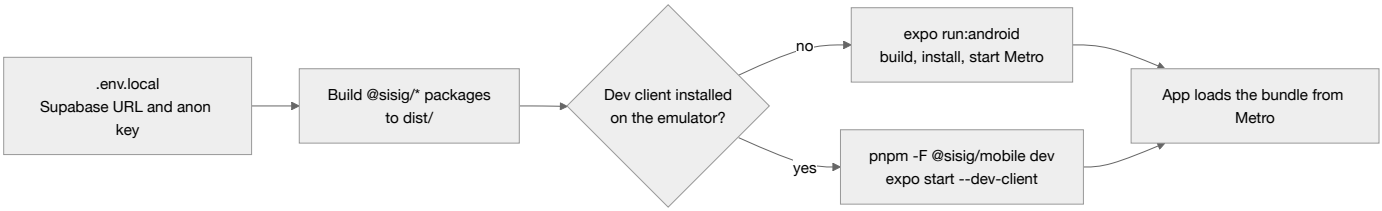
```
pnpm install

# 1. Create apps/mobile/.env.local (see "Environment Variables")

# 2. Build the shared packages. Metro reads them from their dist/ folders.
pnpm turbo run build --filter='@sisig/mobile^...'

# 3. First run: build and install the dev client, then start Metro
pnpm -F @sisig/mobile exec expo run:android

# Later runs: start Metro for the installed dev client
pnpm -F @sisig/mobile dev
```



Run the package build again after you change code under `packages/*/src` or pull changes to it. After you pull new dependencies, restart Metro with `--clear`.

Development Client

The `dev` script is `expo start --dev-client`, so Metro serves a development build, not Expo Go. `app.json` lists config plugins (`expo-router`, `expo-secure-store`, `expo-dev-client`, `expo-font`) and are applied when the native project is built. `app.json` also sets `newArchEnabled: true` under `ios` and `android`, but those keys have no effect on SDK 51: the generated `android/gradle.properties` has `newArchEnabled=false`. Turning the new architecture on in SDK 51 would need the `expo-build-properties` plugin.

Android

```
pnpm -F @sisig/mobile exec expo run:android
```

This generates `apps/mobile/android`, builds a debug app, installs it on the running emulator or a connected device, and starts Metro. See docs/LOCAL-DEVELOPMENT.md for the emulator setup.

iOS

Not set up yet. It needs a full Xcode install. Once Xcode is installed, the command is:

```
pnpm -F @sisig/mobile exec expo run:ios
```

Generated native folders

`expo run:android` and `expo run:ios` generate `apps/mobile/android` and `apps/mobile/ios`. They are local build output. Do not commit them.

When to rebuild the dev client

Rebuild with `expo run:android` after you add or upgrade a package with native code, or change native config in `app.json` (plugins, icons, bundle identifiers). JavaScript and TypeScript changes only need Metro to reload.

Release builds

Not configured. The `build` script runs `expo build`, which the local Expo CLI rejects, and there is no `eas.json`. Store builds through EAS are planned (DEVOPS-04).

Environment Variables

Create `apps/mobile/.env.local`. The root `.gitignore` ignores `.env.local`, so it is never committed.

```
EXPO_PUBLIC_SUPABASE_URL=https://<project-ref>.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=<publishable anon key>
```

- Expo reads the file when Metro starts and inlines every `EXPO_PUBLIC_*` value into the JavaScript bundle. Anyone with the app can read these values, so only public values may use the prefix. The anon key is

Supabase's publishable key. Never put a service-role key or any other secret here; CI scans client code for them.

- `src/lib/supabase.ts` throws at launch if either variable is missing: `EXPO_PUBLIC_SUPABASE_URL and EXPO_PUBLIC_SUPABASE_ANON_KEY must be set.`
- After you change the file, restart Metro. Add `--clear` if the old value still shows.
- `src/types/process-env.d.ts` declares the only two variables app code may read. A new variable must be added there too.

Project Structure

```
apps/mobile/
├── app/                        # Expo Router routes. Every file here is a route.
│   ├── _layout.tsx             # Providers and RootNavigator (splash, redirects, modals)
│   ├── index.tsx               # Redirects to /home
│   ├── (tabs)/                 # Bottom tabs: home, order, rewards, code, more
│   │   ├── _layout.tsx         #   Tabs, pop-to-root on active tab press
│   │   └── <tab>/              #   _layout.tsx (Stack) and index.tsx per tab
│   ├── (auth)/                 # welcome, login, sign-up, verify-email, reset-password
│   ├── redeem-confirm.tsx      # Modal (placeholder)
│   ├── item/[id].tsx           # Modal (placeholder)
│   └── ai-*.tsx                # ai-chat, ai-journal, ai-summarizer, ai-translator (UI only)
├── src/
│   ├── components/             # AppThemeProvider, SignInRequired, PlaceholderScreen, TabBarIcon,
│   │                           # ForcedUpdateScreen, UpdateBanner, AnnouncementBanner,
OrderingPausedCard
│   ├── config/                 # Remote config (PLAT-06): provider, useFeature, update gate, caches,
reporter
│   ├── hooks/                  # useAI, useImageAnalysis (not used by any screen)
│   ├── lib/                    # supabase, session-storage, session-lifecycle, logout, query-
persistence
│   ├── navigation/             # auth-gate, pop-to-root, stack-options
│   ├── session/                # SessionProvider, useSession, Supabase session source
│   ├── types/process-env.d.ts  # The EXPO_PUBLIC_* variables app code may read
│   └── no-raw-colors.test.ts   # Fails on hex colours in app/ or src/
├── assets/                     # Placeholder icon, adaptive icon, splash, favicon
├── app.json                    # Expo config: plugins, scheme "sisig", package ids
├── babel.config.js             # babel-preset-expo
├── jest.config.js              # jest-expo preset
└── tsconfig.json               # Extends @sisig/config/tsconfig
```

Tests sit next to the code they cover under `src/`. There is no `metro.config.js`; Expo's default Metro config handles the monorepo.

Routing

Files under `app/` become routes. Folders in parentheses are route groups and do not appear in the URL.

Route	File	Who can see it
`/`	`app/index.tsx`	Redirects to `/home`
`/home`, `/order`, `/more`	`app/(tabs)/<tab>/index.tsx`	Everyone
`/rewards`, `/code`	`app/(tabs)/<tab>/index.tsx`	Signed-in users. Guests see a sign-in prompt from `SignInRequired`.
`/welcome`, `/login`, `/sign-up`, `/verify-email`, `/reset-password`	`app/(auth)/*.tsx`	Everyone. A signed-in user is redirected out to `returnTo` or `/home`.
`/redeem-confirm`, `/item/[id]`	`app/redeem-confirm.tsx`, `app/item/[id].tsx`	Modals
`/ai-chat`, `/ai-journal`, `/ai-summarizer`, `/ai-translator`	`app/ai-*.tsx`	No screen links to them

The auth screens are placeholders until AUTH-01 to AUTH-03, so nobody can sign in from the app yet. When the session changes from signed in to signed out, the app clears every stack and opens `/welcome`.

Keep test files out of `app/`. Expo Router bundles them as routes, and the app crashes on the device with "Property 'jest' doesn't exist". PR #18 moved the index test to `src/navigation/index-redirect.test.tsx`, which imports `../../app/index`.

Session and Data Layer

Details and diagrams are in docs/ARCHITECTURE.md.

- **Supabase client.** `src/lib/supabase.ts` creates the only Supabase client and the QueryClient with `createSupabaseClient` and `createQueryClient` from `@sisig/api`. `attachSessionLifecycle` runs token auto-refresh only while the app is in the foreground.
- **Session storage.** `src/lib/session-storage.ts` encrypts the session with AES-256-GCM. The key is in the device keystore (`expo-secure-store`) and the ciphertext is in AsyncStorage. Never write tokens to AsyncStorage directly.
- **Session status.** `SessionProvider` (`src/session/session.tsx`) gives `useSession()`: `status` (`loading`, `signed-in`, `signed-out`) and `signOut()`. `createSupabaseSessionSource` (`src/session/supabase-source.ts`) restores the session with `getSession`, maps `onAuthStateChange` events to a status, and purges the device on every `SIGNED_OUT`.
- **Sign-out.** `signOutAndPurge` (`src/lib/logout.ts`) signs out locally, then clears the in-memory query cache, the encrypted session entries and key, and the persisted query cache. Add any new per-user store to this function.

- **Query cache on disk.** `src/lib/query-persistence.ts` persists only successful menu and rewards-catalogue queries, for 24 hours. Points, profile, orders and auth are never written to disk.
- **Data hooks.** Query keys, hooks, mutations and `normalizeError` live in `packages/api`. No screen uses the hooks yet; the screens show sample data.

Remote Config, Kill Switches and Forced Update

Details and a diagram are in docs/ARCHITECTURE.md. To change values, follow docs/runbooks/remote-config.md.

- `RemoteConfigProvider` (`src/config/remote-config.tsx`, mounted in `app/_layout.tsx`) reads `public.app_config` on launch, every 5 minutes and on every return to the foreground. It keeps the last good rows in AsyncStorage (`sisig.app-config`), so an offline launch uses them, and falls back to the bundled defaults on a first offline launch.
- **Gate a feature** with `useFeature('ordering')` or `useFeature('receiptScan')` (`src/config/useFeature.ts`). It returns `{ enabled, message }`. The Order tab hides its menu and checkout while ordering is off and shows `OrderingPausedCard` with the message.
- **Update gate.** Below `min_supported_version_<platform>`, the root layout shows `ForcedUpdateScreen` instead of the navigator. Below `recommended_version`, `UpdateBanner` appears on Home at most once a day. Neither ever triggers from the bundled defaults or a failed fetch.
- **Installed version.** `getInstalledVersion()` (`src/config/installed-version.ts`) reads `Constants.expoConfig.version` (the `version` in `app.json`), because `expo-application` is not installed and adding it needs a dev client rebuild. Bump `version` in `app.json` for every store release, or the gate compares the wrong number.
- **Reporting.** Malformed or unknown config values go to `reportConfigIssue` (`src/config/reporter.ts`): `console.warn` in development, nothing in release builds. PLAT-04 plugs Sentry in with `setConfigReporter`.
- **Testing a screen that reads the config.** Without a provider, `useRemoteConfig()` returns the bundled defaults. To test other values, wrap the screen in `RemoteConfigProvider` with a fake client, cache and `AppState`, as `src/config/remote-config.test.tsx` does.

Theme and UI Components

- Use `Text`, `TextInput` and `Button` from `@sisig/ui`, not the React Native components. `Text` and `TextInput` cap OS font scaling at 1.5x.
- Build styles from semantic colours: `const createStyles = (c: AppColors) => StyleSheet.create({ ... })`, then `useThemedStyles(createStyles)` in the component. Use `useAppColors()` when you need a single colour.
- Never write hex colours in `app/` or `src/`. `src/no-raw-colors.test.ts` fails if you do. Brand values live in `packages/ui/src/theme/brand.ts`.
- `AppThemeProvider` follows the OS light or dark setting. There is no in-app override.

Claude AI Screens

The four `app/ai-*.tsx` routes are UI only. They show fixed placeholder text and make no API call:

- `ai-chat.tsx` keeps the conversation in component state and a `ConversationManager` from `@sisig/claude`.
- `ai-summarizer.tsx` shows a placeholder summary with character counts and a compression percentage.
- `ai-translator.tsx` has source and target language pickers (from `SUPPORTED_LANGUAGES`) and a swap button.
- `ai-journal.tsx` keeps entries in component state with placeholder insights. Entries are lost when the screen unmounts.

`src/hooks/useAI.ts` creates a `ClaudeClient` on every render, and `src/hooks/useImageAnalysis.ts` returns a fixed message without calling the API. No screen uses either hook.

> **Warning:** `ClaudeClient` falls back to `process.env.ANTHROPIC_API_KEY`, which Expo does not inline into the bundle, so `new ClaudeClient()` throws in the app. Do not add an `EXPO_PUBLIC_ANTHROPIC_*` variable or pass a key to `useAI` to make it work: a key in the bundle can be extracted from the installed app.

See packages/claude/README.md for the package itself.

Testing

```
# Build the shared packages first: tests import @sisig/* from dist/
pnpm turbo run build --filter='@sisig/mobile^...'

pnpm -F @sisig/mobile test
```

Jest uses the `jest-expo` preset in a Node environment, matches `**/*.(test|spec).(ts|tsx)`, and allows 20 seconds per test. The tests cover:

- `src/session/`: `SessionProvider` and the Supabase session source (restore, subscribe, purge on `SIGNED_OUT`, single purge per sign-out).
- `src/lib/`: encrypted session storage, logout purge order and error handling, query persistence allowlist, foreground-only token refresh.
- `src/navigation/`: auth redirect rules, `returnTo` sanitising, pop-to-root, and the index redirect.
- `src/components/`: `AppThemeProvider` and `SignInRequired`.
- `src/no-raw-colors.test.ts`: no hex colours in app code.

Pure functions are the easiest to test. For example:

```
import { resolveAuthRedirect } from './auth-gate';

it('sends a signed-in user from the auth stack to Home', () => {
  expect(
```

```
    resolveAuthRedirect({ status: 'signed-in', segments: ['(auth)', 'login'], returnTo: undefined })
  ).toBe('/home');
});
```

Components that use theme colours must render inside `TamaguiProvider` and `Theme` from `@sisig/ui`. Components that call `useSession()` also need a `SessionProvider` with a fake `SessionSource` (`restore`, `subscribe`, `signOut`). See `src/components/SignInRequired.test.tsx`.

Linting & Type Checking

```
pnpm -F @sisig/mobile lint           # ESLint on app/ and src/
pnpm -F @sisig/mobile format         # Prettier, writes changes
pnpm -F @sisig/mobile format:check   # Prettier, check only
pnpm -F @sisig/mobile typecheck      # tsc --noEmit (strict); needs built packages
pnpm -F @sisig/mobile run doctor     # expo-doctor
pnpm -F @sisig/mobile exec expo install --check   # dependency versions match SDK 51 (CI runs this)
```

Use `run doctor`: plain `pnpm doctor` runs pnpm's own checker.

Key Dependencies

Area	Packages
App framework	`expo` 51, `expo-router` 3, `react-native` 0.74, `react` 18.2, `expo-dev-client` (dev)
Shared code	`@sisig/ui`, `@sisig/api`, `@sisig/claude`, `@sisig/config` (dev)
Session and storage	`@supabase/supabase-js` (through `@sisig/api`), `expo-secure-store`, `expo-crypto`, `@noble/ciphers`, `@react-native-async-storage/async-storage`
Data	`@tanstack/react-query`, `@tanstack/react-query-persist-client`, `@tanstack/query-async-storage-persister`
UI	`expo-font`, `@expo-google-fonts/inter`, `@expo/vector-icons`, `expo-splash-screen`, `expo-status-bar`
Tests	`jest`, `jest-expo`, `@testing-library/react-native`

`@sentry/react-native`, `sentry-expo`, `lottie-react-native`, `react-native-gesture-handler` and `expo-device` are declared in `package.json` but not imported anywhere yet.

Troubleshooting

"EXPO_PUBLIC_SUPABASE_URL and EXPO_PUBLIC_SUPABASE_ANON_KEY must be set"

Create `apps/mobile/.env.local` with both values, then restart Metro (`pnpm -F @sisig/mobile exec expo start --dev-client --clear`).

Metro cannot resolve `@sisig/...`, or shared code looks stale

Build the shared packages, then restart Metro with a clear cache:

```
pnpm turbo run build --filter='@sisig/mobile^...'
pnpm -F @sisig/mobile exec expo start --dev-client --clear
```

"Property 'jest' doesn't exist" on launch

A test file is under `app/`. Move it to `src/`.

A native module is missing after adding a dependency

The installed dev client predates the dependency. Rebuild it with `pnpm -F @sisig/mobile exec expo run:android`.

Tests cannot import `@sisig/*`, or behave oddly

Build the shared packages first. To clear the Jest cache:

```
pnpm -F @sisig/mobile exec jest --clearCache
```

Further Reading

- docs/ARCHITECTURE.md: architecture and diagrams
- docs/LOCAL-DEVELOPMENT.md: local setup and the Android emulator
- docs/SUPABASE.md: the Supabase guide
- CONTRIBUTING.md: branching, commits and the data layer rules
- Expo documentation and Expo Router

@sisig/claude

Source: packages/claude/README.md

Shared Claude API integration package for the Sisig platform. It wraps the Anthropic TypeScript SDK (`@anthropic-ai/sdk` `^0.24`, locked at 0.24.3) and exports:

- `ClaudeClient` and `createClaudeClient`: chat, text generation and streaming.
- `ConversationManager`: in-memory message history with a size limit.
- Utilities: `parseJSON`, `truncateText`, `createSystemPrompt`.
- Helpers built on `ClaudeClient`: translation, code analysis and documentation generation.

`tsc` builds the package to `dist/`, and other workspaces import it from there.

> **Warning: keep the API key out of the mobile app.** `ClaudeClient` is meant for Node processes such as scripts or a server. In the Expo app, `process.env.ANTHROPIC_API_KEY` is not available (Expo inlines only `EXPO_PUBLIC_*` variables), so `new ClaudeClient()` throws. Do not "fix" that with an `EXPO_PUBLIC_ANTHROPIC_*` variable or by passing a key from app code: a key in the JavaScript bundle can be extracted from the installed app. No screen calls the Claude API today.

Known Issues

These are problems in the current code. This README describes them; it does not fix them.

- **The default model is retired.** `chat` and `streamChat` default to `claude-3-5-sonnet-20241022`, which Anthropic's model deprecation docs list as retired on October 28, 2025. A call that does not pass `options.model` fails. The translation, code-analysis and documentation helpers never pass a model, so they always use the retired default.
- **Every request sends** `temperature` (default `0.7`). According to Anthropic's docs, Claude Opus 4.7 and later and the 5.x models reject a non-default `temperature` with a 400 error.
- **The SDK is old.** `^0.24` resolves to 0.24.3, which sends API version `2023-06-01`. Anthropic's docs describe newer SDK releases (1.0 and later) that send a newer API version by default.
- **Helpers cannot take a key or a model.** Each helper creates its own `new ClaudeClient()`, so it reads the key from `process.env.ANTHROPIC_API_KEY` and uses the default model.
- **Text only.** `chat` and `streamChat` accept string message content only, so images and tool results cannot be sent.

Setup

1. Install and Build

```
pnpm install
pnpm -F @sisig/claude build
```

2. Provide an API Key (Node only)

Pass the key to the constructor, or set `ANTHROPIC_API_KEY` in the environment of the Node process:

```
export ANTHROPIC_API_KEY=sk-ant-...
```

The package does not load `.env` or `.env.local` files. A key written only to `.env.local` is not picked up.

Usage

The examples show the API shape. Because of the known issues, they fail against the live API unless you pass a `model` that is still available and accepts `temperature: 0.7`. Check the models overview for current IDs.

Basic Chat

```
import { ClaudeClient } from '@sisig/claude';

const client = new ClaudeClient(); // reads ANTHROPIC_API_KEY

const response = await client.chat(
  [{ role: 'user', content: 'Hello, Claude!' }],
  { model: '<model id>' },
);

const text = response.content.find((block) => block.type === 'text')?.text;
console.log(text);
```

Generate Text

```
const text = await client.generateText('Write a haiku about monorepos', {
  model: '<model id>',
  maxTokens: 500,
});
```

Streaming Chat

```
await client.streamChat(
  [{ role: 'user', content: 'Tell me about AI' }],
  (chunk) => {
    process.stdout.write(chunk);
  },
  { model: '<model id>' },
);
```

Conversation Manager

Keeps message history in memory and drops the oldest messages past the limit:

```
import { ConversationManager } from '@sisig/claude';

const manager = new ConversationManager(20); // 20 is also the default

manager.addMessage('user', 'What is AI?');
manager.addMessage('assistant', 'AI is artificial intelligence...');

const messages = manager.getMessages(); // { role, content }[] for chat()
const history = manager.getHistory(); // the same messages with timestamps

manager.clear();
```

In the Mobile App

- `apps/mobile/app/ai-chat.tsx` uses `ConversationManager` for local history and replies with fixed placeholder text.
- `apps/mobile/app/ai-translator.tsx` uses `SUPPORTED_LANGUAGES` for its pickers and shows placeholder text.
- `apps/mobile/src/hooks/useAI.ts` wraps `ClaudeClient` in a hook with `generate`, `chat` and `reset`. No screen uses it. It creates a new `ClaudeClient` on every render, so without a key it throws during render.

API Reference

ClaudeClient

Constructor

```
constructor(apiKey?: string)
```

Uses `apiKey`, then `process.env.ANTHROPIC_API_KEY`. If neither is set, it throws `ANTHROPIC_API_KEY environment variable is required`. `createClaudeClient(apiKey?)` does the same.

Methods

`chat(messages, options?)` returns `Promise<MessageResponse>`, which has `id`, `content` (an array of `ContentBlock`) and `stopReason`.

```
interface CreateMessageOptions {
  model?: string; // default: 'claude-3-5-sonnet-20241022' (retired)
  maxTokens?: number; // default: 1024
  temperature?: number; // default: 0.7
```

```
  system?: string; // system prompt
}
```

`generateText(prompt, options?)` sends one user message and returns the first text block as a string. It throws `No text content in response` if there is none.

`streamChat(messages, onChunk, options?)` streams the response and calls `onChunk` with each text delta.

`messages` is an array of `{ role: 'user' | 'assistant'; content: string }`. The SDK retries 408, 409, 429 and 5xx responses and connection errors twice by default; `ClaudeClient` adds no retry logic of its own.

ConversationManager

`new ConversationManager(maxMessages = 20)` with `addMessage(role, content)`, `getMessages()`, `getHistory()` and `clear()`.

Utilities

- `parseJSON<T>(jsonString)` is async and returns `{ success: true, data }` or `{ success: false, error }`.
- `truncateText(text, maxLength)` returns the text unchanged if it fits, otherwise cuts it and adds `...` within `maxLength`. `truncateText('Long text here...', 10)` returns `'Long te...'`.
- `createSystemPrompt(context)` returns `'You are a helpful AI assistant. '` followed by `context`.

Helpers

All helpers use `new ClaudeClient()` internally (see Known Issues).

Module	Exports
`translator.ts`	`translateText(text, sourceLanguage = 'English', targetLanguage = 'Spanish')` returns a `TranslationResult`. `detectLanguage(text)` returns the model's answer cast to `Language`, unchecked. `translateToMultipleLanguages(text, source, targets)` sends one request per target language in parallel. `SUPPORTED_LANGUAGES`: English, Spanish, French, German, Japanese, Chinese, Korean, Portuguese, Russian, Arabic.
`code-analyzer.ts`	`analyzeCode(code, language = 'typescript')` returns a `CodeAnalysis` (`issues`, `improvements`, `summary`) parsed with the greedy pattern `/\{[\s\S]*\}/`, which spans from the first `{` to the last `}` in the reply, so a reply with two JSON objects fails to parse. `generateCodeDocumentation`, `suggestRefactoring` and `explainCode` take the same arguments and return a string.
`doc-generator.ts`	`generateAPIDocs(code, { style, includeExamples, includeTypeInfo })`, `generateREADME(description, features, setup)`, `generateTestCases(code, framework)`, `generateChangelog(currentVersion, previousVersion, changes)`, `generateContributingGuide(projectName, techStack)`. Each returns a string.

Exported types: `CreateMessageOptions`, `ContentBlock`, `MessageResponse`, `ParsedResponse`, `ConversationMessage`, `CodeAnalysis`, `Language`, `TranslationResult`, `DocGenerationOptions`.

Testing

```
pnpm -F @sisig/claude test
```

Jest runs with `ts-jest` in Node. The utility tests and the `SUPPORTED_LANGUAGES` tests run offline. The translator, code-analyzer and doc-generator tests call the real API when `ANTHROPIC_API_KEY` is set in your shell, which costs money. Without a key they pass by catching the expected error. To be sure no request is sent:

```
env -u ANTHROPIC_API_KEY pnpm -F @sisig/claude test
```

Models

The only model ID in the code is the retired default, `claude-3-5-sonnet-20241022`. The IDs this README used to suggest (`claude-opus-4-1`, `claude-3-sonnet-20240229`, `claude-3-haiku-20240307`) are retired too. See the models overview for current IDs.

API Keys

- Create keys in the Claude Console.
- Never commit a key. `.env` and `.env.local` are gitignored. CI runs gitleaks, and a separate `git grep` step in `.github/workflows/ci.yml` fails on a string that looks like an Anthropic key (`sk-ant-` followed by 20 or more characters) in `apps/` or `packages/`.
- Never put a key in the mobile app (see the warning at the top).

Troubleshooting

`ANTHROPIC_API_KEY environment variable is required`

No key was passed and `ANTHROPIC_API_KEY` is not set in the process environment. In Node, export it in the shell that runs the process. In the mobile app this is expected: the key is not, and must not be, in the bundle.

A 404 or "model not found" error

The default model is retired. Pass a current `model` in the options.

A 400 error about `temperature`

The model does not accept `temperature: 0.7`. `ClaudeClient` always sends it, so this needs a change in `src/client.ts`.

`ClaudeClient is not defined`

Use the named import:

```
// Correct
import { ClaudeClient } from '@sisig/claude';

// Wrong: the package has no default export
import ClaudeClient from '@sisig/claude';
```

Claude AI Skills Guide

Source: CLAUDE_SKILLS.md

This document lists the Claude AI features in the Sisig platform and what state each one is in on `main`.

Overview

The repository has **7 Claude-related features** across the mobile app and the `@sisig/claude` package. None of them calls the Claude API from the app today:

- The four AI screens in `apps/mobile/app/` are UI only and show fixed placeholder text.
- The helpers in `packages/claude/src/` call the API, but only from Node, and the known issues in that package (retired default model, `temperature` always sent) make live calls fail until `src/client.ts` changes.

`@sisig/claude` provides:

- **ClaudeClient**: chat, text generation and streaming over `@anthropic-ai/sdk` 0.24.3.
- **ConversationManager**: in-memory message history with a size limit.
- **Utilities**: JSON parsing, text truncation, a system prompt helper.

> **Warning:** do not put an Anthropic API key in the mobile app. Expo inlines `EXPO_PUBLIC_*` variables into the JavaScript bundle, and anyone with the installed app can extract them. `ClaudeClient` reads `process.env.ANTHROPIC_API_KEY`, which Expo does not inline, so it throws in the app. That is the safe outcome; keep it that way.

#	Feature	Where	Status on `main`
1	AI Chat	`apps/mobile/app/ai-chat.tsx`	UI only, placeholder replies
2	Text Summarizer	`apps/mobile/app/ai-summarizer.tsx`	UI only, placeholder summary
3	Code Analyzer	`packages/claude/src/code-analyzer.ts`	Node helpers, affected by the known issues
4	Image Analysis	`apps/mobile/src/hooks/useImageAnalysis.ts`	Stub, never calls the API, not used by any screen
5	Translator	`packages/claude/src/translator.ts`, `apps/mobile/app/ai-translator.tsx`	Node helpers; the screen is UI only
6	Documentation Generator	`packages/claude/src/doc-generator.ts`	Node helpers, affected by the known issues
7	AI Journal	`apps/mobile/app/ai-journal.tsx`	UI only, placeholder insights

No screen links to the four AI routes (`/ai-chat`, `/ai-summarizer`, `/ai-translator`, `/ai-journal`).

Skills Breakdown

1. AI Chat (`apps/mobile/app/ai-chat.tsx`)

Purpose: multi-turn conversation with Claude.

What it does today:

- Shows a message list, a text input and a Send button.
- Adds each message to component state and to a `ConversationManager`.
- Replies with a fixed placeholder message. There is no API call and no streaming.
- Messages live in memory only and are lost when the screen unmounts.

Code it would use: `useAI` from `apps/mobile/src/hooks/useAI.ts` exposes `chat(messages)`, but the screen does not use it, and it cannot run in the app without a key in the bundle (see the warning above).

---

2. Text Summarizer (`apps/mobile/app/ai-summarizer.tsx`)

Purpose: condense long text into key points.

What it does today:

- Takes pasted text and shows a fixed placeholder summary.
- Shows the original length, the summary length and a compression percentage.
- Makes no API call.

Node equivalent:

```
import { ClaudeClient } from '@sisig/claude';

const client = new ClaudeClient();
const summary = await client.generateText(`Summarize: ${longText}`, {
  model: '<model id>', // the default model is retired
  maxTokens: 500,
});
```

---

3. Code Analyzer (`packages/claude/src/code-analyzer.ts`)

Purpose: review, document, refactor and explain code.

Exports:

```
import {
  analyzeCode,
  generateCodeDocumentation,
```

```
  suggestRefactoring,
  explainCode,
} from '@sisig/claude';

// Each takes (code, language = 'typescript')
const analysis = await analyzeCode(codeString, 'typescript'); // Promise<CodeAnalysis>
console.log(analysis.issues);
console.log(analysis.improvements);

const docs = await generateCodeDocumentation(codeString); // Promise<string>
const suggestions = await suggestRefactoring(codeString); // Promise<string>
const explanation = await explainCode(codeString); // Promise<string>
```

Returns:

```
interface CodeAnalysis {
  issues: string[];
  improvements: string[];
  summary: string;
}
```

`analyzeCode` asks for JSON and extracts it with the greedy pattern `/\{[\s\S]*\}/`, which spans from the first `{` to the last `}` in the reply, so a reply with two JSON objects fails to parse. Each function creates its own `ClaudeClient`, so it reads `ANTHROPIC_API_KEY` from the environment and uses the retired default model.

---

4. Image Analysis (`apps/mobile/src/hooks/useImageAnalysis.ts`)

Purpose: describe an image with a Claude vision model. Not implemented.

What the hook does today:

- `analyzeImage(uri)` fetches the URI and reads it as a data URL with `FileReader`.
- It then sets a fixed message, "Image analysis feature requires Claude vision model setup.", and never calls the API.
- It returns `description`, `loading`, `error`, `analyzeImage` and `reset`.
- No screen uses it, and the app has no camera or image picker dependency.

What a real version needs: `ClaudeClient.chat` only accepts string message content, so sending an image needs a change to `packages/claude/src/client.ts`. The key restriction above still applies, so the call would have to happen outside the app.

---

5. Translator (`packages/claude/src/translator.ts`)

Purpose: translate text between languages.

Supported languages (`SUPPORTED_LANGUAGES`): English, Spanish, French, German, Japanese, Chinese, Korean, Portuguese, Russian, Arabic.

Exports:

```
import {
  translateText,
  detectLanguage,
  translateToMultipleLanguages,
  SUPPORTED_LANGUAGES,
} from '@sisig/claude';

// Defaults: from English to Spanish
const result = await translateText('Hello world', 'English', 'Spanish');
console.log(result.translated);

// Returns the model's one-word answer, cast to Language without checking
const lang = await detectLanguage('Bonjour');

// One request per target language, sent in parallel
const results = await translateToMultipleLanguages('Hello', 'English', [
  'Spanish',
  'French',
  'German',
]);
```

Mobile UI (`apps/mobile/app/ai-translator.tsx`):

- "From" and "To" language pickers built from `SUPPORTED_LANGUAGES` .
- A swap button.
- Translate shows placeholder text that repeats the input. There is no API call.

---

6. Documentation Generator (`packages/claude/src/doc-generator.ts`)

Purpose: generate documentation text from code or a description.

Exports:

```
import {
  generateAPIDocs,
  generateREADME,
  generateTestCases,
  generateChangelog,
  generateContributingGuide,
```

```
} from '@sisig/claude';

// style: 'markdown' | 'html' | 'jsdoc' | 'sphinx'
const docs = await generateAPIDocs(code, {
  style: 'markdown',
  includeExamples: true,
  includeTypeInfo: true,
});

const readme = await generateREADME(
  'A simple calculator library',
  ['Add', 'Subtract', 'Multiply', 'Divide'],
  'npm install calculator',
);

// framework: 'jest' | 'mocha' | 'pytest'
const tests = await generateTestCases(code, 'jest');

const changelog = await generateChangelog('1.1.0', '1.0.0', ['Added feature X', 'Fixed bug Y']);

const guide = await generateContributingGuide('MyProject', ['TypeScript', 'React', 'Node.js']);
```

Each function returns a string and creates its own `ClaudeClient`.

---

7. AI Journal (`apps/mobile/app/ai-journal.tsx`)

Purpose: journaling with AI insights.

What it does today:

- Saves each entry with a timestamp to component state, newest first.
- Attaches the same fixed placeholder "insights" text to every entry. There is no emotion, theme or sentiment analysis.
- Entries are lost when the screen unmounts.

Integration Points

useAI Hook (React)

`apps/mobile/src/hooks/useAI.ts` returns `response`, `loading`, `error`, `generate(prompt)`, `chat(messages)` and `reset()`, and takes `{ apiKey?, systemPrompt? }`.

No screen uses it. It creates a `new ClaudeClient(options.apiKey)` on every render, so in the app it throws during render unless a key is passed, and passing a key from app code puts the key in the bundle. Treat the hook as an interface sketch, not something to wire up as it is.

ClaudeClient (Direct)

For Node scripts or a server:

```
import { ClaudeClient } from '@sisig/claude';

const client = new ClaudeClient(process.env.ANTHROPIC_API_KEY);

const response = await client.chat([{ role: 'user', content: 'Hello' }], {
  model: '<model id>',
  maxTokens: 500,
});

const text = await client.generateText('Write a poem', { model: '<model id>', maxTokens: 300 });

await client.streamChat(
  [{ role: 'user', content: 'Tell me a story' }],
  (chunk) => console.log(chunk),
  { model: '<model id>' },
);
```

Every request also sends `temperature` (default 0.7). According to Anthropic's docs, Claude Opus 4.7 and later and the 5.x models reject a non-default `temperature`. See the package README.

Conversation Manager

```
import { ConversationManager } from '@sisig/claude';

const manager = new ConversationManager(20); // max 20 messages, the default

manager.addMessage('user', 'What is AI?');
manager.addMessage('assistant', 'AI is...');

const messages = manager.getMessages(); // for chat()
const history = manager.getHistory(); // with timestamps

manager.clear();
```

Setup & Configuration

1. Set the API Key (Node only)

```
export ANTHROPIC_API_KEY=sk-ant-...
```

The package does not read `.env` or `.env.local` files, so the key must be in the environment of the Node process, or passed to the `ClaudeClient` constructor. Never put it in `apps/mobile/.env.local`.

2. Install & Build

```
# From the repository root
pnpm install
pnpm -F @sisig/claude build
```

3. Run the Mobile App

The app needs a development build; Expo Go is not supported. See apps/mobile/README.md and docs/LOCAL-DEVELOPMENT.md. The AI screens show placeholder text whether or not a key is set.

4. Run the Tests

```
pnpm -F @sisig/claude test
pnpm -F @sisig/mobile test
```

The translator, code-analyzer and doc-generator tests call the real API when `ANTHROPIC_API_KEY` is set in your shell. Run `env -u ANTHROPIC_API_KEY pnpm -F @sisig/claude test` to keep them offline.

Best Practices

1. API Key Security

```
// Good: a Node process reads the key from its environment
const client = new ClaudeClient(process.env.ANTHROPIC_API_KEY);

// Bad: a key written in source code
const client = new ClaudeClient('sk-ant-xxx');
```

- Never commit a key. CI runs gitleaks, and a separate `git grep` step in `.github/workflows/ci.yml` fails on a string that looks like an Anthropic key in `apps/` or `packages/`.
- Never ship a key in the mobile bundle, including through an `EXPO_PUBLIC_` variable.

2. Error Handling

```
try {
  const text = await client.generateText(prompt, { model: '<model id>' });
} catch (error) {
  if (error instanceof Error) {
    console.error('Claude API error:', error.message);
  }
}
```

3. Retries

The SDK already retries 408, 409, 429 and 5xx responses and connection errors twice by default. Do not wrap calls in a loop that also retries 400-level errors, which fail the same way every time.

4. Type Safety

The helpers are typed. `analyzeCode` already returns `Promise<CodeAnalysis>`, so no cast is needed:

```
const analysis = await analyzeCode(code);
analysis.issues.forEach((issue) => console.log(issue));
```

Performance Tips

1. **Cache results** you will reuse, such as translations or summaries.
2. **Parallel translations:** `translateToMultipleLanguages` sends one request per language at the same time. It does not use the Message Batches API.
3. **Stream long replies** with `streamChat`.
4. **Limit context:** `ConversationManager` keeps only the most recent messages.
5. **Set** `maxTokens` to control reply length. The default is 1024.

Not Built Yet

There are no Jira tickets for these in the repository:

- Real API calls from the AI screens, which need a server-side caller so the key stays out of the app.
- Image input (`useImageAnalysis` is a stub, and `chat` accepts text only).
- Streaming in the mobile UI.
- Fixing the known issues in `packages/claude/src/client.ts`.

Resources

- Claude API documentation
- Models overview
- Pricing
- @sisig/claude package
- Mobile app README

Questions? See the README files in individual packages or contact the dev team.